

Adaptive Difficulty Through Bayesian Opponent Modelling in ISMCTS-Based Card Game Agents

Author: José Pablo González Poblette
Student ID: 2507739
Degree: Master of Science in Computer Games Technology
Faculty: Faculty of Design, Informatics and Business
Abertay University
Date: August 2026

Abstract

Information Set Monte Carlo Tree Search (ISMCTS) is a version of Monte Carlo Tree Search (MCTS) designed to handle games with hidden information. This dissertation expands on this approach by replacing its standard uniform sampling of hidden states with an adaptive, belief-driven alternative based on Bayes' theorem. Instead of assigning equal probability to each possible hidden state, the proposed method incorporates the observable evidence revealed by the actions of an opponent to bias the determinization towards the hidden states that their observed play makes more reasonable. The study investigates whether this Bayesian approach affects the performance of the ISMCTS agent and how real players perceive the resulting opponent.

The approach will be implemented and evaluated using the card game Cambio. The proposed agent will be tested in two ways: first, against a baseline ISMCTS agent, and second, through playtesting with human participants to support more objective conclusions about its performance and player experience.

Keywords: Information Set Monte Carlo Tree Search (ISMCTS), Bayesian inference, opponent modelling, Determinization, imperfect-information games, dynamic difficulty adjustment, player perception, Cambio.

Contents

Abstract	i
List of Figures	vi
List of Tables	vi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Aim and Objectives	3
1.3 Research Question	3
1.4 Scope and Limitations	4
1.5 Dissertation Structure	4
2 Literature Review	6
2.1 Imperfect-Information Games and Belief Modelling	6
2.2 Search-Based Methods in Game AI	7
2.2.1 Monte Carlo Tree Search	7
2.2.2 Information Set MCTS (ISMCTS)	7
2.3 Counterfactual Regret Minimization	8
2.4 Opponent Modelling and Believable Agents	9
2.5 Dynamic Difficulty Adjustment	10
2.6 Probabilistic Reasoning and Bayesian Inference	10
2.7 Research Gap	11
3 Methodology	12
3.1 Research Approach	12
3.2 Formalization of Cambio	12
3.2.1 Game Rules and Objective	13
3.2.2 Card Values and Powers	13
3.2.3 State Representation and Action Space	13
3.2.4 Sources of Uncertainty	14
3.3 System Architecture	15

3.4	Evaluation Methodology	16
3.4.1	User Study Design	16
3.4.2	Behavioural Metrics	17
3.4.3	Computational Metrics	19
4	Design and Implementation	20
4.1	Architecture Overview and the Single Mutation Path	20
4.2	Card Model and Scoring	22
4.3	Determinization Support	22
4.4	The Baseline ISMCTS Agent	23
4.5	The Belief Layer	24
4.5.1	Representation	25
4.5.2	Updating Beliefs from Observed Effects	25
4.5.3	The Opponent-Evidence Model	26
4.6	Belief-Weighted Determinization	27
4.6.1	The Posterior Where Prior Meets Evidence	29
4.6.2	Sampling One World	29
4.7	The Cambio Guard	30
4.8	Unity Integration	32
4.9	Sources of Variability and Believability	32
4.10	Telemetry and Reporting	33
5	Evaluation and Results	34
5.1	AI analysis	34
5.2	User Study Results	46
5.3	Computational Performance	48
6	Discussion	50
7	Conclusion and Future Work	54
7.1	Conclusion	54
7.2	Future Work	55
7.2.1	Improving the Study Design	55
7.2.2	Communicating the Agent’s Actions	55
7.2.3	Making Mistakes Read as Human	56
7.2.4	Scaling to Larger Hidden-Information Games	56
	References	57
	Acknowledgment of AI Tools	59

Appendices	60
Appendix A Project Materials and Reproducibility	61
Appendix B Statistical Analysis Scripts	62
Appendix C Questionnaire	67
Appendix D Telemetry Column Dictionary	69
Appendix E Per-Participant Result Sheets	71

List of Figures

2.1	The four MCTS phases: selection, expansion, simulation,backpropagation.	8
3.1	Overview of the system architecture, tracing a single action from either participant through the game engine to its observable effects.	15
4.1	The belief-weighted determinization pipeline.	28
5.1	Agents win rate in mirror match	35
5.2	Agents win rate against human players	35
5.3	Agents hands average score in mirror match	36
5.4	Agents hands average score against human players	36
5.5	Agents score margin in mirror match	37
5.6	Agents score margin against human players	38
5.7	Agents swap quality against human players	38
5.8	Agents swap quality in mirror match	39
5.9	Agents penalty cards against human players	39
5.10	Agents penalty cards in mirror match	40
5.11	Agents hidden slots against human players	40
5.12	Agents match hit rates against human players	41
5.13	Agents match hit rates in mirror match	41
5.14	Agents cambio call turn in mirror match	42
5.15	Agents cambio call while being ahead ratio in mirror match	42
5.16	Agents cambio call while being ahead ratio against human players	43
5.17	Agents cambio guard calibration against human players	43
5.18	Agents belief distribution on hidden cards	44
5.19	Enjoyment ratings for playing against Ben	46
5.20	Enjoyment ratings for playing against Eva	46
5.21	Ratings of Ben’s adaptation to participants’ strategies	47
5.22	Ratings of Eva’s adaptation to participants’ strategies	47
5.23	Participants’ preference for the more enjoyable AI agent	48
5.24	Agents mean decision time per game	49
6.1	Play tester win rate	51

List of Tables

3.1	Card values and powers in Cambio.	13
5.1	Mann-Whitney U test results comparing the baseline and Bayesian agents in games against human players.	45

Chapter 1

Introduction

1.1 Background and Motivation

Artificial intelligence is widely used in video games, where one of its main purposes is to provide players with opponents to interact with and compete against. Early implementations, such as the ghost behaviours in Pac-Man, demonstrate how AI can enhance the player experience and create engaging gameplay. However, one challenge for game AI is predictability, as players become familiar with its behaviours over time. As the complexity of a game increases, the AI must also process a greater amount of information and consider a larger number of possible decisions, making it increasingly difficult to maintain challenging and engaging opponents.

Many algorithms have been developed to address this problem. Monte Carlo Tree Search (MCTS) uses a method for constructing a search tree by repeatedly simulating possible future actions. MCTS has been particularly successful in perfect-information games such as chess or Go, where the complete game state is visible to all players at all times. However, many games do not provide players with complete information about the current state. Consequently, the AI must make decisions while inferring information about the hidden state. These games are generally referred to as imperfect-information games.

In a card game, for example, a player's hand may be hidden from their opponent, meaning that multiple possible game states are consistent with the information currently available to each player. This creates an additional layer of uncertainty that does not exist in a standard perfect-information game. An AI cannot simply determine the exact current state and search forward from it; instead, it must consider a set of possible states that could represent the actual game state.

This issue is particularly relevant to the card game Cambio, which is the game environment considered in this research. In Cambio, players have hidden cards and must make decisions without knowing the complete contents of their opponents' hands. How-

ever, players can obtain information about these hidden cards through the progression of the game. Cards that have already been revealed or discarded reduce the set of cards that could still be hidden, while actions such as swapping cards, using powers, or making other decisions can provide additional information about an opponent’s possible hand. Therefore, the decisions made by a player can provide evidence about information that would otherwise remain hidden. This makes Cambio an appropriate environment for investigating whether an AI can use observations of player behaviour to adapt its representation of hidden information.

To address imperfect-information games, Cowling, Powley, and Whitehouse (2012) introduced Information Set Monte Carlo Tree Search (ISMCTS), extending the principles of MCTS to situations in which the complete game state is not known.

A key component of ISMCTS is the use of determinization. Before a simulation, the unknown parts of the game state are assigned values to produce one possible complete game state, after which the simulation can proceed using the principles of standard MCTS. In a conventional implementation, these determinizations are sampled uniformly from the states that are consistent with the information currently available to the player. In other words, if several possible hidden configurations exist, each is treated as equally likely when selecting a determinization.

Although this approach allows ISMCTS to perform fine in an imperfect-information environment, treating every consistent hidden state as equally probable is not necessarily representative of how uncertainty occurs during actual gameplay. In a card game, the information available to a player changes continuously as cards are revealed, discarded, swapped, or otherwise affected by player actions. Similarly, an opponent’s behaviour may provide additional evidence about the hidden state. For example, in Cambio, the cards an opponent chooses to exchange, the way they use available powers, and the actions they choose to take may provide information about the cards they are likely to hold. Two hidden states that are both technically possible may not be equally plausible given the evidence observed during the game.

This motivates the use of a belief over possible hidden states rather than treating all possible determinizations uniformly. A belief distribution can assign different probabilities to different hidden states according to the evidence available at a particular point in the game. As additional evidence is observed, these probabilities can be updated, allowing the representation of hidden information to adapt throughout the game.

Bayes’ theorem provides a mathematical foundation for updating such beliefs when new evidence becomes available.

The ability to adapt to opponents is particularly relevant when developing efficient and engaging game AI. Many card games rely heavily on interaction between multiple players, and the quality of the opponents can have a significant influence on the overall player experience. A static AI may provide an effective challenge initially but can become

predictable once players understand its decision-making patterns. Developing AI models that approach games in different ways may provide a way to create opponents that remain more engaging as players become familiar with them. At the same time, the computational efficiency of the approach remains important, since an AI must make decisions within the time and computational constraints of an interactive game.

This research therefore proposes the integration of an adaptive Bayesian layer with a standard ISMCTS agent. Rather than replacing ISMCTS, the proposed approach aims to modify the way the agent represents and samples uncertainty about hidden information. The Bayesian component can update beliefs about possible hidden states as evidence is observed throughout the game, while opponent modelling can be used separately to represent beliefs about the behaviour or strategy of the opponent. These beliefs can then inform the determinization process used by ISMCTS.

The research investigates whether integrating these ideas into an adaptive layer can produce a meaningful difference in the perceived quality and difficulty of an AI opponent in the card game Cambio.

1.2 Aim and Objectives

The objectives can be divided into three main categories:

1. Create a belief-weighted sampler to use in ISMCTS based on Bayes' theorem.
2. Test standard ISMCTS against my take on it, running them against each other and against actual human players.
3. Compare the AIs' results with two metrics: an objective one that validates their score and win rate, and a qualitative one gathered via a form given to the play testers.

The aim is to see whether adding this layer provides any value to the current ISMCTS or is not necessary. It could add value in the way of player perception, or as an objectively better model for this short game.

1.3 Research Question

Does integrating an adaptive Bayesian opponent-modelling layer into a static ISMCTS agent influence player-perceived opponent quality and difficulty in a card game compared to a static ISMCTS?

1.4 Scope and Limitations

This research expands the ISMCTS framework by adding an adaptive Bayesian layer that supplies dynamically updated beliefs to shape the determinizations of the underlying search responsible for decision-making. The goal is not optimal play or convergence to a Nash equilibrium. Since the agent acts on probabilistic beliefs rather than complete knowledge, its decisions will sometimes be effective and sometimes mistaken. This is an intentional design decision, as the research investigates whether an agent that adapts its beliefs to observed behaviour produces a more organic and less predictable opponent, rather than a stronger one.

Several factors limit the generalisability of the results. Cambio was chosen precisely because its small set of variables and actions makes the approach feasible to build and evaluate in the time available, and lets the study isolate the interaction between ISMCTS, Bayesian belief updating, and opponent modelling. That same simplicity limits generalisability: behaviour in Cambio may not carry over to games with larger state and action spaces or more sophisticated opponents.

The three-month development period is a further constraint, restricting how polished the interface can be and leaving the game reliant in part on text-based feedback. If the agent’s adaptation is not clearly conveyed, players may not perceive it, so a lack of difference in perceived opponent quality would not necessarily mean the belief layer failed to adapt internally, only that the adaptation was not visible. Finally, the sample consists of 20 participants across 400 games, which limits statistical power and generalisability. The findings should therefore be interpreted within the context of this specific game, implementation, and participant group.

1.5 Dissertation Structure

This dissertation is organised as follows. Chapter 2 reviews the relevant literature, covering imperfect-information games and belief modelling, the family of search-based methods that leads to ISMCTS, counterfactual regret minimisation, Bayesian inference, and dynamic difficulty adjustment. It then identifies the gap this work addresses. Chapter 3 sets out the research approach, formalises the card game Cambio as the experimental environment, describes the system architecture, and defines the evaluation methodology, including the user study design and the behavioural and computational metrics used to answer the research question.

Chapter 4 details the design and implementation of the system. It describes the game core and its single mutation path, the card model and scoring, the determinization support used by the search, and the ISMCTS agent itself, before presenting the Bayesian belief layer, the adaptive opponent-modelling layer, and the mechanisms introduced to

keep the agent unpredictable. Chapter 5 reports the results of the evaluation across three strands: computational performance, the user study, and an analysis of the behavioural weights learned during play. Chapter 6 interprets these results in relation to the research question and situates them against the reviewed literature. Chapter 7 concludes the dissertation and outlines directions for future work, including how the approach could be extended beyond the constraints of the current study. The questionnaire administered to participants is reproduced in Appendix C.

Chapter 2

Literature Review

This chapter reviews the work on which this dissertation builds. It begins by introducing imperfect-information games and the representation of hidden states through beliefs. It then traces the development of search-based methods leading to ISMCTS and contrasts these with counterfactual regret minimisation as an equilibrium-based alternative. Finally, it examines Bayesian inference as the foundation for updating beliefs and identifies the research gap addressed by this work.

2.1 Imperfect-Information Games and Belief Modelling

Chapter 1 introduced imperfect-information games as games in which part of the state is hidden, meaning that a player can identify only the set of states consistent with their observations rather than the single true state. This section examines how an agent can represent this uncertainty. A natural approach is to maintain a belief, which is a probability distribution over the possible hidden states. Each state is assigned a probability according to how well it agrees with the evidence observed so far, and these probabilities are revised as further evidence becomes available (Frank and Basin, 1998). Rather than committing to a single guess about the hidden state, the agent maintains a graded estimate that can become more precise or change as the game progresses.

This is different from opponent modelling. A hidden-state belief represents uncertainty about the underlying game state, such as the cards an opponent may hold. An opponent model by contrast represents beliefs about how an opponent behaves or makes decisions. The two concepts can be related because an opponent’s behaviour can provide evidence that changes the probability assigned to different hidden states. This relationship is where the present work is situated. The agent assumes a fixed model of rational opponent behaviour and uses observed actions as evidence to update its belief about the opponent’s hidden cards. It does not learn or adapt its model of the opponent’s strategy itself. The opponent model therefore acts as a mechanism for hidden-state inference rather than as a broader model of opponent behaviour of the kind described by, for example, Southey

et al. (2005).

2.2 Search-Based Methods in Game AI

2.2.1 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is a best-first search algorithm that builds an asymmetric tree incrementally by running many simulated playouts rather than exhaustively evaluating a fixed-depth frontier (Coulom, 2007). Each iteration consists of four phases: selection, expansion, simulation, backpropagation.

First in selection, the algorithm descends from the root through already-expanded nodes according to a tree policy until it reaches a node with unexpanded children. In expansion, one such child is added to the tree. In simulation, a playout is run from the new node to a terminal state using a default random policy, producing an outcome. Finally in backpropagation, that outcome is propagated back along the visited path, updating the visit count and accumulated value of every node on it. Over many iterations, the tree grows deeper in regions that repeated simulations suggest are promising, so the search effort is concentrated where it is most likely to matter.

The tree policy governs the balance between exploring under-sampled moves and exploiting moves that have performed well so far. The standard choice is the Upper Confidence Bound applied to Trees (UCT), which treats the selection at each node as a multi-armed bandit problem and selects the child maximising $\bar{X}_j + C\sqrt{\ln n/n_j}$, where $\bar{X}_j$ is the mean outcome of child j , n and n_j are the visit counts of the parent and child, and C controls the weight given to the exploration term (Kocsis and Szepesvári, 2006). This formulation gives MCTS an anytime property and removes the need for a hand-crafted evaluation function, which contributed to its success across a wide range of domains (Browne et al., 2012). However, standard MCTS assumes that the state at the root is fully known, since both the tree policy and the simulation phase operate on a single, completely specified game state. In an imperfect-information game this assumption does not hold, because the true state is not directly observable and many states are consistent with what the player can see. This limitation is what Information Set MCTS, described next, is designed to address.

2.2.2 Information Set MCTS (ISMCTS)

In imperfect-information games, an information set can be understood as a collection of game states that are indistinguishable to a player given the information available to them (Cowling, Powley, and Whitehouse, 2012). A player knows which information set they occupy, but cannot determine which specific state within that information set is the true

state. ISMCTS modifies the structure of standard MCTS so that nodes represent information sets rather than individual game states. This allows information from simulations corresponding to multiple possible states to be collected within a common search tree, making more efficient use of the available computational budget.

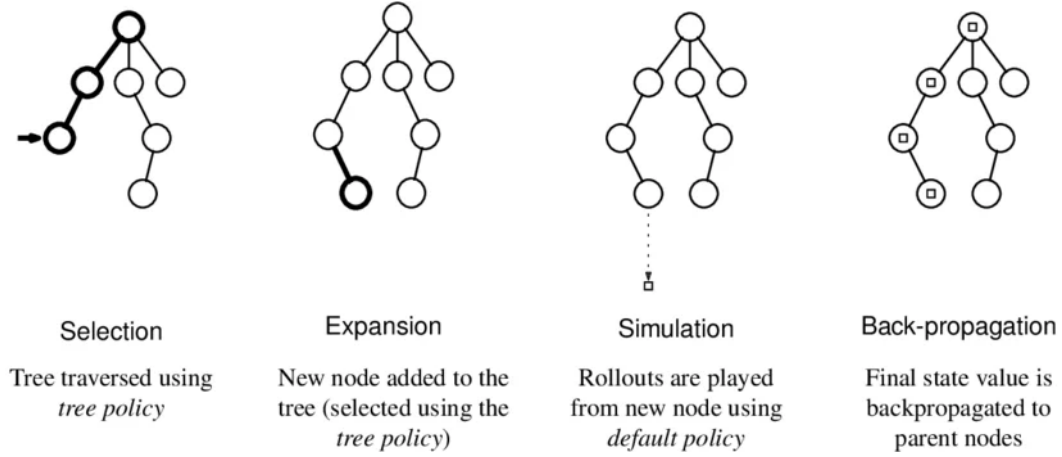


Figure 2.1: The four MCTS phases: selection, expansion, simulation, backpropagation.

2.3 Counterfactual Regret Minimization

Counterfactual Regret Minimisation (CFR) approaches imperfect-information games from a different direction than the search-based methods described above. Rather than building a search tree forward from the current state, CFR is an iterative self-play algorithm that repeatedly plays the entire game against itself and refines a strategy at every information set. On each iteration it records, for each action available at an information set, a measure of *regret*: how much better the player would have done, in retrospect, had it always chosen that action rather than the strategy it actually followed. Actions are then reweighted in proportion to their accumulated positive regret, so that choices which would have paid off are played more often in subsequent iterations. A central result is that although the strategy on any single iteration may be poor, the *average* strategy over many iterations converges towards a Nash equilibrium (Zinkevich et al., 2007).

A Nash equilibrium is a profile of strategies in which no player can improve their expected outcome by unilaterally changing their own strategy. In two-player zero-sum games this corresponds to an unexploitable strategy: one that guarantees a bounded loss against any opponent, however that opponent chooses to play. This property has made CFR and its successors the foundation of the strongest computer poker agents. Brown and Sandholm’s (2019) Pluribus, built on these ideas, was able to defeat professional human players at multiplayer no-limit Texas hold’em, a landmark result for imperfect-information game AI.

However, the objective that makes CFR powerful is also what limits its relevance to this project. CFR is designed to approximate an equilibrium, and an equilibrium strategy is by definition one that does not depend on the specific opponent it faces: it aims to be difficult to beat against every possible opponent rather than to exploit the tendencies of any particular one. Such a strategy is also essentially static once it has been computed, and computing it can be expensive, typically requiring a very large number of self-play iterations over the game’s information sets. The goal of this research is not an unexploitable opponent but an adaptive and believable one — an agent that responds to how a specific human is playing and remains engaging rather than mathematically optimal. CFR therefore provides a useful conceptual contrast: it defines what optimal play means in an imperfect-information game, while showing why optimality is not the same as adaptivity and why an equilibrium solver is not an appropriate basis for the opponent this project aims to build.

Accordingly, counterfactual regret minimisation is reviewed here for the perspective it provides on optimality in imperfect-information games rather than as a technique this dissertation implements; equilibrium solving falls outside the scope of this work, and is included as one of several approaches surveyed in order to situate the design that follows.

2.4 Opponent Modelling and Believable Agents

The methods described so far aim to achieve strong play. The objective of opponent modelling is for an agent to observe the particular opponent it faces and adapt its behaviour accordingly, rather than attempting to play well against every possible adversary. Bakkes, Spronck, and Herik (2009) use case-based opponent models to support adaptive game AI, retrieving behaviour that has previously performed well against similar opponents and applying it to the current one. In imperfect-information games, opponent modelling and hidden-state inference can overlap because an opponent’s observable actions may provide evidence about information that would otherwise remain hidden. Southey et al.’s (2005) Bayesian treatment of poker, discussed further in the next section, provides a representative example.

A related body of work questions whether the strongest possible opponent is necessarily the most desirable. Arrabales, Ledezma, and Sanchis (2009) argue for agents whose behaviour appears reasoned and human-like rather than strictly optimal, while Soni and Hingston (2008) found that bots trained to behave like human players were rated as more enjoyable than stronger but more mechanical opponents. These findings suggest that perceived quality and believability, including plausible mistakes, may have a greater influence on player experience than pursuing a perfect-play opponent. This is directly relevant to the present work, as the aim is not to create an unexploitable Cambio agent, but an opponent who uses belief-driven decisions to produce a more engaging playing

experience, even if it makes mistakes. The user study therefore evaluates player perception alongside win rate rather than treating playing strength as the only measure of success.

2.5 Dynamic Difficulty Adjustment

Dynamic Difficulty Adjustment (DDA) is the practice of altering a game’s challenge at runtime in response to the individual player, rather than exposing a small number of fixed difficulty settings chosen in advance. Hunicke (2005) makes the case that adapting challenge to the player can sustain engagement more effectively than static tiers, provided the adaptation is not conspicuous enough to break immersion. Andrade et al. (2006) frame difficulty balancing explicitly around user satisfaction, adjusting the agent so that the game remains neither trivially easy nor discouraging, and Yannakakis and Hallam (2009) pursue the same goal by optimising agent behaviour against models of player satisfaction in real time.

DDA motivates this work. The belief layer follows the same premise, since reading opponents’ actions as evidence makes the agent’s play contingent on who it is facing. The distinction is that this project does not scale a difficulty parameter towards a target win rate or satisfaction signal; the adaptivity is a by-product of belief-driven inference, not an explicit difficulty controller.

2.6 Probabilistic Reasoning and Bayesian Inference

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)} \quad (2.1)$$

where:

- H is a hypothesis, such as a possible hidden card configuration;
- E is the observed evidence, such as an opponent’s action during the game;
- $P(H)$ is the prior probability assigned to the hypothesis before the evidence is observed;
- $P(E | H)$ is the likelihood of observing the evidence if the hypothesis is true;
- $P(E)$ is the marginal probability of the evidence, which normalises the distribution;
- $P(H | E)$ is the posterior probability of the hypothesis after incorporating the observed evidence.

In the context of this research, Bayes’ theorem therefore provides a theoretical basis for updating the probabilities assigned to possible hidden states as the game progresses.

As new evidence becomes available, the probability assigned to each possible hidden state can be adjusted according to how consistent that state is with the observed evidence. For example, Southey et al. (2005) demonstrated the use of Bayesian modelling to infer a posterior distribution over opponent strategies based on prior beliefs and observations of play.

2.7 Research Gap

In the original formulation of ISMCTS, determinizations are sampled from the information set uniformly at random, treating every consistent hidden state as equally likely (Cowling, Powley, and Whitehouse, 2012). The authors themselves later flagged this as a weakness and corrected it by injecting hand-authored domain knowledge rather than by inferring it from play (Whitehouse, Powley, and Cowling, 2011). Meanwhile, most subsequent work extends ISMCTS outward, testing it across different games and benchmarking its strength against other agents (Cowling, Ward, and Powley, 2012; Goodman, 2019), rather than asking whether biasing its sampling towards an inferred opponent changes how it is experienced.

This dissertation addresses that gap. It keeps the search fixed and replaces only the uniform sampling with a belief-weighted model inspired by Bayes’ theorem, using evidence gathered from the opponent’s play style.

Chapter 3

Methodology

3.1 Research Approach

This research uses a comparative study to answer the research question. Two agents are placed in identical conditions and differ in only one respect: the control is the static ISMCTS agent, and the treatment is the same agent with the adaptive Bayesian opponent-modelling layer added. Any observed difference in play or in player perception can be attributed to that layer since the rest of the logic is identical. For readability, the baseline ISMCTS agent is referred to as "Ben" and the Bayesian variant as "Eva" throughout the results and discussion.

Every participant plays against both agents, making the resulting data paired, and justifying the paired statistical tests used later. It is also a mixed-methods design, combining objective in-game telemetry (behaviour, score values, and computational metrics) with subjective questionnaire responses, so that what the agents actually do can be triangulated against how players perceive them. To keep the perception data unbiased, participants are never told which agent is which. This design is appropriate because the research question concerns perceived opponent quality and difficulty, which cannot be captured by self-play or objective strength alone. It therefore requires human ratings alongside the measured behaviour.

3.2 Formalization of Cambio

To make Cambio usable as an experimental environment for search, the game is formalised as an explicit state together with a single set of rules that mutate that state. The guiding principle is that there is exactly one authoritative representation of the game and that every actor (human player or AI agent) interacts with it through the same vocabulary of actions and the same mutation path.

3.2.1 Game Rules and Objective

Cambio is a turn-based card game of imperfect information that can be played by 2 to 6 players. For this project, only two-player games are considered. Each player begins with four hidden cards, and before the game starts, they may look at their bottom two cards and memorise them.

The objective is to finish the game with the lowest total score, based on the cards each player holds. On their turn, a player draws a card and either discards it or swaps it into one of their slots, discarding the card it replaces. Some cards may trigger a power when discarded. A player may also attempt to match the top card of the discard pile with one of their own cards; the match is successful when both cards share the same number or, in the case of a queen, jack, or king, the same symbol. If successful, the player removes the matching card from their hand.

At the start of a turn, a player may call “Cambio,” which commits them to their current hand and triggers one final round for the opponent before the game ends and scores are compared. The player with the lower total wins; equal totals result in a draw.

3.2.2 Card Values and Powers

The deck consists of a standard fifty-two-card deck plus two jokers, for a total of fifty-four cards. For Cambio, only a card’s rank and colour are significant; the specific suit is not. Scoring and powers follow the values in Table 3.1: number cards score their face value, jacks and queens score ten, black kings score ten, red kings score minus one, and jokers score zero. A card’s power is triggered only when it is discarded directly from the draw, not when it is swapped into a slot.

Table 3.1: Card values and powers in Cambio.

Rank	Value	Power
Ace-6 (1-6)	Face value (1-6)	None
7-8	7-8	Look at one own card
9-10	9-10	Look at one opponent card
Jack, Queen (11-12)	10	Blind swap (swap two cards unseen)
Black King (13)	10	Look-and-swap
Red King (13)	-1	None
Joker	0	None

3.2.3 State Representation and Action Space

The game state is represented as plain data. Each side contains an array of hand slots and an array of penalty slots, along with the draw pile, discard pile, and the bookkeeping required to manage turns. This includes whose turn it is, the current phase, the card

currently drawn, any power being resolved, and the endgame flags associated with a called Cambio. Each card is assigned an address in the format (x, x, x) , where the three values represent its side, zone, and index, respectively. This uniquely identifies a position in the layout without exposing the value of the card it contains.

Actions are represented as commands. Each command combines a command type with an optional slot reference, and the complete action vocabulary is fixed. It includes drawing from the deck, discarding the drawn card, swapping the drawn card into a slot, using a power on a slot, attempting a match, giving a card, confirming a trade, finishing a peek, and calling Cambio. Every legal action available to a participant must be expressed through one of these commands, which are determined entirely by the game's current state and phase.

3.2.4 Sources of Uncertainty

Cambio contains two distinct sources of hidden information. The first is the contents of slots that a player has not seen, including their own and the opponent's hand. The second is the order of the draw pile, which determines the cards that each player will draw in future turns.

There is one main difference between both sources that needs to be noted. The identities of hidden slots can be constrained by evidence gathered during play. Cards that have been revealed, discarded, matched, or moved by powers reduce the set of cards that a given slot could contain, while the opponent's choices can further change the likelihood of the remaining possibilities. The order of the draw pile, however, carries no equivalent evidence and is therefore treated as genuinely uninformed.

3.3 System Architecture

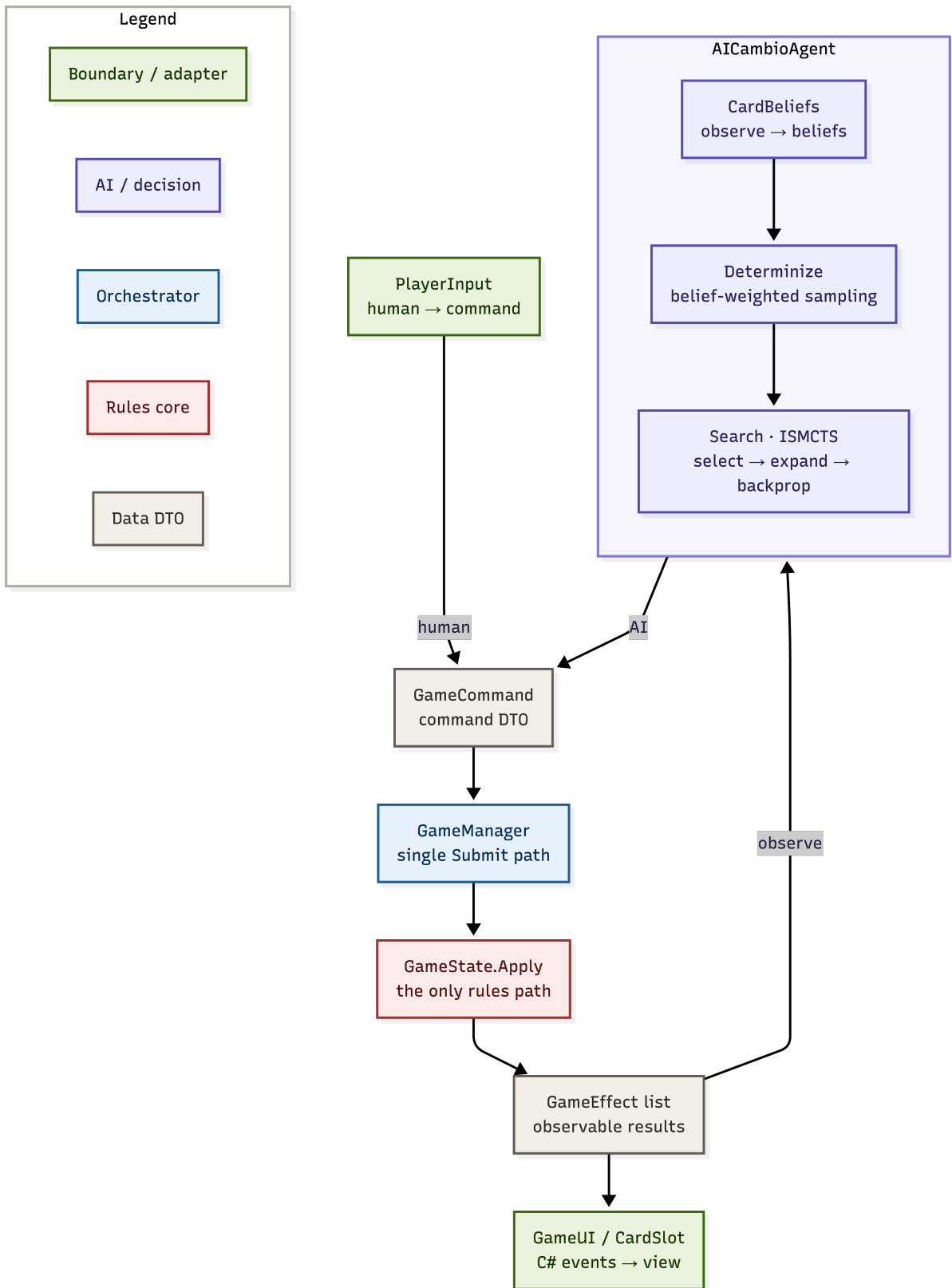


Figure 3.1: Overview of the system architecture, tracing a single action from either participant through the game engine to its observable effects.

Figure 3.1 provides an overview of the system architecture, focusing on how a single action flows from either participant through the game and produces observable results. The human player and AI agent are treated as interchangeable drivers of the same game engine. Each expresses its chosen action as a command, which is submitted through a single path and applied by the only code permitted to modify the game state. The resulting changes are then reported as a stream of observable effects that drive both the game view and the agent’s beliefs. Chapter 4 describes the implementation of this path and explains how it ensures live gameplay and the search operate under the same rules.

3.4 Evaluation Methodology

3.4.1 User Study Design

This project aims to recruit 20 participants to play-test both AIs. Each tester plays 20 games in total, yielding approximately 400 data samples, enough to draw an objective conclusion about the project.

The testing procedure gives each player complete freedom in choosing their first opponent. Because the players are never told which AI is the baseline ISMCTS and which is the Bayesian variant, this choice is effectively random. Preserving this blindness matters: it removes bias when players later report which AI they preferred, since revealing which one contains my contribution could make them more generous when rating it.

To measure how players perceive the AIs, a Google Forms questionnaire was designed to gather qualitative information about their gameplay experiences, which is then related to the objective behavioural metrics to reach a well-founded conclusion. The questionnaire first collects background information about each participant, including their familiarity with video games and card games, to assess how this may affect their perception of the AI. The same questions are asked about both AIs to ensure a fair comparison, followed by a question asking which AI the participant preferred or whether they remained neutral. The proposed questions are as follows:

Background.

1. Do you play video games?
2. Do you play card/board games?
3. Have you played card games similar to Golf, Skyjo or Poker?
4. Have you played Cambio before?
5. Please select your age group.

Questions 1 and 2 offer four options ranging from “Yes, regularly” to “No”; questions 3 and 4 are yes/no; and question 5 offers age groups from 18 to 50+.

AI evaluation (asked identically for each AI).

1. Did you enjoy playing against (AI Name)?
2. Do you feel (AI Name) adjusted its strategy based on how you were playing?
3. Did (AI Name) feel predictable at any moment during the match?
4. Did (AI Name) feel like it had a strategy?
5. Did (AI Name) respond differently when you used special cards such as peek or swap?
6. Do you feel (AI Name) took too long to play?

Question 5 has the options yes, no and neutral; the remaining questions use a four-point Likert scale of strongly disagree, disagree, agree and strongly agree.

Preference.

1. Which AI did you feel was more interesting and fun to play against?

The options are either AI or neutral.

The responses are analysed using a Wilcoxon signed-rank test because the data are paired, with each participant rating both AIs, and ordinal, as they are collected using a Likert scale. The data are also not assumed to be normally distributed, making a non-parametric paired test appropriate. The same rationale is applied to the paired tests used through out the evaluation. The complete questionnaire is provided in the appendix.

3.4.2 Behavioural Metrics

The behavioural metrics capture how the AIs approach the game. They provide the objective side of the evaluation, with an in-game telemetry layer recording every relevant AI action in a set of CSV files. This makes it possible to assess which agent performs better independently of player perceptions and then compare the behavioural results with the questionnaire responses.

Only the AI’s own actions are logged. The human opponent is treated as a confounding factor rather than a subject of analysis, preventing differences between individual human players from affecting the comparison between the two agents. The telemetry is stored across three CSV tables, all of which can be joined using ‘(session_id, match_index)’.

The telemetry is organised into three tables. The `cambio_matches` table holds one row per game: the outcome (winner and both scores) together with the AI’s behaviour that match, including how often it swaps versus discards, the average value change of its swaps, its matching accuracy and penalties, its use of power cards, its mean number of still-hidden own cards, and its mean decision time. The `cambio_calls` table holds one row per game in which Cambio was called, recording who called, on which turn, the scores at that instant, whether the caller was actually ahead, and the believed-score variable each guard used to justify the call. The `cambio_beliefs` table holds one row per hidden card per decision and is the data produced by the Bayesian layer: for each uncertain card it logs the belief-driven shift the layer applies to that card’s expected value (the *tilt*) against the card’s ground-truth value, so that the calibration of the layer can be measured. Because the tilt is identically zero when the Bayesian layer is disabled, these rows act as a null control that isolates exactly what the layer adds. All three tables share the session and match keys that let them be joined, and the complete column dictionary is given in Appendix D.

Cambio-call table (`cambio_calls`) This justifies each agent’s decision to end the game on its own terms. It records who called and on which turn (`cambio_caller`, `cambio_caller_turn`), the scores at that instant, and whether the caller was actually ahead (`cambio_caller_was_ahead`), that is, whether the call was correct. It also stores the variable each guard used to justify the call: the baseline’s flat believed-own-score (`cambio_ai_believed_score`), and, for the Bayesian agent, the believed own and opponent end-score means together with the probability of finishing ahead (`guard_believed_own_mean`, `guard_believed_opp_mean`, `guard_p_ahead`).

Belief table (`cambio_beliefs`), one row per hidden card per AI decision. This is the additional data produced by the Bayesian layer, and it validates the per-card value estimates that the layer feeds into the search. For every card the AI is uncertain about, it logs whether the card belongs to the opponent (`is_opponent_slot`), whether the opponent is believed to know it (`opp_knows`), the card’s ground-truth value (`true_value`), and two *tilt* values: `tilt_raw`, the shift the belief layer applies to that card’s expected value relative to a flat deck prior, and `tilt_eff`, the shift the search actually consumed. Comparing the tilt against `true_value` measures the calibration of the layer: a card believed to be low should, on average, actually be low. Crucially, when the Bayesian layer is disabled the tilt is identically zero, so these rows act as a null control and let me quantify exactly what the layer adds.

Statistical testing. The behavioural data are compared with a Mann-Whitney U test. Although each participant faces both agents, the two agents were evaluated over inde-

pendent sets of games of unequal size, so the observations cannot be paired at the game level. The belief-calibration metrics (the raw and effective tilt against ground truth) are tested the same way, with the zero-tilt rows produced when the Bayesian layer is disabled acting as the null control described above.

3.4.3 Computational Metrics

Whether the AI is computationally optimal reduces, for the purposes of this project, to its play time and the speed of the tree search. The design target is a very low hardware requirement so that the game runs on as many laptops as possible, and the telemetry captures this directly through `ai_decision_ms_avg`, the mean wall-clock time the search spends per decision. The search reporter additionally exposes the number of iterations completed against the target and the number of nodes expanded, which help explain that timing.

Because the Bayesian layer adds the belief update and the distribution-based Cambio guard on top of the search, comparing `ai_decision_ms_avg` between the two agents also quantifies the runtime overhead that my contribution introduces; the same paired Wilcoxon test applies here.

Frames per second and other real-time benchmark metrics are irrelevant to this study, because the game is turn-based, with the small exception of the brief real-time window in which the AI may attempt a match during the opponent’s turn.

Chapter 4

Design and Implementation

This chapter describes how the system was designed and built. To explain it better, I organised it into three layers that mirror the experimental design of the study. The first layer, covered in Sections 4.1–4.3, is the shared foundation used by both agents: the game core and its single mutation path, the card model and scoring, and the determinization support that lets a hidden state be sampled and searched.

The second layer, in Section 4.4, is the baseline ISMCTS agent, which works as a baseline for the Bayesian layer and can also operate independently. The third layer, in Sections 4.5–4.7, is the contribution of this dissertation, which is split as follows: the belief layer, the belief-weighted determinization that feeds those beliefs into the search, and the distribution-based Cambio guard. The chapter then covers how the agent is integrated into Unity, how the variety and unpredictability the study probes actually arise, and how the system is instrumented.

The important point to keep in mind throughout is that the control and the contribution are not two separate agents but the same engine with the belief machinery switched off or on. In the code, this is controlled by a single toggle. When false, the agent samples hidden states uniformly and calls Cambio on a flat absolute cap; when true, the same agent samples according to its beliefs and calls Cambio using a relative, distribution-based test (Bayesian layer). Presenting the baseline first and then layering the Bayesian components on top reflects exactly this relationship, and it is what makes any measured difference between the two attributable to the belief machinery alone.

4.1 Architecture Overview and the Single Mutation Path

This section explains how the formalisation of Cambio is implemented in code, in particular how actions are dispatched, what is permitted to modify the state, and how the results of those modifications reach the two different consumers, players and AI. The entire design architecture is built so there is only one way to modify the game state. Figure 3.1 traces the path of a single action, and the remainder of this section follows that path in

order.

The state itself is contained within **GameState**, which stores the layout as plain data, including the hand and penalty slots for both sides, the draw and discard piles, whose turn it is, the current **GamePhase**, the card currently drawn, the power being resolved, and the endgame flags associated with a called Cambio. All are private to prevent both the player and the AI from directly accessing the value of a hidden card; the data the outside world may need is exposed as public read-only.

Both the player and the AI interact with this same state through the same action vocabulary. Every possible action is represented as a **GameCommand**, which combines a **CommandType** (drawing, discarding, swapping the drawn card, using a power, attempting a match, giving a card, confirming a trade, finishing a peek, or calling Cambio) with an optional **SlotRef**. The slot reference identifies a target using its side, zone, and index, forming a numeric value in the form of “(X, X, X)”. The player and the AI produce these commands in different ways but ultimately use the same interface: the player through game clicks and the AI through the ISMCTS search. This makes them interchangeable drivers of the same game engine and ensures that both agents being compared, as well as the human opponent, are restricted to the same set of legal actions. **GameState.LegalMoves** examines the current phase and returns the valid commands, and this same set forms the branching factor enumerated by the search at each node.

Every command, regardless of its source, is submitted through **GameManager**, which is the only object that interacts with the live state, and is applied using the method **GameState.Apply**. This provides the single mutation path through the implementation and represents the most important structural decision in the system.

Apply dispatches each command to the appropriate internal handler, which first validates the command against the current phase before modifying the state. For example, a discard is rejected if no card has been drawn, while a Cambio call is rejected unless the game is in the drawing phase and a call has not already been made. If validation fails, the state remains unchanged. If validation succeeds, the handler applies the required changes and records what occurred.

This structure is important to highlight because the same implementation serves two different purposes. During live play, **Apply** advances the real game, while during search the AI clones the state and uses the same **Apply** method to advance simulated games. Because there is only one implementation of the game rules, a simulated Cambio behaves in the same way as a real game, with no separate simulation logic that could diverge from the live implementation as the code develops. This is key because the belief and opponent-modelling layers introduced later in this chapter rely on simulated outcomes, which are only meaningful when the simulation accurately represents the game being played.

Applying a command modifies the state and returns a list of **GameEffect** values that describe the observable consequences of the action. Each effect is a small record represent-

ing something that could have been observed during play, such as a card being drawn, a slot being revealed to a particular side, two slots being swapped, a match being resolved, etc. This effect stream provides the system’s single representation of observable events and is consumed by two separate audiences. The first is the game view. **GameManager** dispatches each effect as a C# event, which the Unity UI subscribes to and uses to update the presentation. The second audience is the agent. Its **CardBeliefs** component receives the same effects through its **Observe** method and preventing the belief layer from accessing information that the agent should not know. **CardBeliefs** never reads **GameState** directly and instead learns about the game exclusively through the effect stream, which represents the information available to an observer of the game. It only updates its belief about a revealed card when the relevant side was entitled to observe its value.

This completes the loop on which the remainder of the chapter is built. The agent produces a command, submits it through the same mechanism used by the human player, observes the effects, and updates its beliefs before selecting its next action. The uniform baseline and Bayesian variant differ only in how those beliefs are used when sampling hidden states.

4.2 Card Model and Scoring

A card is the smallest unit of state in the game; rather than storing rank, colour, value, and power as separate fields, a **Card** is represented by a single integer identifier, with all other properties derived from it arithmetically when required. This keeps cards inexpensive to copy across the thousands of determinizations the search clones, and lets the determinizer sample a single value per hidden slot rather than juggling several fields that could become inconsistent. The trade-off is that the decoding logic depends on the identifier layout remaining fixed: any change to how ranks, colours, jokers, and the empty-slot sentinel are packed into the identifier would require the derivation formulas to be updated accordingly.

4.3 Determinization Support

The way determinization allows an imperfect information game to be tackled as a perfect information one is that, before each simulation, the unknown parts of the state are assigned concrete values to produce a fully specified world that can then be searched as though all information were visible. This section describes the supporting functionality in **GameState** that produces such a world, while leaving aside the question of how the hidden values are selected. This separation is important because the uniform baseline and the belief-weighted sampler differ only in their sampling policies, while the surrounding determinization machinery remains identical.

The process begins with `Clone`, which creates a deep copy of the public state using a fresh seed derived from the iteration number, which ensures that successive determinizations can produce different worlds. All simulation then takes place on the cloned state, without altering the outside world.

`UnseenCardIds` provides the pool of cards available to fill the hidden slots. It contains every identifier that is not currently known, is not in the discard pile, and is not the card currently being held as the drawn card. The hidden slots are then assigned concrete cards through `OverwriteHidden`, while the remaining cards are placed into the future draw order through `SetDrawPile`. This preserves the distinction established in Chapter 3, where the sampling policy is applied to hidden slots while the draw pile is treated as carrying no evidence.

The method `IsCardSetWorking` provides a consistency check by verifying that every identifier appears exactly once across all slots, both piles, and the currently drawn card. If the resulting world fails this validation, the agent can discard it rather than allowing an impossible state to enter the search, reducing the amount of noise in the search.

4.4 The Baseline ISMCTS Agent

The baseline agent is a standard ISMCTS implementation, and it is described here only to the extent required to understand the contribution that follows. Implementing ISMCTS is not itself a goal of this project. Instead, it provides the control condition against which the belief machinery in Sections 4.5 and 4.6 is evaluated. The purpose of this section is therefore to establish how the agent makes decisions when it has no beliefs about hidden information.

A single decision is produced by running many search iterations over a shared tree, with four thousand iterations used by default, before selecting the most-visited move at the root. This figure was chosen empirically during development. Once the Cambio Guard of Section 4.7 is applied, four thousand iterations proved sufficient for the agent to simulate enough games to reach a competent decision, while remaining low enough to keep a single decision within roughly 500 to 1500 milliseconds. This is comfortably fast for a turn in a card game, where the agent is not competing for a real-time frame budget. Keeping the iteration count low also allowed the game to be developed and tested quickly without waiting for the search. Each iteration begins by using the determinization functionality described in Section 4.3 to sample a concrete world consistent with the information available to the agent. The search descends through the tree using the selection policy until it reaches a node with an untried move, expands a new node, evaluates the resulting position, and propagates the evaluation back through the visited path. Since a different world can be sampled on each iteration, the shared tree accumulates statistics across multiple possible states rather than representing a single fixed board. This allows

the tree to represent an information set rather than a single fully known state.

Selection uses Upper Confidence Bound applied to Trees, balancing exploitation of moves that have performed well with exploration of moves that have been visited less frequently. Rewards are stored from the AI’s perspective throughout the tree. When the selected node belongs to the opponent, the exploitation term is inverted so that the opponent is modelled as attempting to minimise the AI’s reward rather than maximise it. This maintains the adversarial nature of the search without requiring separate reward statistics for each player.

Evaluation reflects what the agent prefers. Terminal positions receive a simple win, draw, or loss score. A non-terminal leaf, which is the more common case given the relatively short search horizon, is evaluated by combining two terms: how far ahead the agent is compared with the opponent, with the result passed through a tanh function so that a small advantage is rewarded while larger advantages gradually saturate and how low the agent’s own hand is relative to a target score, again using a saturating function plus a penalty for any “penalty cards” held by the agent. The reason to measure how low the agent’s hand is as an absolute number is to provide an additional incentive to continue reducing its own hand and not get satisfied with an average hand. Together, these terms favour positions in which the agent is both ahead of the opponent and objectively in a strong position.

With the Bayesian layer disabled, each unknown slot is filled by drawing uniformly from the unseen pool, treating every card consistent with the available information as equally likely. This is the standard ISMCTS determinization approach and the control condition against which the belief-weighted variant is measured.

4.5 The Belief Layer

Standard ISMCTS reasons about hidden information without ever committing to what that information is likely to be. Each search iteration guesses a complete state and searches it, with the guesses being drawn without any preference or likelihood. The belief layer is the first half of this dissertation’s contribution. Its purpose is to give the agent an explicit, continuously updated view of the cards it cannot see, so that these guesses can later be made in an informed way rather than blindly.

The layer maintains one of two states for every card in play. A slot is either known, meaning the agent has legitimately seen the card and therefore knows its value with certainty, or hidden, in which case the agent maintains a probabilistic belief about which value the slot is likely to contain. These beliefs are based only on information the agent is actually entitled to use, including cards it has seen through its own looks and powers, as well as inferences drawn from the opponent’s visible choices. The agent never accesses the true value of a hidden slot, so the belief layer models the game from the same limited

perspective available to a human player. The remainder of this section explains how these beliefs are represented, how they are updated in response to observable events, and how the opponent’s behaviour is incorporated as evidence. How these beliefs are subsequently used by the search is discussed in Section 4.6.

4.5.1 Representation

For each hidden slot, the layer maintains an accumulated log-likelihood vector over the twelve possible card values; card values range from -1 to 10, and these values are mapped to buckets using the index $v+1$. Each entry represents the extent to which the evidence gathered so far supports or contradicts the possibility that the slot contains a card of that value. The evidence is stored additively in log-space, allowing independent observations to be combined through addition rather than requiring probabilities to be multiplied directly.

For this representation to work, we need a neutral state which would be a vector containing only zeros, meaning it expresses no preference among the possible values and represents a slot for which no evidence has been gathered. When the layer is queried for a slot’s log-likelihood, it returns this flat vector unless evidence has changed it. An uninformed slot therefore contributes no additional preference of its own and instead relies entirely on the deck prior when the determinizer combines the two (Equation 4.1). The same handling applies to known slots.

4.5.2 Updating Beliefs from Observed Effects

Beliefs are updated by the effect stream. Each `GameEffect` produced by the game is passed to the belief layer together with a flag indicating whether the agent was the actor responsible for the action or not. The layer then updates itself using this information. This flag is important because it allows the layer to distinguish between information the agent is entitled to know and information that it may have observed without being allowed to see the underlying card, preventing cheating.

Let’s analyse when a card is revealed to see the flow of information. When a slot is revealed, the layer records the card as known when the agent was the side that acted. A reveal triggered by the opponent therefore does not give the agent knowledge of the card’s value, even though the agent can observe that the reveal occurred. Swaps also move knowledge together with the card. When two slots exchange their contents, any known information or belief associated with the cards moves to their new locations, allowing a card that has already been identified to remain known after being moved. When the drawn card is swapped into a slot, the handling depends on which side acted; in the case the agent performs the swap, it records the card that it placed; otherwise, the inserted card remains hidden, and the action instead provides behavioural evidence, as described below.

Matches are handled similarly. When a match succeeds and a card leaves a slot, the corresponding slot is cleared. When a match fails, the mismatched card is revealed to both sides and is therefore recorded as known. Across all of these cases, the same principle is maintained. Certainty is granted only when the agent has accessed the card’s value, while any evidence associated with a slot is discarded whenever the contents of that slot change.

4.5.3 The Opponent-Evidence Model

The true depth of this opponent model is that it also reads meaning into the opponent’s choices, treating each observable decision as evidence that shifts the belief about otherwise unseen cards. This is the mechanism by which the agent adapts to the particular player in front of it rather than assuming, introducing concepts of Dynamic Difficulty Adjustment (DDA). Three signals are modelled, each expressed as an additive shift to the relevant log-likelihood vectors, and each deliberately bounded so that no single observation can overwhelm the rest of the evidence.

The strongest is the keep signal. When the opponent draws a card and chooses to keep it, that choice is informative, and the model guesses that a rational player only keeps a drawn card if it improves on what they gave up. The displaced card’s value is public, so the player leans the hidden slot’s belief towards values below it, sharply for values well under the discarded one and gently for values close to it, using a sigmoid on the gap between the two, plus a small blanket lean-low reflecting that the mere act of keeping a draw is itself weak evidence the card is good for them.

The second signal comes from persistence. A slot the opponent is believed to know and has left untouched across several turns is probably one they want to keep, so the layer applies a gentle lean-low that grows with the number of turns the card has stayed untouched, up to a cap so that a long game does not let this accumulate without limit. The third signal analyses when the opponent discards a low card face-up; this is weak evidence that their hand as a whole is running low, so a small lean-low is applied across all of their slots at once, scaled by how far below a typical value the discard was and again held under a ceiling.

Each of these is only ever a lean, never a certainty. They bias which worlds the search spends its effort on without ever ruling a physically possible card out, and because they are read from the specific opponent’s specific choices, two different players who play the same position differently will push the agent’s beliefs in different directions. This is the concrete sense in which the agent answers to how a human is playing, which is the behaviour the research question sets out to test.

4.6 Belief-Weighted Determinization

This section presents the core of the contribution, the mechanism that connects the beliefs described in Section 4.5 to the ISMCTS search described in Section 4.4, moving from treating every possible hidden configuration as equally plausible to focusing its search on the worlds it has reason to believe are most likely to be true. The complete pipeline is shown in Figure 4.1.

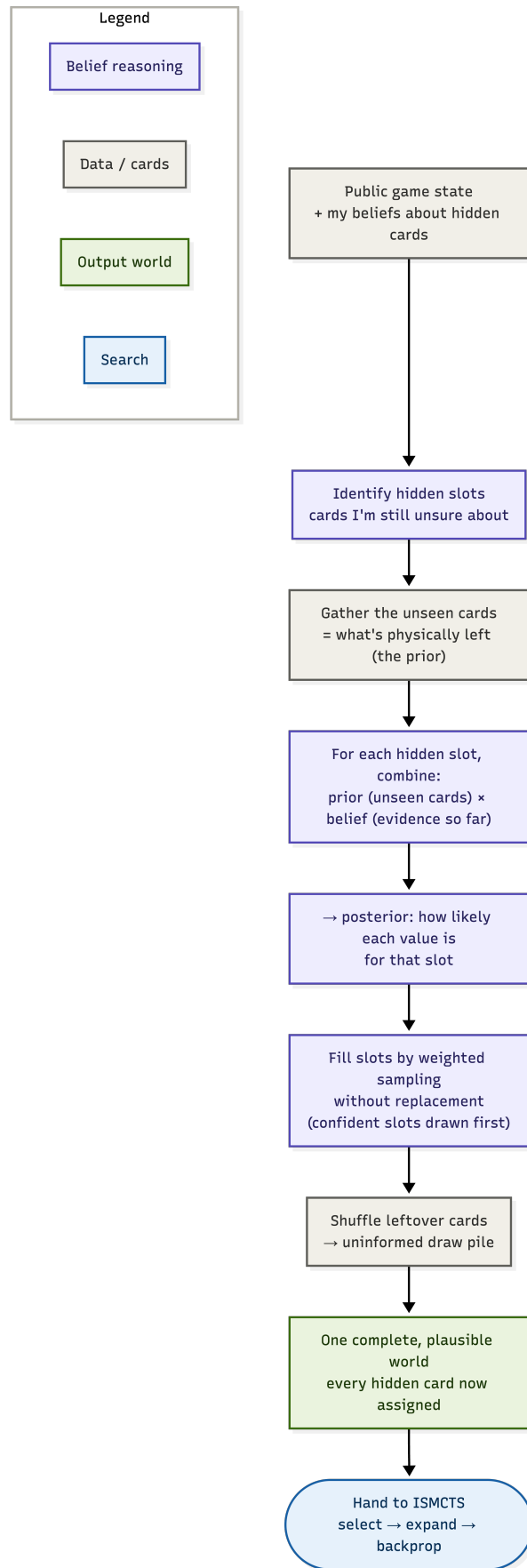


Figure 4.1: The belief-weighted determinization pipeline.

4.6.1 The Posterior Where Prior Meets Evidence

The foundational idea behind the design is that the physical set of cards not yet accounted for constitutes the unseen pool. If the agent knows the location of certain cards, then every remaining card must be distributed among the hidden slots and the draw pile, and the composition of that pool dictates how likely each value is before any behavioural evidence is considered. A value that appears many times in the remaining pool is, all else being equal, more likely to occupy a given hidden slot than a value that appears once.

Combining that pool with the per-slot log-likelihood vector from Section 4.5 gives a per-slot posterior over values,

$$P(\text{slot} = v) \propto N_{\text{pool}}(v) \exp(\ell(v)), \quad (4.1)$$

Here, $N_{\text{pool}}(v)$ represents the number of cards of value v that remain in the unseen pool, while $\ell(v)$ represents the accumulated log-likelihood for that value in the slot. When the log-likelihood is flat, as it always is when the Bayesian layer is disabled or when no evidence has been gathered about a slot, the exponential term is constant. The posterior therefore reduces to the pool prior alone, which is exactly the behaviour of the uniform determinizer. This makes it so when it uses a flat belief the two agents sample identically. The belief layer adds evidence on top of the deck’s natural distribution without contradicting it. A card that is physically impossible because it is absent from the pool can never be sampled, regardless of how strongly the belief favours it, since its prior count is zero.

4.6.2 Sampling One World

A single determinization follows the pipeline shown in Figure 4.1. The agent first clones the public state using a fresh random seed derived from the iteration number, ensuring that successive iterations explore different worlds. It then identifies the hidden slots, including every active slot on either side, and constructs the unseen pool from the concrete list of card identities that have not been assigned to a known location to later apply a consistency check. If the pool contains fewer cards than the number of hidden slots, or if the resulting card multiset fails validation, the world is discarded, and the iteration is skipped rather than allowing an impossible state to corrupt the search.

The hidden slots are then filled one at a time using weighted sampling without replacement. For each slot, the effective log-likelihood is calculated by combining the stored belief for that slot with a situational adjustment described below. This value is then exponentiated to produce a weight for each possible card value. Each remaining card in the pool is assigned a weight based on the value it carries, and one card is sampled in proportion to these weights before being removed from the pool, preventing the same card from being assigned to multiple slots. Sampling without replacement is essential since the same

physical card cannot occupy two locations. Removing each card as it is placed ensures that every sampled world remains internally consistent with a real deck.

In order to make this process more robust, I added two details. First, the exponentiation uses a max-subtraction, computing $\exp(\ell(v) - \max_u \ell(u))$ rather than $\exp(\ell(v))$ directly. The common offset cancels out in the sampling ratio, while the shift prevents numerical overflow when a belief is strongly concentrated on a particular value. Second, the hidden slots are not filled in an arbitrary order. Because sampling without replacement is order-sensitive, cards assigned to earlier slots are unavailable to later ones. The agent therefore sorts the hidden slots according to the strength of their beliefs and fills the most confident slots first. A slot whose posterior is strongly concentrated on a particular value has the most to lose if that value is assigned to a slot with no strong preference. Filling the most confident slots first therefore reduces the bias introduced by naive sequential sampling. Slots with no evidence and a flat belief are filled last, taking whatever cards remain. This is the appropriate behaviour because these slots express no preference between the available values.

Once every hidden slot has been assigned, the remaining cards in the pool become the draw pile. The order of the draw pile contains no evidence, since nothing the agent has observed constrains which unseen card will be drawn next. The remaining pool is then shuffled uniformly and used as an uninformed draw order. This produces a plausible world in which each hidden card has been assigned according to the posterior, while the future draw order remains random. This world is passed to ISMCTS for a single selection, expansion, and backpropagation pass. Across the thousands of iterations used to make a decision, the search therefore encounters a distribution of worlds that reflects the agent’s beliefs and concentrates its effort on the configurations that the available evidence suggests are most likely.

4.7 The Cambio Guard

During development, standard ISMCTS treated calling Cambio in the very first or second round as a reasonable strategy, and so it did so very often. While an early call can occasionally be effective, the repetition made it clear that this was not a healthy or realistic way to approach the game. At that stage, the decision is essentially a blind RNG gamble, resting entirely on the total number of cards in the deck and the possible combinations rather than on any acquired knowledge of the hand.

To prevent this, I developed the Cambio Guard. In essence, it is a filter applied to the root move set of the search. While the AI does not yet believe its own hand is low enough, the `CallCambio` move is removed from the legal actions the search is allowed to consider, forcing the AI to keep playing until its believed hand value reaches the threshold or falls below it. In its baseline form, the guard is an absolute cap; the summed believed own

score must be less than or equal to `CambioGuardScore`, whose default value is 10, since that gave the best Cambio-calling behaviour:

$$\sum_{s \in \text{own slots}} \widehat{\text{value}}(s) \leq \text{CambioGuardScore}. \quad (4.2)$$

This believed score uses a flat prior: slots the AI is certain of contribute their exact value, and every unknown own slot contributes a fixed prior value, which is 5.889:

$$\widehat{\text{value}}(s) = \begin{cases} \text{value}(s) & \text{if } s \text{ is known,} \\ 5.889 & \text{if } s \text{ is unknown.} \end{cases} \quad (4.3)$$

Another key feature to stand out is that Cambio Guard will never filter the move set down to zero; if removing the call would leave no legal moves, the call is left in.

This does interfere with the way ISMCTS ordinarily reaches decisions via selection policy (UCT). By pruning `CallCambio` at the root, the guard overrides that reward-driven choice in this one narrow case. In the context of Cambio, this produced a markedly better experience since the game involves very few variables: each player holds only four cards and starts the game with just two unknowns. In such a small state space, a hard behavioural prior is a reasonable substitute for what the search cannot reliably learn from so little information.

This became a key aspect of my design, because the moment of calling Cambio also became dynamic and belief-based within my Bayesian layer. Standard ISMCTS decides whether to call Cambio purely on how good its hand is in absolute terms, with no regard for how weak the opponent's hand might be. The Bayesian variant instead replaces the absolute cap with a relative, distribution-based test. It estimates believed end-score distributions for both itself and the opponent under the same deck coherent posterior, where known slots contribute their exact value with zero variance, and each hidden slot contributes a mean and variance drawn from the posterior

$$P(v) \propto \text{poolHist}[v] \cdot \exp(\text{effLogL}[v]), \quad (4.4)$$

combining what remains in the unseen pool with the accumulated likelihood evidence. The difference between the two hands,

$$D = \text{own} - \text{opponent}, \quad (4.5)$$

is modelled as a Normal distribution whose mean is $\mathbb{E}[\text{own}] - \mathbb{E}[\text{opp}]$ and whose variance is $\text{Var}[\text{own}] + \text{Var}[\text{opp}]$ (a mean-field approximation treating the two hands as independent):

$$D \sim \mathcal{N}\left(\mathbb{E}[\text{own}] - \mathbb{E}[\text{opp}], \text{Var}[\text{own}] + \text{Var}[\text{opp}]\right). \quad (4.6)$$

The call is permitted only when the AI is confidently ahead by a margin, when

$$P(D < -\text{CambioMargin}) \geq \text{CambioConfidence} \quad (4.7)$$

The defaults are set at 2.0 points and a 0.60 probability. This margin is deliberate. Calling ends the AI’s turn and guarantees the opponent one final turn, accounting for the potential improvement a competent opponent could gain from that last draw and preventing the AI from calling when its lead is not as relevant

The evidence feeding this posterior is the belief state built by the opponent-evidence model of Section 4.5, updated from every observed game effect. The Cambio decision adds one further signal which happens when the opponent has called Cambio; this adds lean-low to their slots, on the reasoning that one rarely calls from behind. Drawing on all this revealed information, the Bayesian layer takes an informed gamble, weighting how good it believes its own hand is against how weak it believes the opponent’s is, and decides whether calling Cambio is worth it.

4.8 Unity Integration

The architecture is designed to be completely independent of the game engine, with `GameManager` acting as the boundary between Unity and the game engine. It owns the live `GameState` and dispatches each resulting `GameEffect` to the UI as a C# event, which the presentation layer subscribes to in order to update itself.

4.9 Sources of Variability and Believability

It is worth being explicit about where this agent’s variety does and does not come from, because an earlier design intention of introducing deliberate mistakes was not part of the final system. The agent is not stochastic in its action selection. Once the search has completed, it selects the most-visited move at the root, meaning that the same search tree will always produce the same choice. There is no random move selection, no explicit probability of making a blunder, and no hand-tuned imperfection applied after the search. If not, its behaviour would therefore be deterministic.

The first source of variability comes from the way the inputs to the search differ between iterations or decisions. Each determinization samples a new set of possible worlds, as described in Section 4.6. The tree that the agent constructs, and consequently the move it selects, is influenced by which plausible states happen to be sampled, making it so that when the belief layer is enabled, these worlds are sampled from the posterior rather than uniformly. This concentrates the search on states considered more likely without reducing the distribution to a single world, so variation between sampled configurations remains

and can influence the final decision.

The second source of variety comes from the fact that belief-based reasoning is uncertain and can sometimes be wrong. The agent acts according to its estimate of what the opponent is likely to hold. When that estimate is accurate, it can lead to informed and effective decisions and wins during a game, but when the available evidence produces an incorrect belief, the agent may choose an action that a fully informed player would not and leads to really bad scores. This is an intentional part of the behaviour defined by the scope of the project, and it is exactly the profile that the believable-agent literature reviewed in Section 2.4 associates with a more engaging opponent following the idea of making one whose play appears reasoned while still allowing for occasional mistakes.

4.10 Telemetry and Reporting

To support the evaluation in Chapter 5, the agent is instrumented to record what it did and why, without disturbing the decision process itself. On each decision the agent emits an `IsmctsReport` capturing the search statistics for that move, such as the time taken, the iterations completed, and the root move eventually chosen, and alongside it a `BeliefReport` snapshot that pairs each hidden slot's current belief against the card's ground truth. Because ground truth is read only for logging and never enters the belief layer or the search, this reporting does not compromise the agent's no-cheating guarantee.

Chapter 5

Evaluation and Results

This chapter reports the results of the evaluation. It is organised into three analyses that together answer the research question.

The first is the AI analysis (Section 5.1), which combines data from all 400 games played by real players into a single comparison and analyses 1,000 simulated games in which the baseline agent is compared against the Bayesian agent to produce overall statistics for both agents and objectively evaluate which performs better. This evaluation is based on variables such as win rate, average hand-score value, number of successful matches, and number of penalty cards. Because most of these results are presented as box plots, a Mann–Whitney U test will also be performed to determine whether the differences between the two agents are statistically significant. This test is used rather than the signed-rank test because the agents were evaluated using independent sets of games of unequal size, meaning that the observations cannot be paired. The qualitative analysis that follows in the user study (Section 5.2) then examines how players perceived differences in the agents’ play styles and how these differences affected their experience. This ensures that the evaluation is not biased towards optimal play or objective performance alone, as the scope of the project is to develop an AI that is perceived more favourably, which is not necessarily related to its playing strength.

Finally, a computational analysis (Section 5.3) examines how efficiently both agents play and whether the added machinery of either one carries a performance cost.

5.1 AI analysis

This section reports the objective results from two separate sets of games. The first is the baseline ISMCTS (Ben) agent against the Bayesian-layer (Eva) agent across 1000 games, which will be refereed as "mirror match" through out this section. The second is the merged set of games played against human participants, where each agent is measured against the pool of players.

The first and more noticeable output we can read is the win rate. In the mirror

match the Bayesian agent wins 53% of games to the baseline’s 41%, a clear but moderate advantage (Figure 5.1). Against human players, the same comparison is far more extreme, with the baseline winning 40% of its games, while the Bayesian agent wins 84% (Figure 5.2). This is the largest and most drastic difference in the whole dataset.

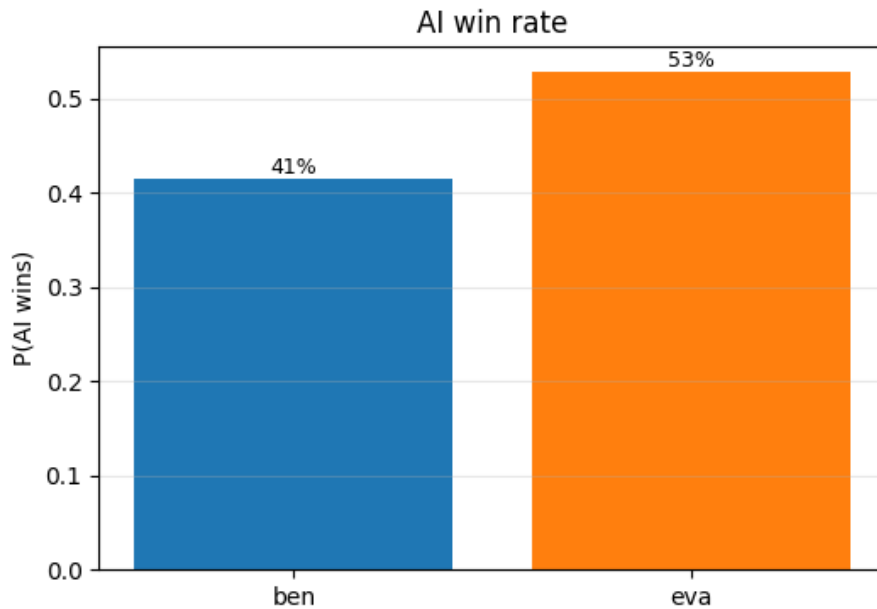


Figure 5.1: Agents win rate in mirror match

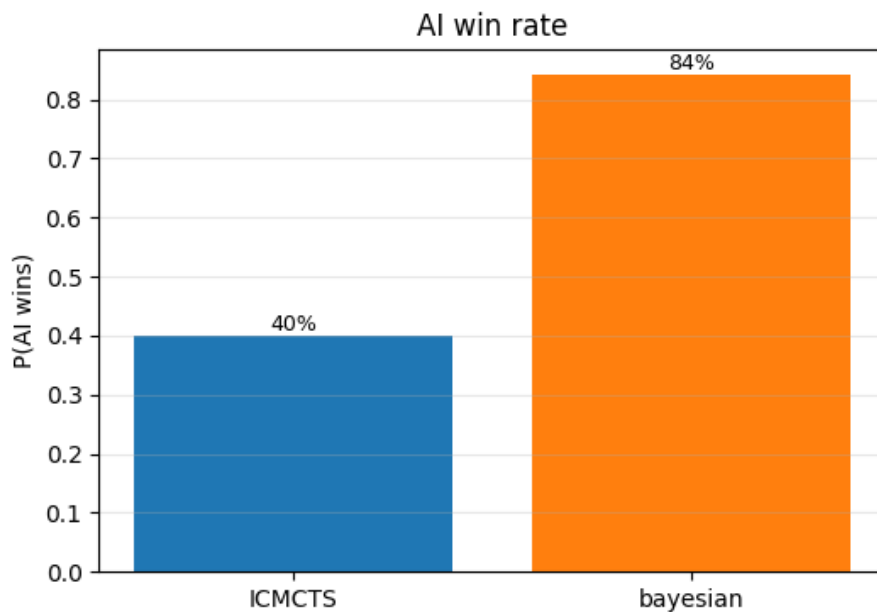


Figure 5.2: Agents win rate against human players

Despite the substantial difference in win rate, the final hand value each agent reaches remains consistent. In the mirror match, the baseline’s final hand value sits around 15, and the Bayesian agent’s at around 13 (Figure 5.3), and against human players both

agents again finish in a similar range (Figure 5.4). This proves how the AI score stays largely stable across both opponents and both testing modes, regardless of the win rate swinging dramatically. This suggests that the limited number of variables and the low uncertainty in Cambio keep the achievable end scores within a narrow band, so that the difference between the agents shows up in who wins far more than in the raw score either one reaches.

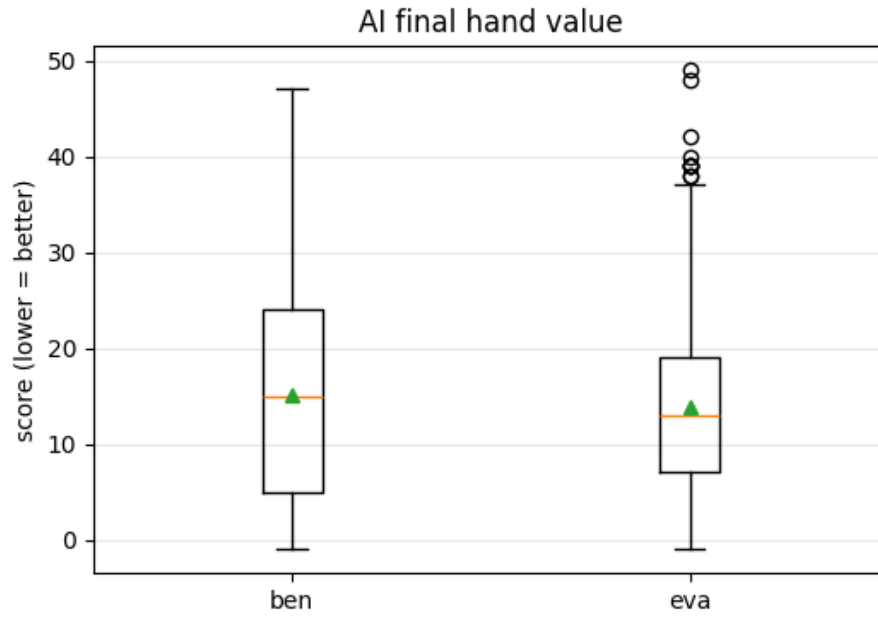


Figure 5.3: Agents hands average score in mirror match

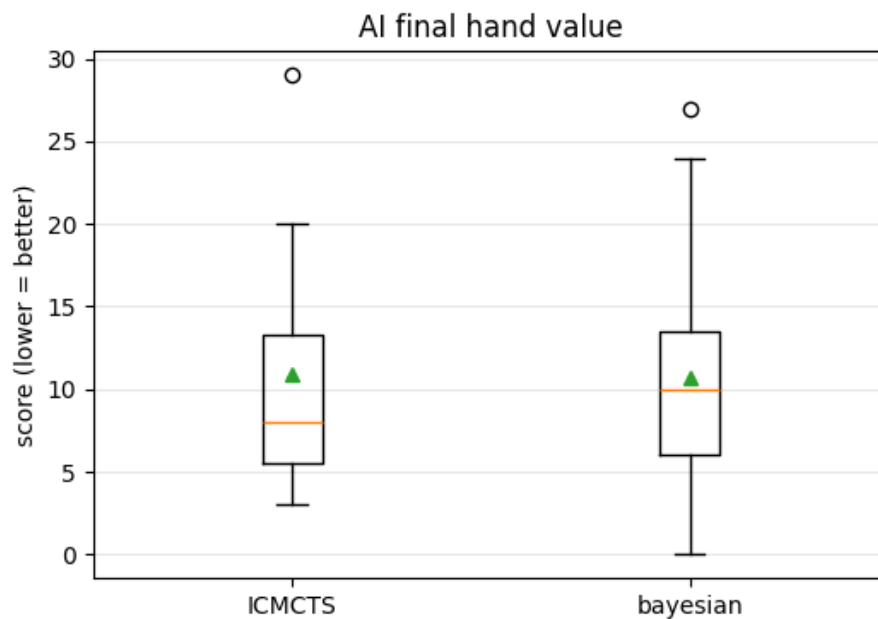


Figure 5.4: Agents hands average score against human players

The score margin between the player and the AI supports the same conclusion. In the

mirror match, both agents finish close to level, with the Bayesian agent only slightly above zero (Figure 5.5). Against human players, however, the two agents separate more clearly. The baseline has a median margin of approximately -4, indicating that it finishes slightly behind the human player, while the Bayesian agent has a median margin of around +13, placing it clearly ahead (Figure 5.6). As with the win rate, the score margin remains relatively close in the mirror matches but becomes substantially wider against human opponents, while the absolute hand value remains broadly similar. The spikes observed in the Bayesian agent's hand score are also worth noting, as they appear in this figure and throughout the wider analysis. The reason for these spikes is discussed in the discussion section.

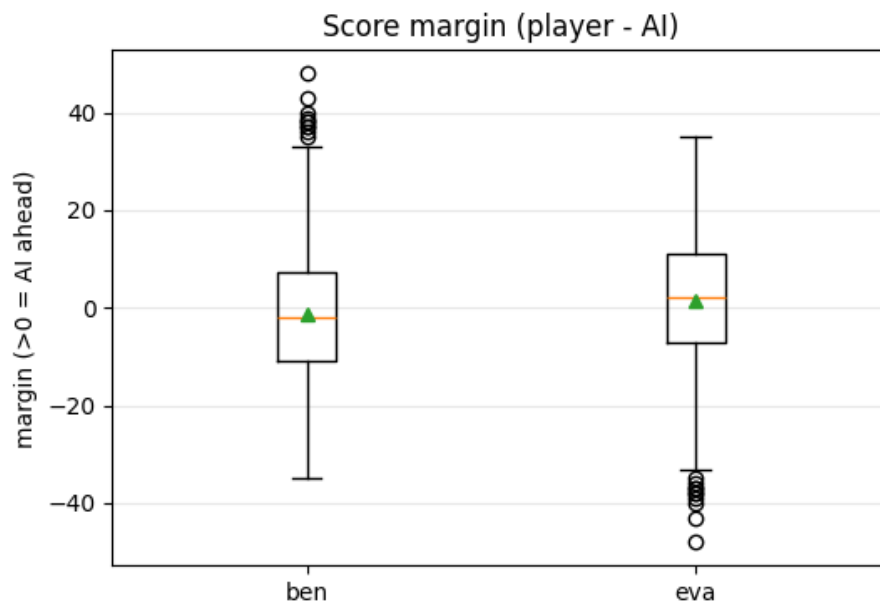


Figure 5.5: Agents score margin in mirror match

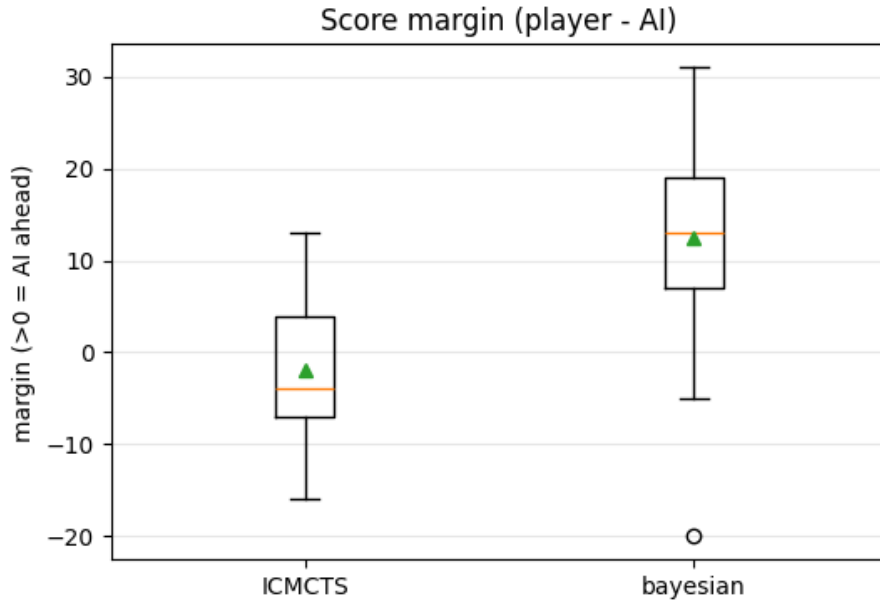


Figure 5.6: Agents score margin against human players

Now after analyzing how each agent plays, evidence most favour the Bayesian agent by a smaller amount. Its swaps are better with an average value change of a swap around -4.6 for the Bayesian agent against -2.4 for the baseline (Figure 5.7) against human players, and a -3.5 versus -3.0 in the mirror match(Figure 5.8).Also it draws far fewer penalty cards with a gap of 1.20 versus 0.32 against humans (Figure 5.9), but not as significant in the mirror match (Figure 5.10).

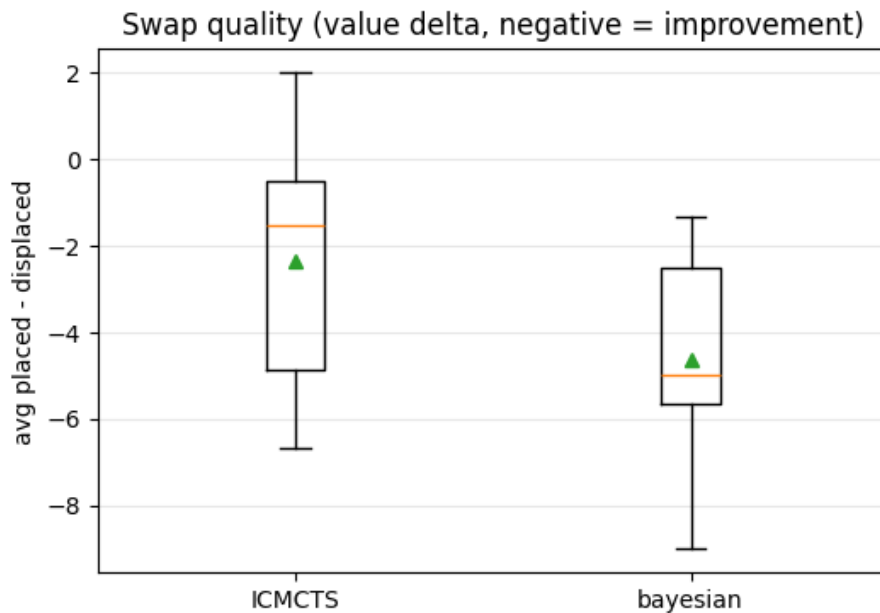


Figure 5.7: Agents swap quality against human players

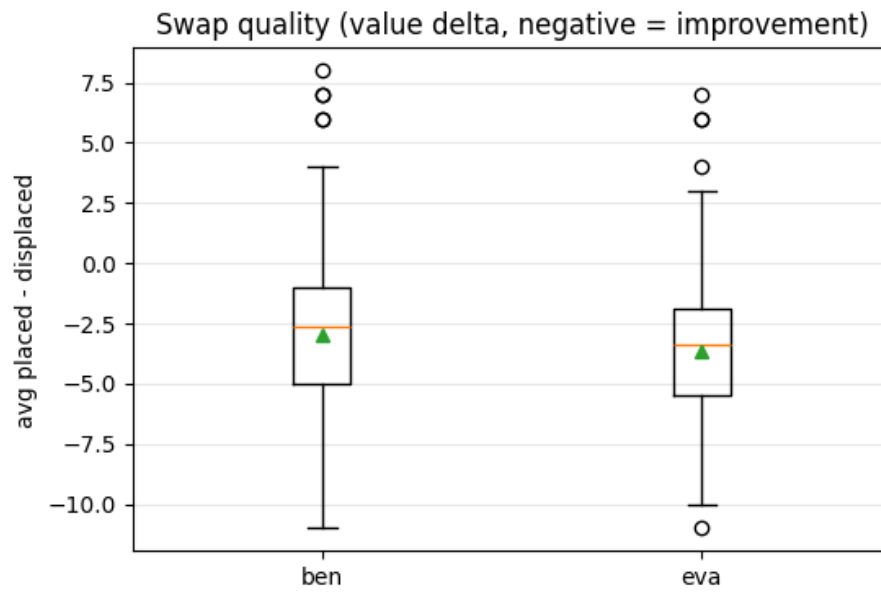


Figure 5.8: Agents swap quality in mirror match

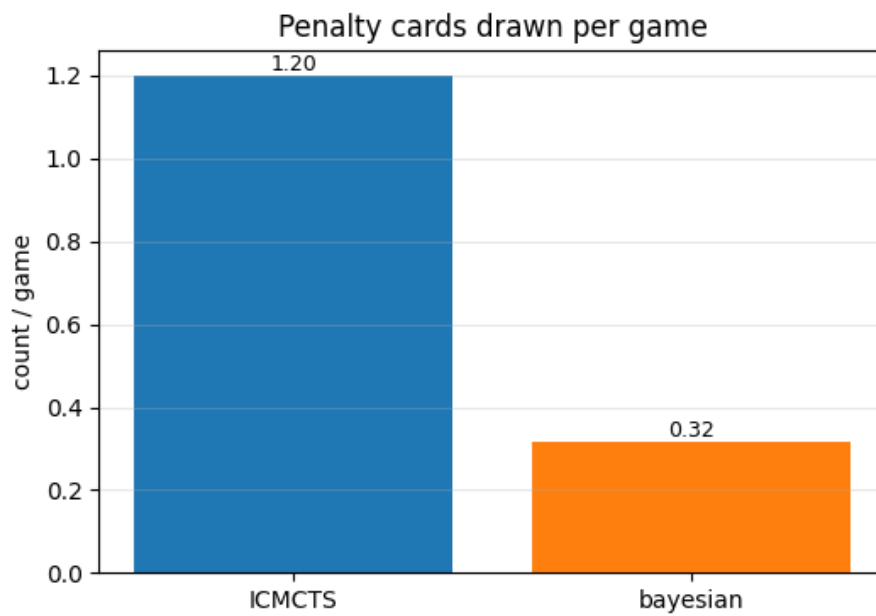


Figure 5.9: Agents penalty cards against human players

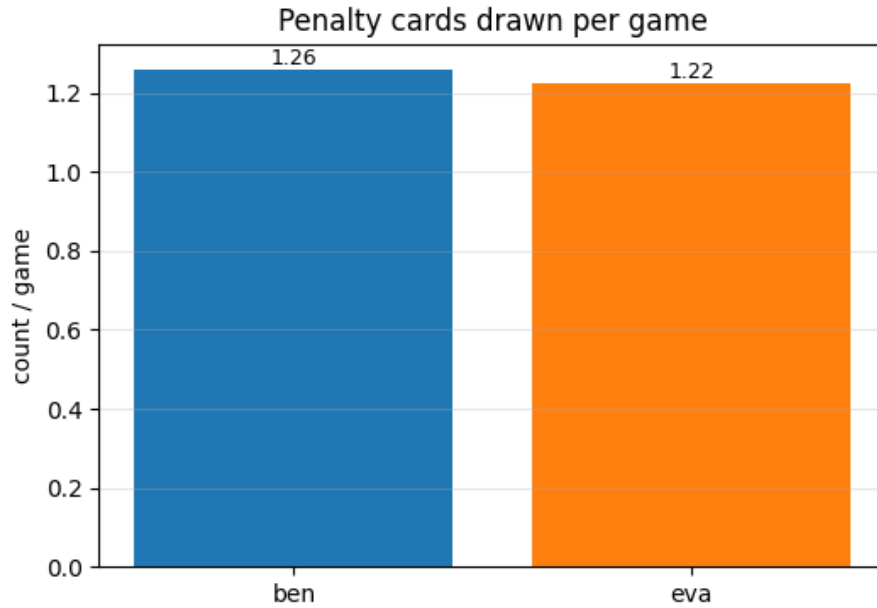


Figure 5.10: Agents penalty cards in mirror match

However, the mean number of the agent's own still-hidden cards per decision is effectively identical, at 1.98 against 1.97 against human players (Figure 5.11) which suggest that the way both AI improve their own hand does not change at all. For the match hit rate, in the mirror match, Bayesian got a 53% against the 51% of the baseline AI, whereas against humans the Bayesian agent is clearly ahead, at 76% against 55% (Figure 5.12). In the mirror match the successful-match split also differs in kind, with the Bayesian agent matching more often on its own cards and much less on the opponent's (Figure 5.13).

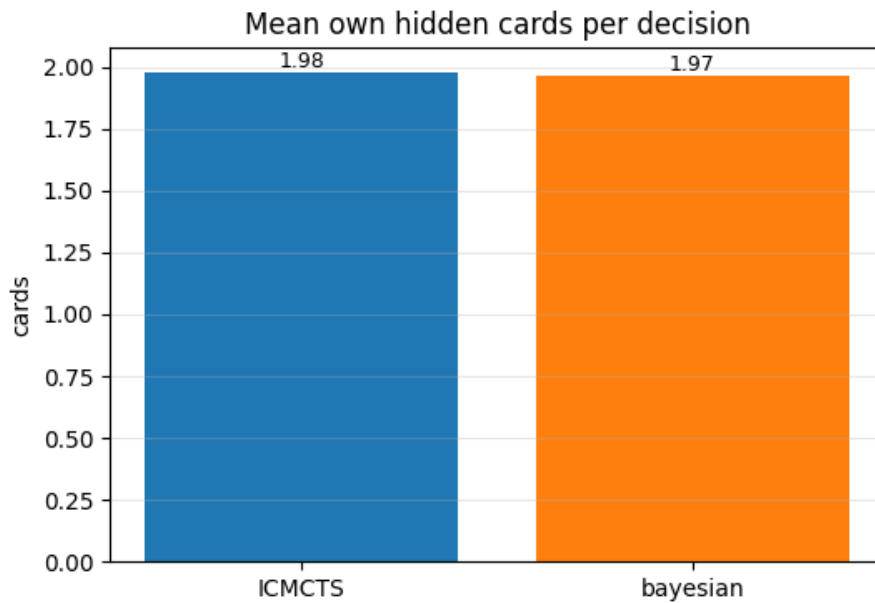


Figure 5.11: Agents hidden slots against human players

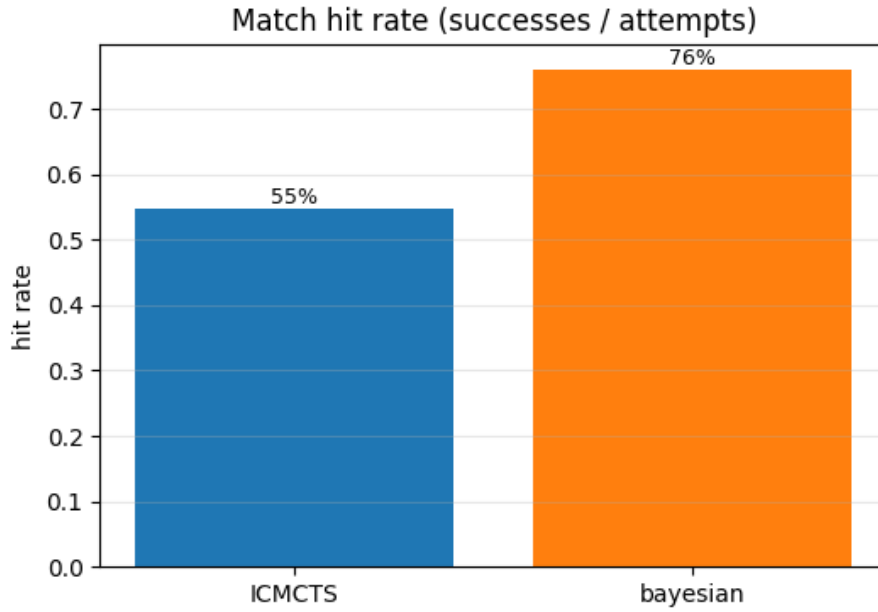


Figure 5.12: Agents match hit rates against human players

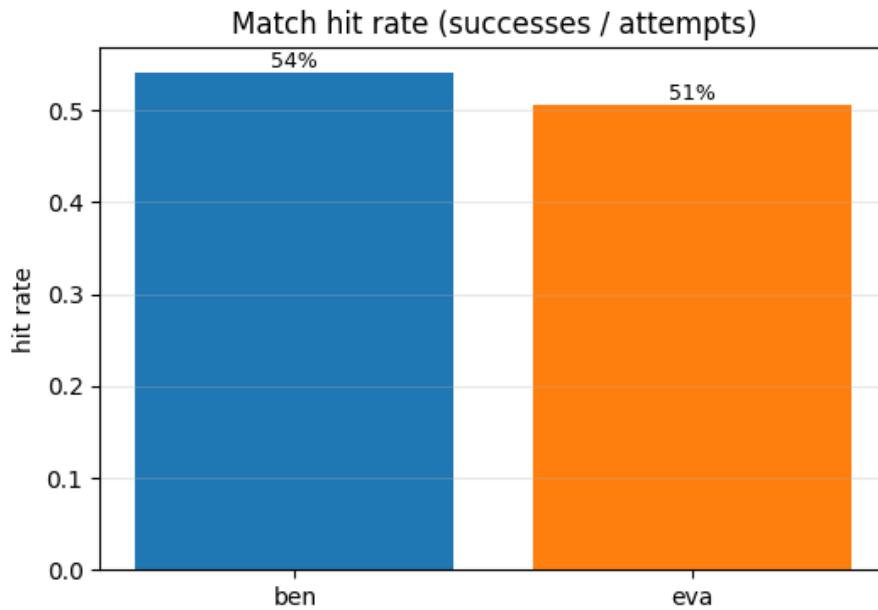


Figure 5.13: Agents match hit rates in mirror match

The clearest behavioural difference is in how the two agents end the game. The Bayesian agent calls Cambio earlier and more frequently. In the mirror match, its median call occurs on turn three, compared with turn six for the baseline agent (Figure 5.14), and it produces substantially more calls overall. Its accuracy when calling remains high, although it is not higher than the baseline's, as reflected in the baseline being ahead in 143 of 146 calls, while the Bayesian agent was ahead in 295 of 336 calls (Figure 5.15).

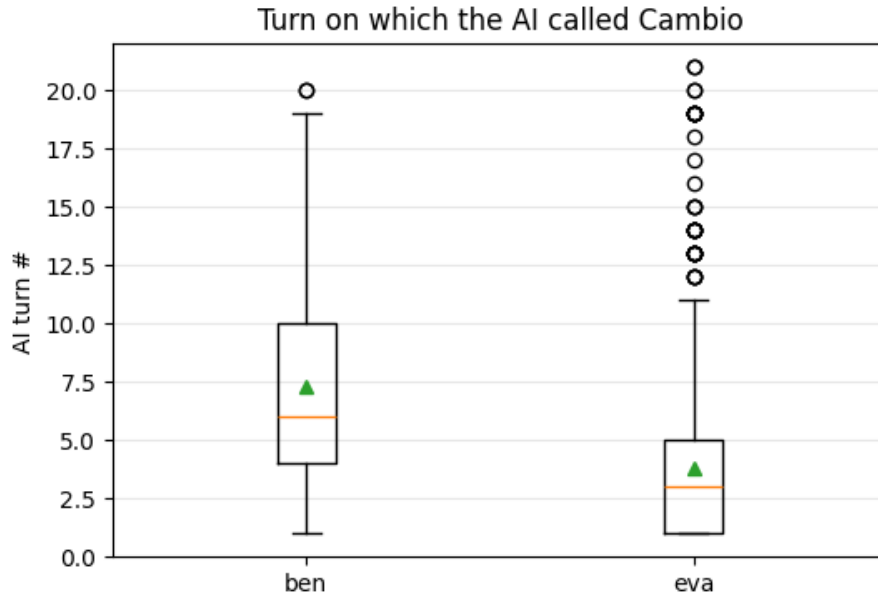


Figure 5.14: Agents cambio call turn in mirror match

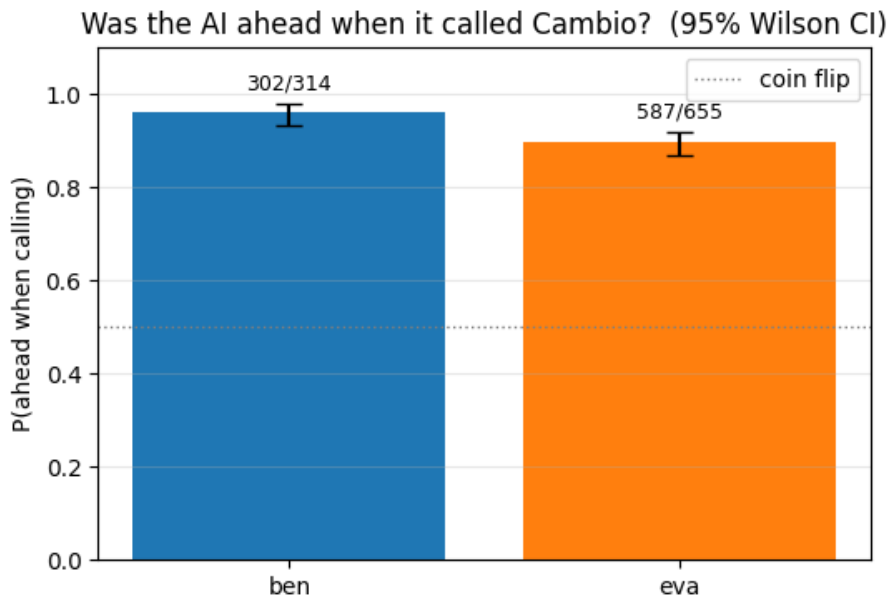


Figure 5.15: Agents cambio call while being ahead ratio in mirror match

Against human players, the same pattern was observed: the Bayesian agent was ahead in all 16 of its calls, while the baseline was ahead in 5 of 7. These figures should be interpreted cautiously because of the small number of observations (Figure 5.16). The per-call guard calibration provides further support for this behaviour, showing that Bayesian calls generally occur above the 0.60 confidence threshold enforced by the guard, with most calls being made when the agent was genuinely ahead (Figure 5.17).

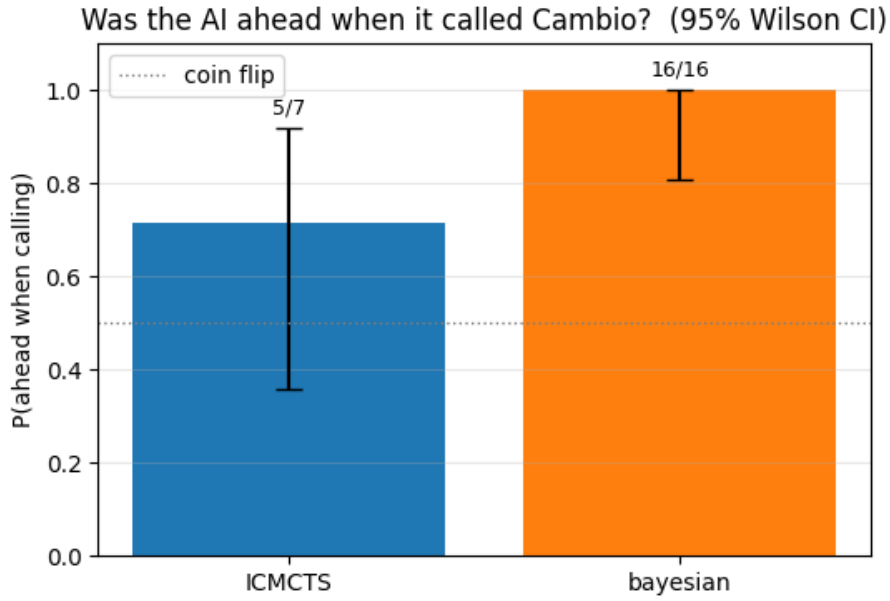


Figure 5.16: gents cambio call while being ahead ratio against human players

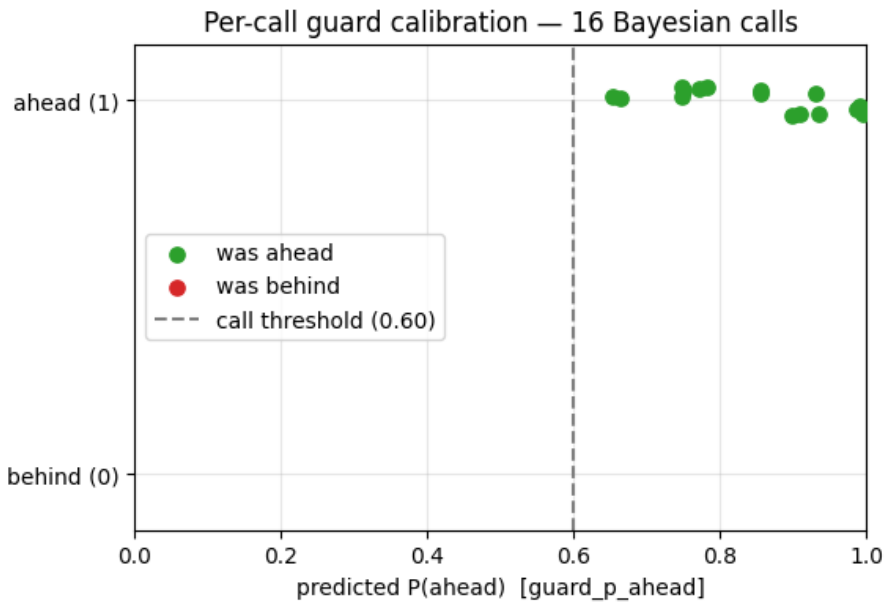


Figure 5.17: Agents cambio guard calibration against human players

A final metric examines the internal behaviour of the belief layer itself and is available only from the human games because it depends on modelling a distinct opponent. Figure 5.18 shows the effective tilt applied by the layer to each hidden slot, grouped by the card's true value and separated between opponent slots and the agent's own slots. For opponent slots, the tilt varies with the true card value and shows a weak negative correlation ($r=-0.32$). Lower-valued cards tend to receive larger positive tilts, while the applied tilt decreases towards zero as the true value increases. The effect is strongest across the low-to-mid value range, although there is considerable variation within each

value. For the agent’s own slots, the tilt remains effectively zero across all true values. The layer therefore applies very little adjustment to the agent’s own cards and instead concentrates its belief updates on opponent slots, which is where the opponent-evidence model is designed to operate.



Figure 5.18: Agents belief distribution on hidden cards

To validate whether the differences observed between the two agents are statistically significant, a Mann-Whitney U test was performed across every shared metric in the match, call, and belief data. The full output is shown below:

File	Metric	p-value	Significant
matches	ai_win_rate	0.0317	Yes
matches	ai_score	0.8721	No
matches	player_score	0.0005	Yes
matches	ai_turns	0.0002	Yes
matches	ai_swaps	0.0005	Yes
matches	ai_discards	0.0274	Yes
matches	ai_swap_value_delta_avg	0.1113	No
matches	ai_unknown_swaps	0.0351	Yes
matches	ai_hidden_own_avg	1.0000	No
matches	ai_match_attempts	0.0870	No
matches	ai_match_success	0.1108	No
matches	ai_match_fail	0.2278	No
matches	ai_matches_on_own	0.0184	Yes
matches	ai_matches_on_opponent	0.2068	No
matches	ai_penalties	0.2278	No
matches	ai_power_cards_drawn	0.0005	Yes
matches	ai_powers_played	0.0030	Yes
matches	ai_power_look_own	0.0101	Yes
matches	ai_power_look_opp	0.0631	No
matches	ai_power_blind_swap	0.2797	No
matches	ai_power_look_swap	0.5138	No
matches	ai_decision_ms_avg	0.0000	Yes
calls	cambio_caller_turn	0.0003	Yes
calls	cambio_ai_score	0.9234	No
calls	cambio_player_score	0.0002	Yes
calls	cambio_caller_was_ahead	0.0601	No
calls	cambio_ai_believed_score	0.0049	Yes
beliefs	tilt_raw	0.0316	Yes
beliefs	tilt_eff	0.0000	Yes
beliefs	true_value	0.0351	Yes

Table 5.1: Mann-Whitney U test results comparing the baseline and Bayesian agents in games against human players.

Overall, the results show a statistically significant difference between the two agents: 18 of the 30 metrics fall below the $p < 0.05$ threshold, including win rate, player score, decision time, and the belief-tilt values that the Bayesian model directly influences. In particular, the AI’s own score is not significantly different ($p = 0.8721$), indicating that the two agents achieve comparable results while differing in how they approach the game, generating different experiences for the opponent. The script used to generate these results can be found in the appendix.

5.2 User Study Results

The sample was drawn almost entirely from people that actively play video games, with 17 of the 20 reporting that they play video games regularly, while familiarity with card and board games was more mixed. Only 5 of the 20 had played Cambio before, so for most participants the ruleset was new. This context matters for drawing deeper conclusions from the results and understanding players' preference for one AI or the other.

The central observation is that the two agents were perceived very similarly. Across the six evaluation questions, the distributions of responses for the baseline agent (Ben) and the Bayesian agent (Eva) overlap heavily, and no question shows a marked separation between them. Figure 5.19 and Figure 5.20 plus Figure 5.21 and Figure 5.22 illustrate how in both cases the response profiles are close to interchangeable.

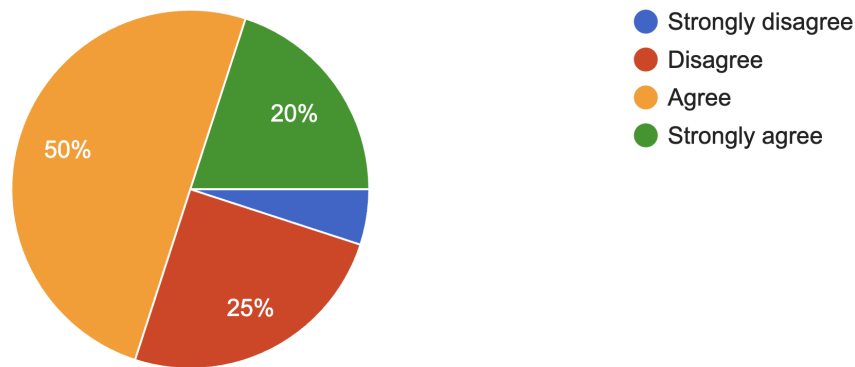


Figure 5.19: Enjoyment ratings for playing against Ben

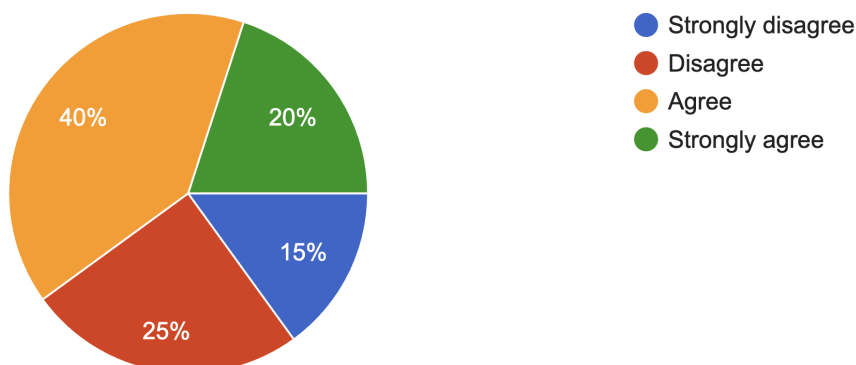


Figure 5.20: Enjoyment ratings for playing against Eva

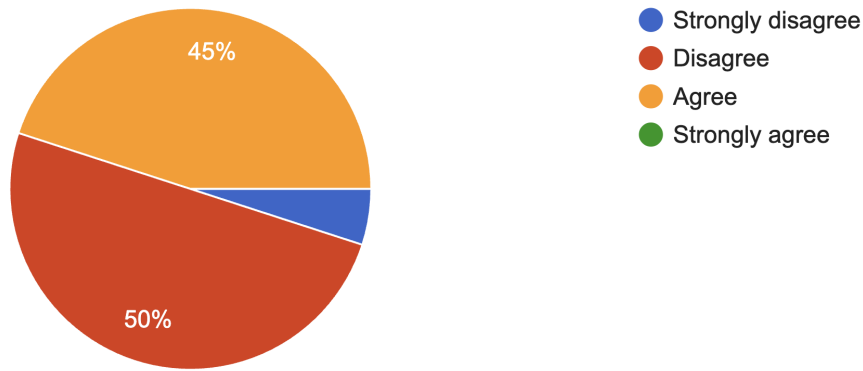


Figure 5.21: Ratings of Ben's adaptation to participants' strategies

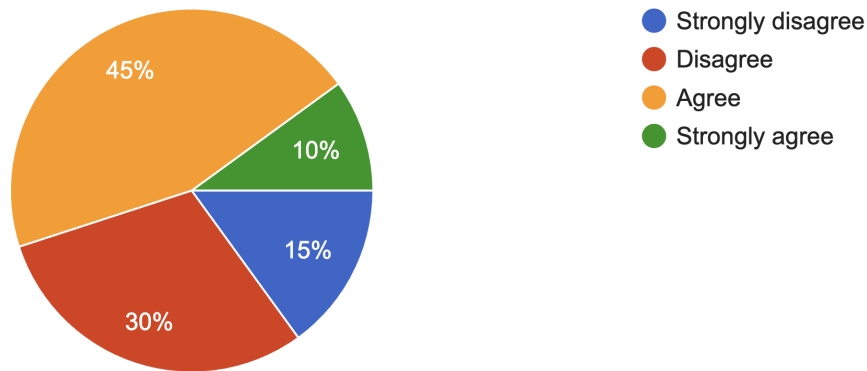


Figure 5.22: Ratings of Eva's adaptation to participants' strategies

To confirm that this apparent similarity is accurate, each Likert item was compared between the two agents using a Wilcoxon signed-rank test. The script used to perform these tests can be found in the Appendix. The results were as follows:

- **Enjoyment:** No statistically significant difference between Ben and Eva ($p = 0.294$).
- **Adjusted strategy:** No statistically significant difference between Ben and Eva ($p = 0.623$).
- **Predictability:** No statistically significant difference between Ben and Eva ($p = 0.714$).
- **Had a strategy:** No statistically significant difference between Ben and Eva ($p = 0.354$).

- **Took too long:** No statistically significant difference between Ben and Eva ($p = 0.179$).

None of the five items returned a statistically significant difference between the two agents.

The final question asked participants which agent they found more interesting and fun to play against. The Bayesian agent was preferred by 8 participants and the baseline by 7, with the remaining 5 neutral (Figure 5.23). This is a one-vote margin in the Bayesian agent’s favour; therefore, it does not represent a meaningful difference.

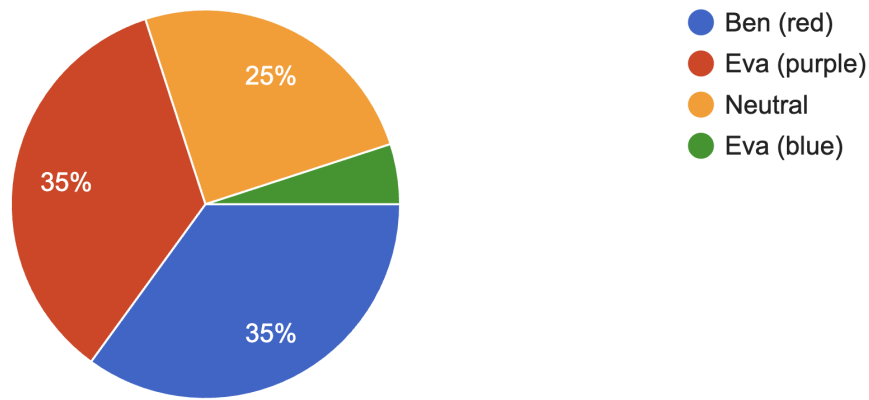


Figure 5.23: Participants’ preference for the more enjoyable AI agent

5.3 Computational Performance

The only computational cost of interest is the time each agent takes to make a decision, measured as the mean wall-clock time of its tree search per game across the 400 games. The baseline agent averaged 67 ms per decision and the Bayesian agent 84 ms (Figure 5.24). The belief layer added roughly 17 ms on top of the search, an overhead that is negligible in the context of a turn-based card game where the agent is not competing for a real-time frame budget.

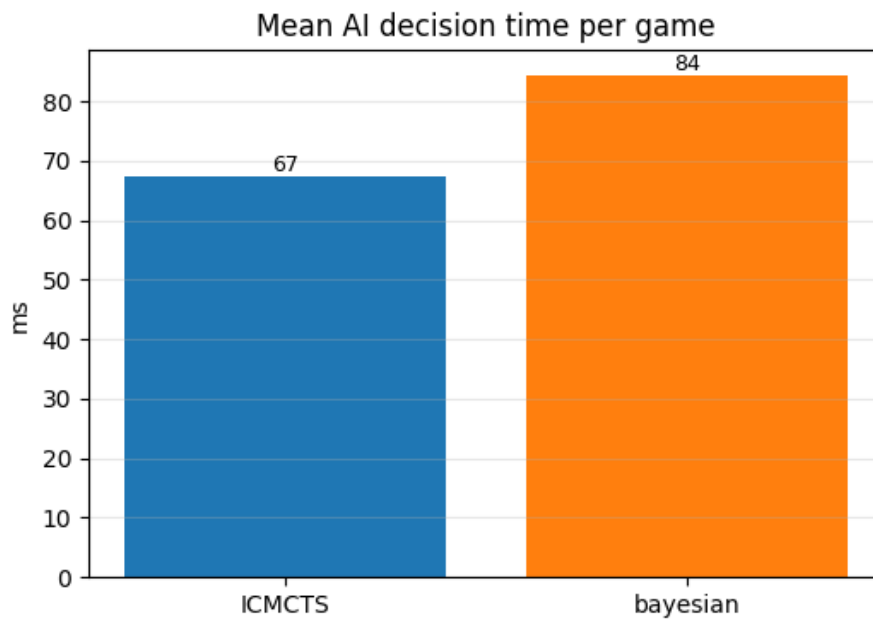


Figure 5.24: Agents mean decision time per game

Chapter 6

Discussion

The results reveal a disagreement between player perceptions and the recorded game outcomes. The questionnaire responses show no significant difference in how the two agents were perceived, with a distribution of 40% of the votes for Eva (Bayesian agent), 35% towards Ben (baseline ISMCTS) and 25% as neutral. The quantitative results, however, do not support the same conclusion. Across the games played against humans, the Bayesian agent outperforms the baseline. The 1000-game mirror match shows the same ordering with much greater statistical confidence, with the win rate increasing from 41% to 53%. This chapter argues that this apparent contradiction does not indicate that the belief layer has no effect, but instead results from the conditions under which the human data was collected.

The first and most important of those conditions is that the majority of playtesters had never played Cambio before, which inevitably introduces a layer of friction since a player meeting the game for the first time is simultaneously learning the rules and forming a judgement of the opponent, and the opponent they happened to face first was rated against a background of not yet understanding the game and how it should be played optimally. It was observed that this learning process depresses the enjoyment attributed to the first agent regardless of which agent it was, and because the order in which the two agents were presented was randomised, that penalty falls roughly evenly on Ben and Eva. The lack of a clear difference in player perceptions is likely because the variation introduced by players learning the game covered the differences between the two agents. When the analysis is restricted to games involving players who already understood Cambio, Eva still retains an advantage, although it is more subtle than the headline figures suggest, as it can be seen in Figure 6.1.

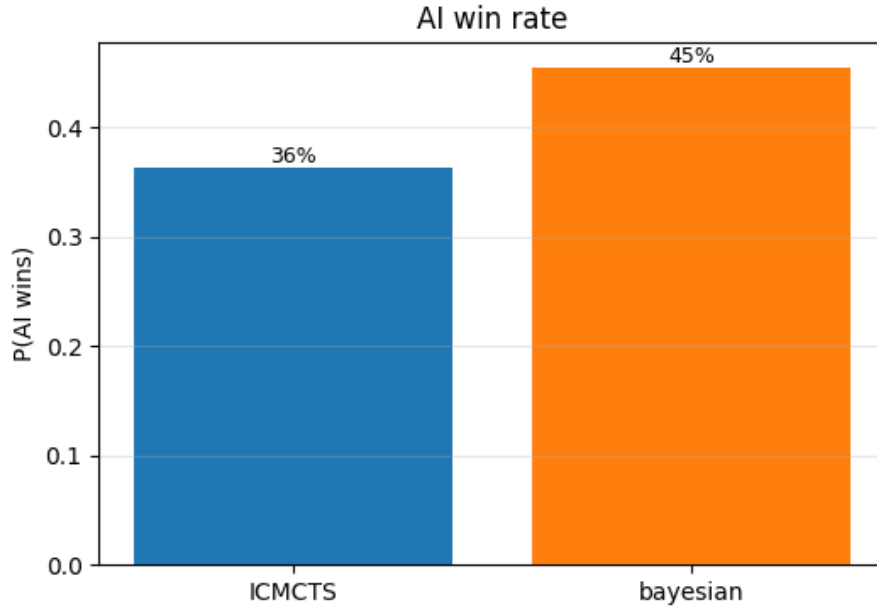


Figure 6.1: Play tester win rate

Another condition that affected the perception of the players with the AI was that each action the agents took was written to a text log and shown to the player as raw text, rather than visually in the play area. In addition, the pace at which those actions were produced was often faster than a player could follow by reading. This made players miss what had happened; for example, an agent might use a card power to swap a card, and the player would go on to attempt a match without realising that it was swapped or make a decision based on a board state that no longer existed. This frustration from the players attached itself to whichever agent they were playing against. This is a limitation of the measurement instrument rather than a property of the agents themselves, and it is a further reason why the perceived difference between them was compressed toward nothing even as their measured behaviour diverged.

That divergence can be seen in how the two agents decide to end a game. Ben calls Cambio based on a fixed threshold. In contrast, Eva calls whenever she believes her own hand to be lower than her opponent's, and this difference in mechanism produces a significantly more aggressive play style. Crucially, this aggression is exactly an output expected from the design implemented, as she remains ahead at the moment of calling in the large majority of cases, a rate well above chance, which shows the earlier calls are grounded in the belief state and not uninformed gambles. The trade-off is visible in the guard calibration, where Eva's willingness to call admits a cluster of calls made from behind that Ben's more conservative fixed style rarely does, as it is almost always ahead when calling. Still, he takes several more turns to get there. This is precisely the profile the literature associates with a more engaging opponent: one whose play is confident and reasoned but not infallible, rather than one that is cautious to the point of being

predictable.

This is also the source of the score spikes noted in Section 5.1 (Figure 5.5). Because Eva’s decision to call rests on a belief rather than on certainty, a call founded on an inaccurate belief occasionally ends the game while she is in fact behind, leaving her with a high final hand that registers as an outlier. The spikes are therefore not noise but the visible cost of belief-driven calling.

The player reactions divide along exactly the fault line this aggression creates based on their experience with the game. Players new to Cambio tended to read Eva’s early, frequent calls not as skilful but as erratic behaviour, in part because the calls cut the games short, which doesn’t let them keep playing and learn the game.

“Ben is better definitely, Eva is way too hard, she plays too fast. Ben felt balanced”

— Play tester 1

Experienced players by contrast tended to value the very thing the newer players disliked, describing Eva’s playstyle as a more varied and interesting challenge.

“Eva was less predictable, so it was more fun to me. I feel it was harder to like guess what she was gonna do, which made it more fun and challenging. Ben became predictable, so yeah.”

— Play tester 2

The clearest sign observed during playtest sessions is that some testers changed their own behaviour in response, beginning to call Cambio earlier themselves to get ahead of Eva’s aggression. Although the following quotation does not state this interpretation explicitly, it can be understood as reflecting such an adaptation.

“Ben because I felt I had a chance to win. Didn’t enjoy playing against Eva she was too difficult. I felt I was in a race to call cambio.”

— Play tester 3

Another observation worth stating is that the missed matches produced by the interface helped inexperienced players stay competitive, since plays they failed to register were also plays they were not punished for. But the same gap between what the agent did and what the player perceived meant that Eva’s occasional weak decisions based on acting on an uncertain belief were not read as human-like fallibility at all. They were read as stupidity: not an agent that plays well, and not an agent that plays like a person who sometimes errs, but an agent that appears not to realise it is making a mistake.

This is the sharpest lesson of the study since it exposes a gap between design intent and player reading. The belief layer does produce measurably different hidden-slot decisions, but without any expressive feedback to communicate that these choices are deliberate, the

behaviour reads as randomness rather than as reasoning; this is expanded in the future work section.

Taken together, these threads answer the research question, and the answer is a qualified no. The question was whether integrating the Bayesian opponent-modelling layer influences player-perceived opponent quality and difficulty, and on the evidence of the questionnaire it does not: players reported no significant difference between the two agents. That negative result, however, sits on top of a clearly positive one that the Bayesian layer does produce a stronger and more adaptive agent, and among the players who already knew Cambio, that improvement was both felt and, in some cases, responded to. The reason the improvement did not surface as a perceived difference across the whole group is that inexperienced players judged Eva while still learning the game, and an interface too fast and too sparse to let her reasoning be seen. This distinction between what was built and what was perceived, and what it would take to close the gap between them, is taken up in the chapters that follow.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This dissertation’s primary goal is to answer whether integrating an adaptive Bayesian opponent-modelling layer into a static ISMCTS agent influences player-perceived opponent quality and difficulty within the context of the card game Cambio. To answer this, the standard uniform determinization of ISMCTS was replaced with a belief-weighted sampler driven by an opponent-evidence model, following the theory behind Bayes’ theorem. The search itself was left unchanged, so that any measured difference between the two agents could be attributed to the belief machinery alone.

The questionnaire results show no statistically significant difference between the baseline agent, Ben, and the Bayesian variant, Eva. Based on player perception alone, the belief layer therefore did not clearly change how the opponent was experienced across the full participant group. This negative result, however, exists alongside a clear positive result, which is that the Bayesian layer produces a measurably different and stronger agent. It wins more often in both mirror matches and games against human players, swaps more efficiently, draws fewer penalties, and ends games using a reasoned, distribution-based call rather than a fixed cap, which adds variety to the matches. These changes also come at negligible computational cost. Among participants who already understood Cambio, the difference was more apparent and, in some cases, affected their responses. The most likely reason this difference did not emerge across the full sample is the set of conditions under which the human data was collected, which is addressed in the future work section.

Taken together, the Bayesian layer appears to add a distinct playstyle that varies the behaviour of standard ISMCTS while remaining slightly stronger objectively in this context, which makes it an overall better agent in the context of this dissertation. Cambio is a small game with little hidden information, so the ceiling for what such a layer can demonstrate here is limited; its true impact would need to be measured in larger hidden-information games, with longer horizons and more players, before strong claims could be

made. Even within these constraints, however, belief-weighted determinization driven by observed opponent behaviour looks like a promising and interesting direction for building adaptive game AI, and one worth developing further.

7.2 Future Work

The study produced a clear result about the agents' behaviour but a much weaker one about how that behaviour was perceived, and most of the future work follows from closing that gap. The directions below are ordered from the changes that would most improve the measurement to the changes that would most extend the contribution itself.

7.2.1 Improving the Study Design

Two aspects of the study introduced bias that was not anticipated beforehand and that a future version should remove. First, the questionnaire never recorded which agent each participant played first. Because most testers were playing Cambio for the first time, the opponent they faced first was judged while they were still learning the rules, which depressed its ratings regardless of which agent it was. Adding a single question that records the order of play would let this learning penalty be measured and controlled for. Second, the sample was small and largely unfamiliar with the game. While 400 matches is a really decent number of data samples, especially taking into account the time given for this dissertation, a larger cohort, deliberately including more players who already know Cambio, would reduce the noise that first-time learning introduced and give the perception data enough power to detect a difference the behavioural data shows is really there.

7.2.2 Communicating the Agent's Actions

The single change that would have had more impact on how the AI was perceived is to add a layer of visual polish and feedback. In the current build, each power, swap, and decision the agent made was written to a text log rather than shown in the play area, and often faster than a player could read. The majority of playtesters reported that this made the game hard to follow; there was a point at which they felt lost and would call Cambio just to end a turn they no longer understood. Replacing the log with visual feedback, with the actual cards moving in the scene as powers and swaps resolve, would let players concentrate on the game itself instead of trying to follow it with the text logs, and would give them a far better basis for judging the opponent. It is worth noting that some difficulty in tracking is inherent to Cambio, since forgetting one's own or the opponent's cards is part of the game, and this is precisely why the agent was allowed to

make mistakes; but the text log amplified that difficulty well beyond what the game itself imposes.

7.2.3 Making Mistakes Read as Human

The agent's capacity for mistakes helped inexperienced players to play longer and sneak some wins, but over time it read as a negative signal because the agent gives no sign of recognising that a belief-driven choice went wrong, making its behaviour come across as robotic and dumb rather than as human fallibility. One way to close this gap can be through expressive feedback: short comments that convey emotion or personality at the moment of a negative match attempt, such as "What?!" or "Wasn't this an eight?" could make the mistake read as a human error a player can empathise with rather than as a machine failing silently. Another way could be a light Dynamic Difficulty Adjustment layer that monitors how well the human is doing and, when the player is winning by a comfortable margin, disables the mistake-prone behaviour through a flag. This is admittedly a hard-coded intervention that removes some of the agent's autonomy, but in a game as short as Cambio it would make the opponent feel noticeably smarter and more responsive at the moments where it matters most.

7.2.4 Scaling to Larger Hidden-Information Games

Finally, Cambio's small state space limits what this layer can demonstrate since it is not being exposed to enough stress. At any moment, there are usually only around six hidden cards, two on the player's side and four on the opponent's, and while this number shifts as powers and swaps reveal and move cards, it rarely grows beyond that. Games also tend to end quickly. In such a small space it is easy for a player to mistake a strong-looking baseline play for a deliberate strategy when it was effectively random, simply because the odds of a good random outcome are high when so few cards are in play. A more demanding test would move the layer into a game with a longer horizon, giving the agent time to accumulate evidence and refine its beliefs and giving the player time to notice that adaptation. Cambio can also be played with more than two players, so another route could be to place three Bayesian agents against a single human; the way this multiplies the hidden state and the interacting beliefs could either reveal the layer's true strength or expose weaknesses that the two-player setting never stressed.

References

- [1] Andrade, Gustavo, Geber Ramalho, Alex Gomes, and Vincent Corruble (2006). “Dynamic Game Balancing: an Evaluation of User Satisfaction”. In: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AI-IDE)*. Vol. 2. 1, pp. 3–8.
- [2] Arrabales, Raúl, Agapito Ledezma, and Araceli Sanchis (2009). “Towards conscious-like behavior in computer game characters”. In: *IEEE Symposium on Computational Intelligence and Games (CIG)*. DOI: 10.1109/CIG.2009.5286473.
- [3] Bakkes, Sander C. J., Pieter H. M. Spronck, and H. Jaap van den Herik (2009). “Opponent modelling for case-based adaptive game AI”. In: *Entertainment Computing* 1.1, pp. 27–37. DOI: 10.1016/j.entcom.2009.09.001.
- [4] Brown, Noam and Tuomas Sandholm (2019). “Superhuman AI for multiplayer poker”. In: *Science* 365.6456, pp. 885–890. DOI: 10.1126/science.aay2400.
- [5] Browne, Cameron B., Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton (2012). “A Survey of Monte Carlo Tree Search Methods”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1, pp. 1–43. DOI: 10.1109/TCIAIG.2012.2186810.
- [6] Coulom, Rémi (2007). “Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search”. In: *Computers and Games (CG 2006)*. Vol. 4630. Lecture Notes in Computer Science. Springer, pp. 72–83. DOI: 10.1007/978-3-540-75538-8_7.
- [7] Cowling, Peter I., Edward J. Powley, and Daniel Whitehouse (2012). “Information Set Monte Carlo Tree Search”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.2, pp. 120–143. DOI: 10.1109/TCIAIG.2012.2200894.
- [8] Cowling, Peter I., Colin D. Ward, and Edward J. Powley (2012). “Ensemble Determinization in Monte Carlo Tree Search for the Imperfect Information Card Game

- Magic: The Gathering”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.4, pp. 241–257. DOI: 10.1109/TCIAIG.2012.2204883.
- [9] Frank, Ian and David Basin (1998). “Search in games with incomplete information: A case study using Bridge card play”. In: *Artificial Intelligence* 100.1-2, pp. 87–123. DOI: 10.1016/S0004-3702(97)00082-9.
- [10] Goodman, James (2019). “Re-determinizing Information Set Monte Carlo Tree Search in Hanabi”. In: *arXiv preprint arXiv:1902.06075*.
- [11] Hunicke, Robin (2005). “The Case for Dynamic Difficulty Adjustment in Games”. In: *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology (ACE '05)*, pp. 429–433. DOI: 10.1145/1178477.1178573.
- [12] Kocsis, Levente and Csaba Szepesvári (2006). “Bandit Based Monte-Carlo Planning”. In: *Machine Learning: ECML 2006*. Vol. 4212. Lecture Notes in Computer Science. Springer, pp. 282–293. DOI: 10.1007/11871842_29.
- [13] Soni, Bhuman and Philip Hingston (2008). “Bots trained to play like a human are more fun”. In: *IEEE International Joint Conference on Neural Networks (IJCNN)*, pp. 363–369. DOI: 10.1109/IJCNN.2008.4633818.
- [14] Southey, Finnegan, Michael Bowling, Bryce Larson, Carmelo Piccione, Neil Burch, Darse Billings, and Chris Rayner (2005). “Bayes’ Bluff: Opponent Modelling in Poker”. In: *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 550–558.
- [15] Whitehouse, Daniel, Edward J. Powley, and Peter I. Cowling (2011). “Determinization and information set Monte Carlo Tree Search for the card game Dou Di Zhu”. In: *2011 IEEE Conference on Computational Intelligence and Games (CIG)*, pp. 87–94. DOI: 10.1109/CIG.2011.6031993.
- [16] Yannakakis, Georgios N. and John Hallam (2009). “Real-Time Game Adaptation for Optimizing Player Satisfaction”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 1.2, pp. 121–133. DOI: 10.1109/TCIAIG.2009.2024533.
- [17] Zinkevich, Martin, Michael Johanson, Michael Bowling, and Carmelo Piccione (2007). “Regret Minimization in Games with Incomplete Information”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.

Acknowledgment of AI Tools

This section declares the generative AI tools used during the preparation of this dissertation, together with the purpose for which each was used.

- **Grammarly:** Used for grammar, spelling, and vocabulary checking throughout the written document.
- **GPT-5.6 Luna (OpenAI):** Used for grammar checking, guidance on document structure, and LaTeX formatting support.
- **Claude Opus 4.8 (Anthropic):** Used for code debugging, assistance in producing the plots and graphs presented in the results, and building debugging tools used to inspect and test the AI agents' decision-making.

Appendices

Appendix A

Project Materials and Reproducibility

All materials required to replicate the results presented in this dissertation are publicly available. The repository contains the complete Unity project source, a standalone Windows build together with play-testing instructions, all raw data collected during the study, and a link to the Google Colab notebooks used to perform the statistical analysis and generate the graphs.

The materials can be accessed at the following address:

`https://github.com/PabloGP88/bayesian-ismcts-cambio`

If the link above is not clickable, the address can be typed manually.

Appendix B

Statistical Analysis Scripts

This appendix reproduces the two Python scripts used to produce the statistical results reported in Chapter 5. Both were run in Google Colab.

Wilcoxon Signed-Rank Test

```
1  # Wilcoxon Signed-Rank Tests
2
3  import pandas as pd
4  import numpy as np
5  from scipy.stats import wilcoxon
6  from google.colab import files
7
8
9  # Upload and load the CSV
10
11 uploaded = files.upload()
12 filename = list(uploaded.keys())[0]
13 df = pd.read_csv(filename)
14
15 print("Data loaded successfully.\n")
16 display(df.head())
17
18
19 # Convert Likert responses to numbers (case/space tolerant)
20
21 likert_map = {
22     "strongly disagree": 1,
23     "disagree": 2,
24     "neutral": 3,
25     "agree": 4,
26     "strongly agree": 5,
27 }
```

```

28
29 def convert_column(col):
30     mapped = col.astype(str).str.strip().str.lower().map(likert_map)
31     # keep original value where it wasn't a Likert label
32     return mapped.where(mapped.notna(), col)
33
34 df_numeric = df.apply(convert_column)
35
36 print("\nConverted data:")
37 display(df_numeric.head())
38
39
40 # 3. Comparisons - matched by KEY PHRASE, not full header
41
42 comparisons = {
43     "Enjoyment": "enjoy playing against",
44     "Adjusted strategy": "adjusted his strategy",
45     "Predictability": "feel predictable",
46     "Had a strategy": "had a strategy",
47     "Took too long": "taking to long",
48 }
49
50 def find_column(df, phrase, name):
51     """Find the one column containing both 'phrase' and the person's '
52         name'."""
53     phrase, name = phrase.lower(), name.lower()
54     matches = [c for c in df.columns
55                 if phrase in c.lower() and name in c.lower()]
56     if len(matches) == 1:
57         return matches[0]
58     elif not matches:
59         raise KeyError(f"No column found for '{phrase}' + '{name}'")
60     else:
61         print(f"[!] Multiple matches for '{phrase}' + '{name}': {matches
62             }. ")
63         f"Using the first.")
64         return matches[0]
65
66 # Wilcoxon signed-rank tests
67
68 results = []
69
70 for measure, phrase in comparisons.items():
71     try:
72         ben_col = find_column(df_numeric, phrase, "Ben")

```

```

73     eva_col = find_column(df_numeric, phrase, "Eva")
74 except KeyError as e:
75     print(f"Skipping '{measure}': {e}")
76     continue
77
78 ben = pd.to_numeric(df_numeric[ben_col], errors="coerce")
79 eva = pd.to_numeric(df_numeric[eva_col], errors="coerce")
80
81 valid = ben.notna() & eva.notna()
82 ben, eva = ben[valid], eva[valid]
83
84 if len(ben) == 0:
85     print(f"Skipping '{measure}': no valid paired responses.")
86     continue
87 if (eva - ben).abs().sum() == 0:
88     print(f"Skipping '{measure}': all differences are zero.")
89     continue
90
91 statistic, p_value = wilcoxon(eva, ben)
92
93 results.append({
94     "Measure": measure,
95     "N": len(ben),
96     "Ben Median": ben.median(),
97     "Eva Median": eva.median(),
98     "Wilcoxon W": statistic,
99     "p-value": p_value,
100     "Significant (p < .05)": "Yes" if p_value < 0.05 else "No",
101 })
102
103
104 # Display results
105
106 if not results:
107     print("\nNo tests could be run - check the phrases in 'comparisons'.")
108 else:
109     results_df = pd.DataFrame(results)
110     print("\nWilcoxon Signed-Rank Test Results:")
111     display(results_df)
112
113     print("\nInterpretation:")
114     print("A p-value below 0.05 indicates a statistically significant")
115     print("difference between participants' ratings of Eva and Ben.\n")
116
117     for _, row in results_df.iterrows():

```

```

118         verdict = "SIGNIFICANT" if row["p-value"] < 0.05 else "NO
           significant"
119     print(f"{row['Measure']}: {verdict} difference "
120           f"(p = {row['p-value']:.3f})")

```

Mann–Whitney U Test

```

1 import pandas as pd
2 import numpy as np
3 from scipy.stats import mannwhitneyu, fisher_exact
4 from google.colab import files
5
6 uploaded = files.upload()
7
8
9 data = {"matches": {}, "calls": {}, "beliefs": {}}
10 for fname in uploaded:
11     low = fname.lower()
12     ftype = next((t for t in data if t in low), None)
13     cond = "baseline" if "baseline" in low else "bayesian" if "bayesian"
           in low else None
14     if ftype and cond:
15         data[ftype][cond] = pd.read_csv(fname)
16
17
18 EXCLUDE = {"match_index", "bayesian_on", "ply", "ai_turn",
19            "is_opponent_slot", "opp_knows", "completed"}
20
21 rows = []
22
23 for ftype, pair in data.items():
24     if "baseline" not in pair or "bayesian" not in pair:
25         continue
26     base_df, bayes_df = pair["baseline"], pair["bayesian"]
27
28     # Win rate via Fisher's exact test
29     if ftype == "matches" and "winner" in base_df.columns:
30         def wins(df):
31             w = df["winner"].astype(str).str.strip().str.lower()
32             return (w == "ai").sum(), (w != "ai").sum()
33         yw, yl = wins(bayes_df)
34         bw, bl = wins(base_df)
35         _, p = fisher_exact([[yw, yl], [bw, bl]])
36         rows.append([ftype, "ai_win_rate", round(p, 4), "Yes" if p <
           0.05 else "No"])

```



```

37
38
39     common = [c for c in base_df.columns
40                 if c in bayes_df.columns and c not in EXCLUDE]
41     for col in common:
42         base = pd.to_numeric(base_df[col], errors="coerce").dropna()
43         bayes = pd.to_numeric(bayes_df[col], errors="coerce").dropna()
44         if len(base) == 0 or len(bayes) == 0:
45             continue
46         if base.nunique() == 1 and bayes.nunique() == 1 and base.iloc[0]
47             == bayes.iloc[0]:
48             continue
49         try:
50             _, p = mannwhitneyu(bayes, base, alternative="two-sided")
51         except ValueError:
52             continue
53         rows.append([ftype, col, round(p, 4), "Yes" if p < 0.05 else "No
54             "])
55
56 results = pd.DataFrame(rows, columns=["File", "Metric", "p-value", "
57     Significant"])
58 display(results)

```

Appendix C

Questionnaire

Part A: Participant background

1. Do you play video games?
(*Yes, regularly / Occasionally / Rarely / No*)
2. Do you play card or board games?
(*Yes, regularly / Occasionally / Rarely / No*)
3. Have you played card games similar to Golf, Skyjo, or Poker?
(*Yes / No*)
4. Have you played Cambio before?
(*Yes / No*)
5. Please select your age group.
(*18-24 / 25-34 / 35-49 / 50+*)

Part B: Agent evaluation

1. Did you enjoy playing against (agent)?
2. Do you feel (agent) adjusted its strategy based on how you were playing?
3. Did (agent) feel predictable at any moment during the match?
4. Did (agent) feel like it had a strategy?
5. Did (agent) respond differently when you used special cards such as peek or swap?
(*Yes / No / Neutral*)
6. Do you feel (agent) took too long to play?

Part C: Overall preference

1. Which AI did you feel was more interesting and fun to play against?
(*Ben / Eva / Neutral*)

Appendix D

Telemetry Column Dictionary

This appendix lists the full set of fields recorded across the three CSV tables described in Section 3.4.2. All three can be joined on (`session_id`, `match_index`).

Match table (`cambio_matches`)

One row per game.

- **Outcome:** `winner`, `ai_score`, `player_score`.
- **Decision quality:** `ai_swaps` and `ai_discards` (how often it keeps versus discards a drawn card), `ai_swap_value_delta_avg` (the average value of the placed card minus the displaced card, where a negative number means the swap improved its hand), and `ai_unknown_swaps` (swaps made into a slot it could not identify).
- **Uncertainty:** `ai_hidden_own_avg`, the mean number of its own still-hidden cards across all of its decisions in the game.
- **Matching:** `ai_match_attempts`, `ai_match_success`, `ai_match_fail`, `ai_matches_on_own`, `ai_matches_on_opponent`, and `ai_penalties`.
- **Power cards:** `ai_power_cards_drawn` and `ai_powers_played`, broken down into look-own, look-opponent, blind-swap, and look-and-swap.
- **Cost:** `ai_decision_ms_avg`, the mean wall-clock time of its tree search per decision.

Cambio-call table (`cambio_calls`)

One row per game in which Cambio was called.

- **Call:** `cambio_caller`, `cambio_caller_turn`, the scores at that instant, and `cambio_caller_was_`

- **Baseline guard:** `cambio_ai_believed_score` (the flat believed-own-score).
- **Bayesian guard:** `guard_believed_own_mean`, `guard_believed_opp_mean`, and `guard_p_ahead` (the believed own and opponent end-score means and the probability of finishing ahead).

Belief table (`cambio_beliefs`)

One row per hidden card per AI decision.

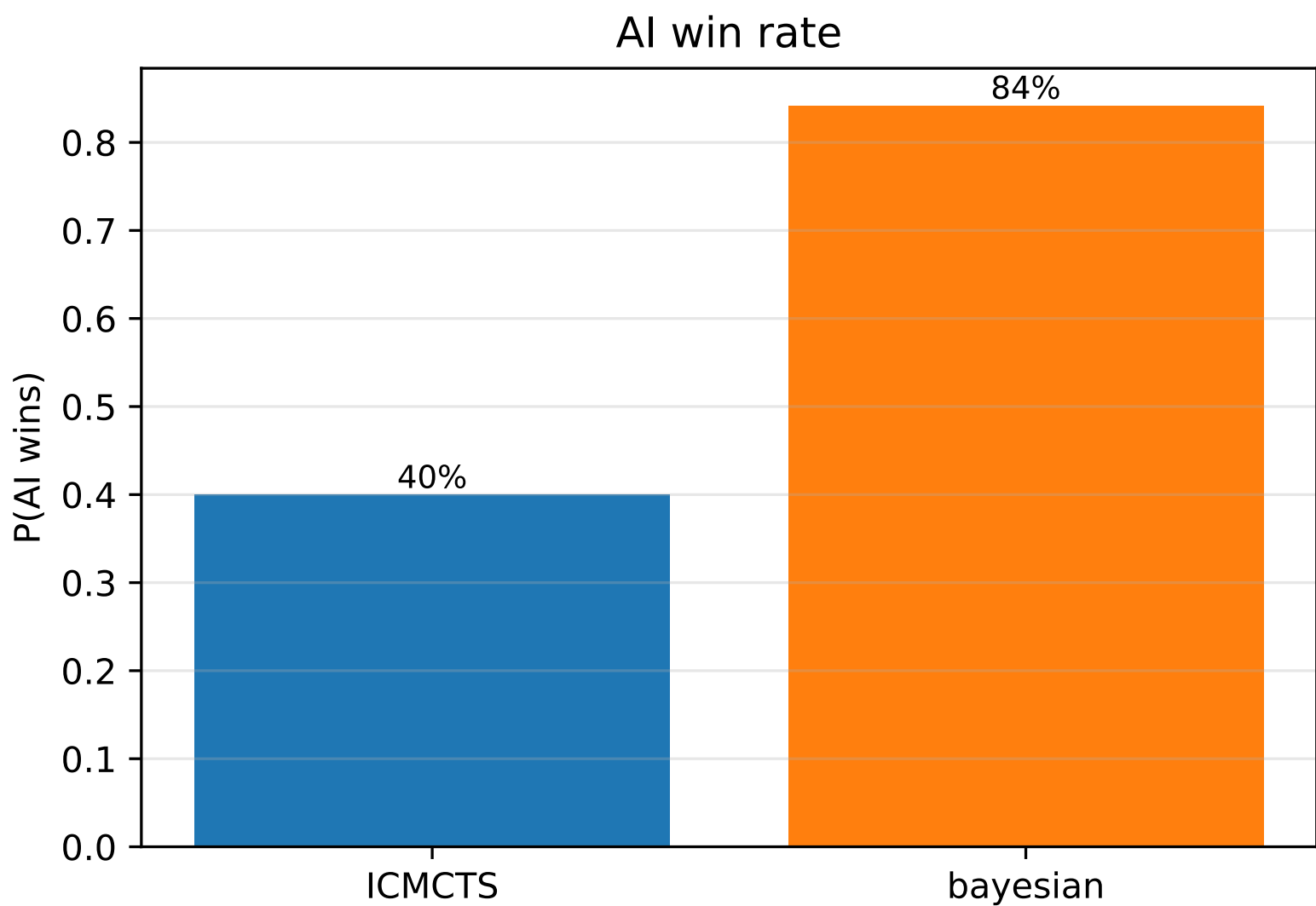
- `is_opponent_slot` (whether the card belongs to the opponent), `opp_knows` (whether the opponent is believed to know it), `true_value` (the card's ground-truth value), `tilt_raw` (the shift the belief layer applies to the card's expected value relative to a flat deck prior), and `tilt_eff` (the shift the search actually consumed).

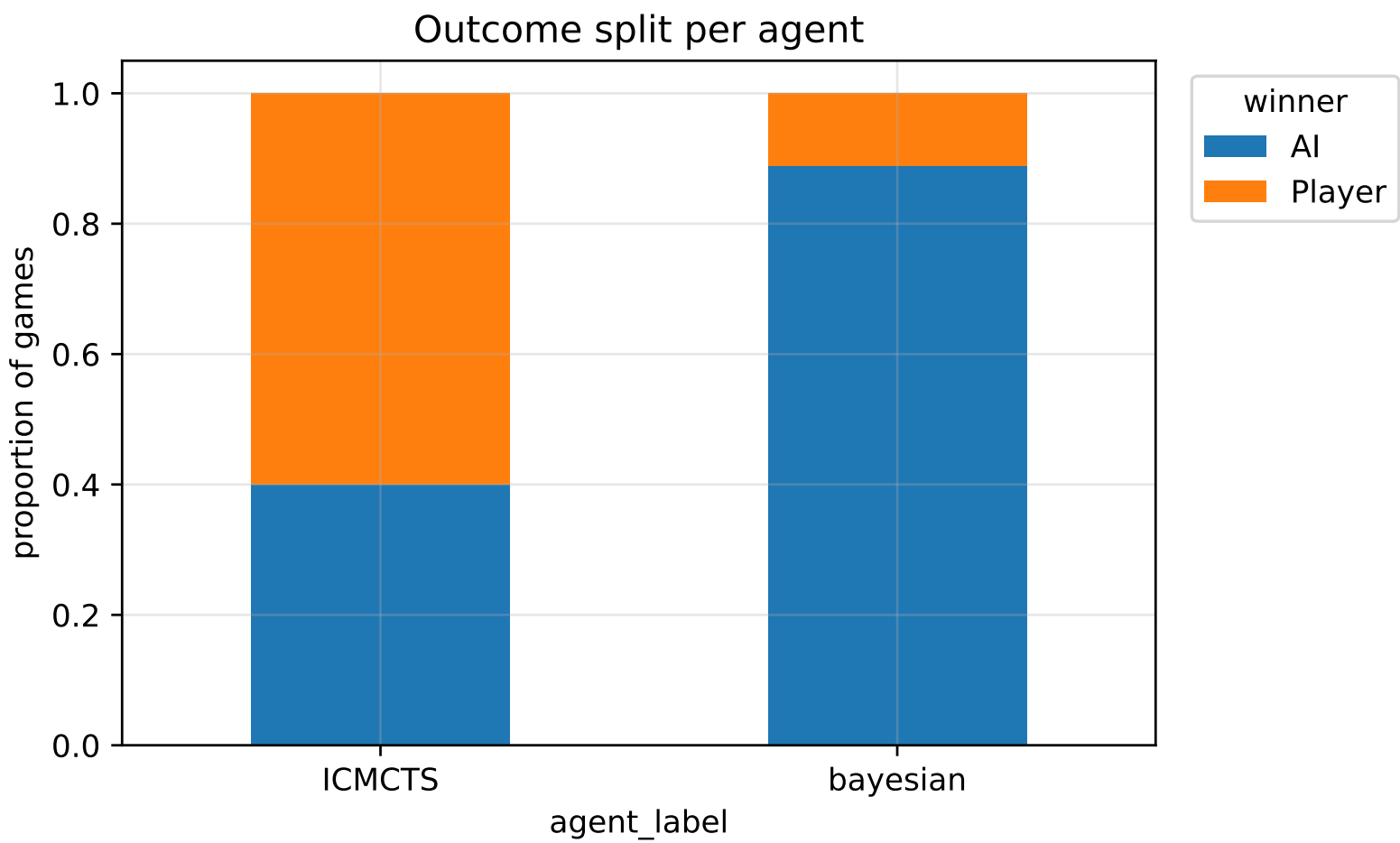
Appendix E

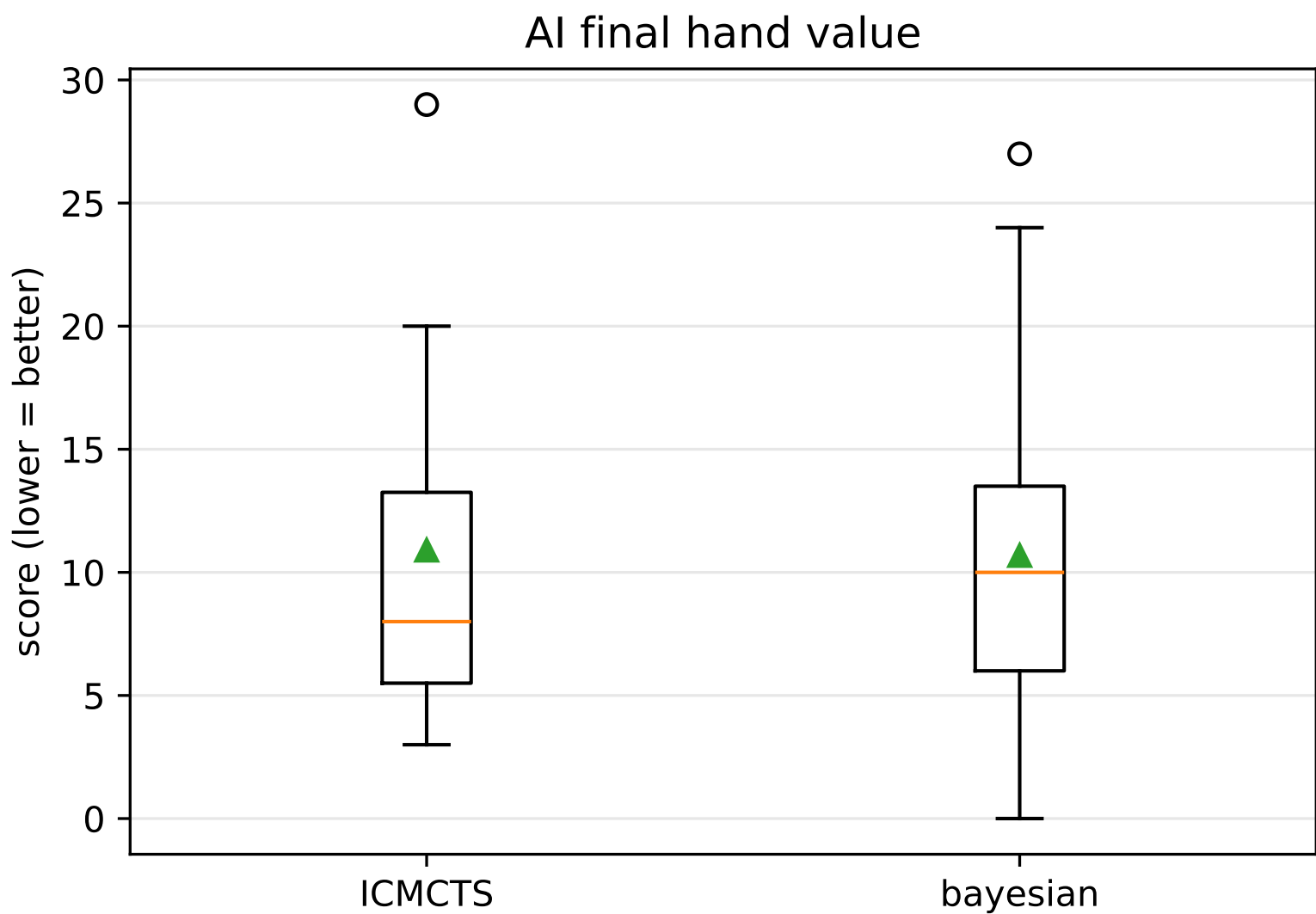
Per-Participant Result Sheets

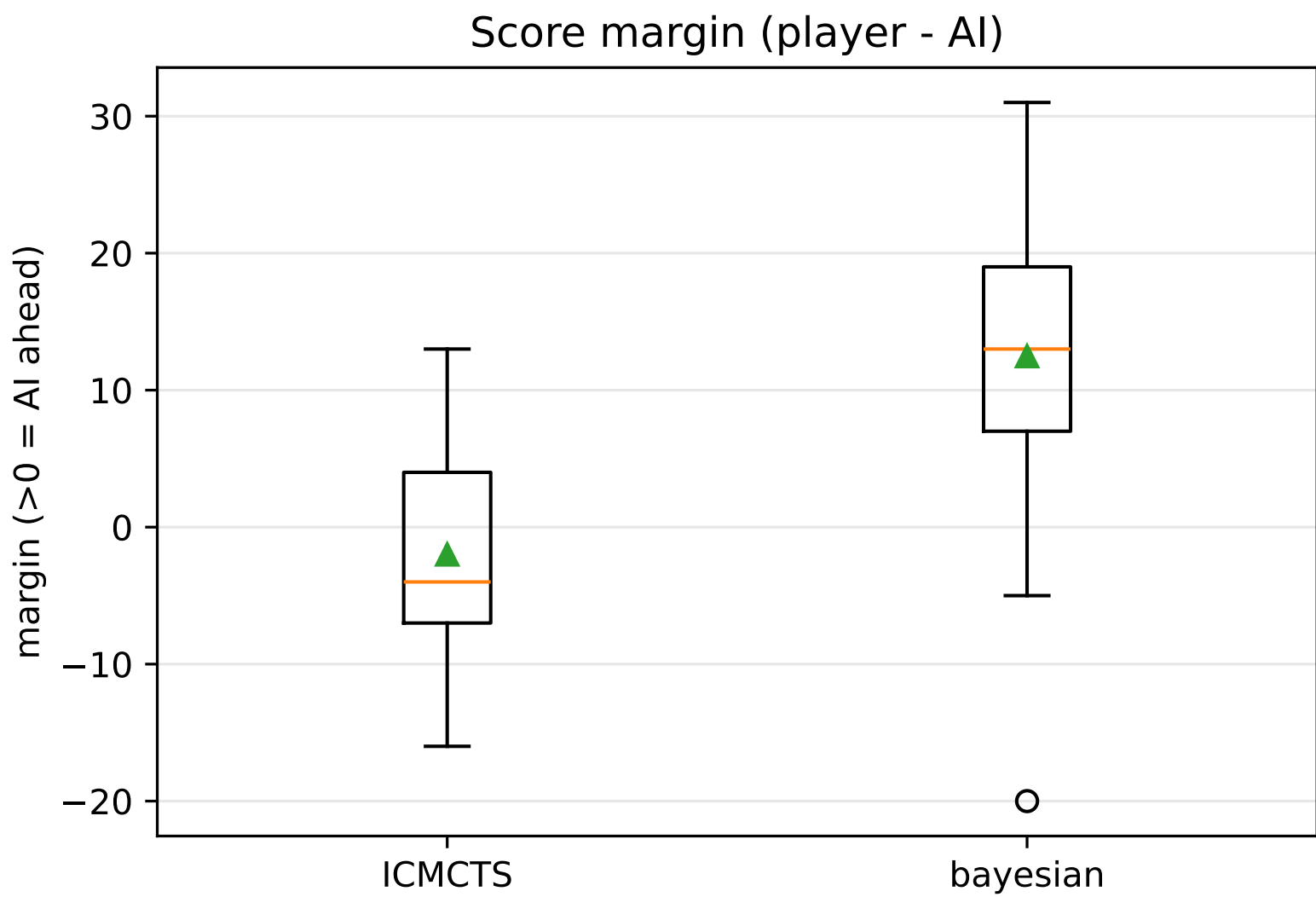
This appendix reproduces the two full result sheets underlying the analysis in Chapter 5. The first aggregates all 400 games played against the twenty human participants; the second covers the 1,000-game mirror match between the two agents.

Merged human games (400)

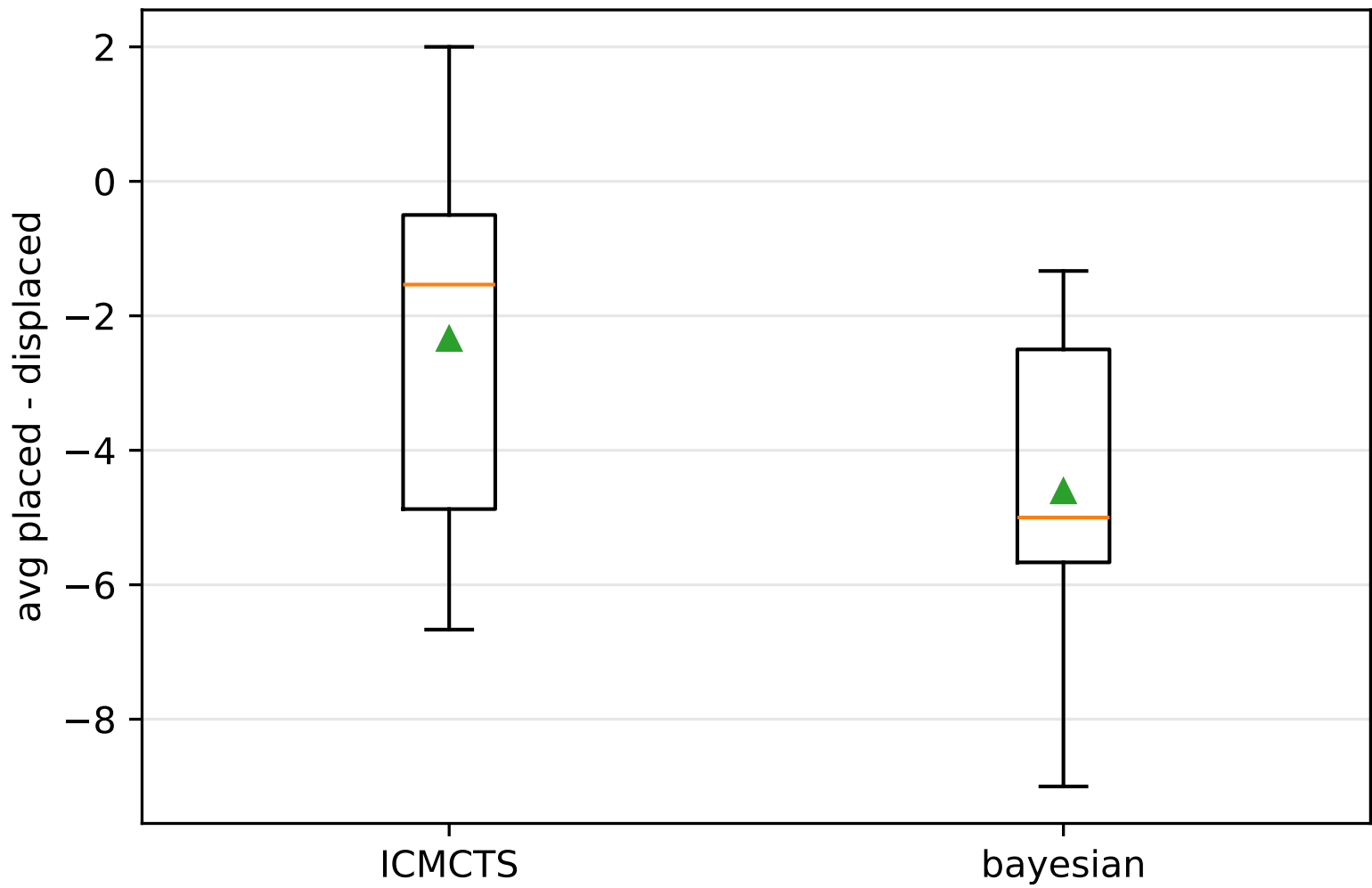




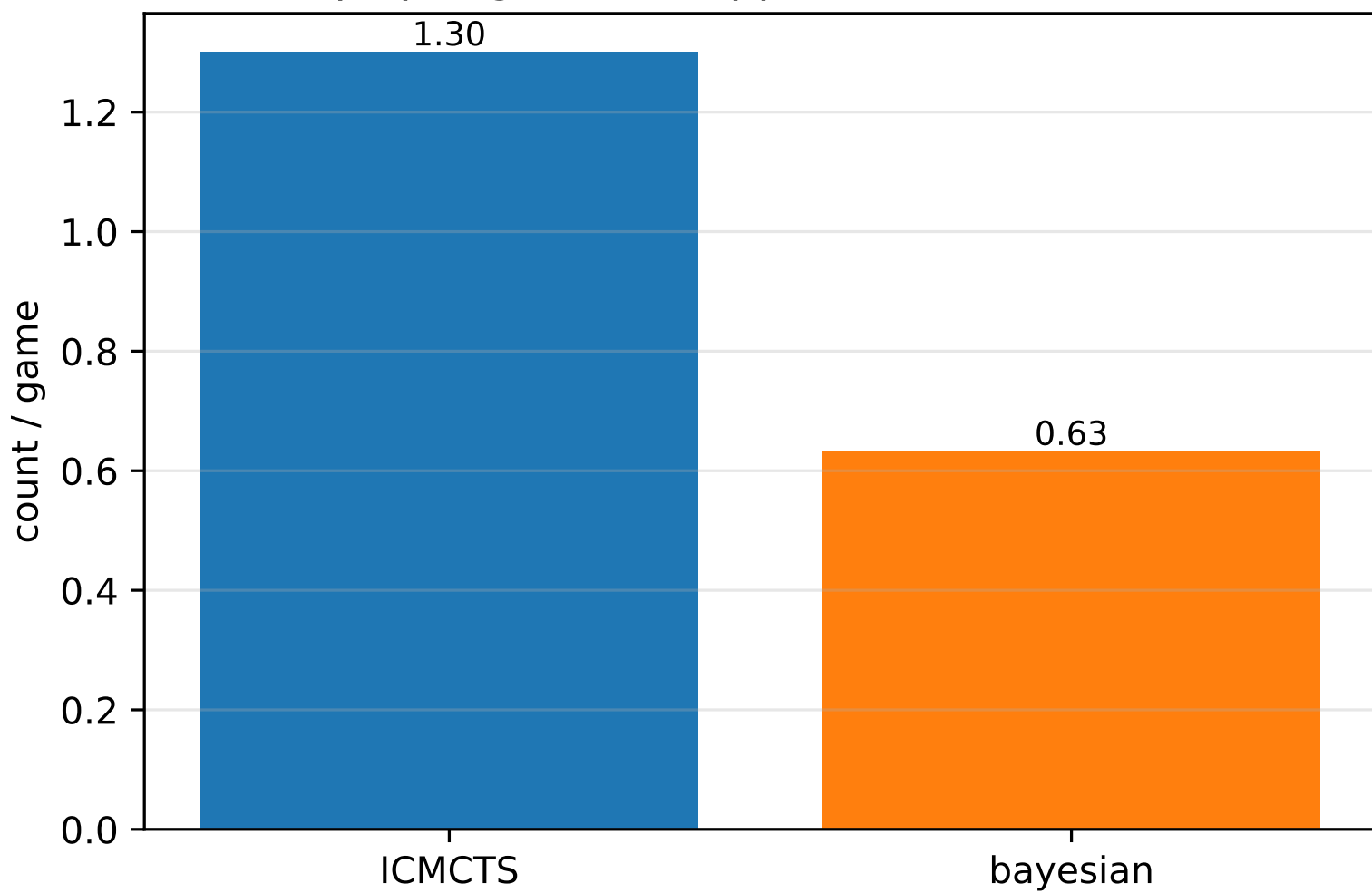




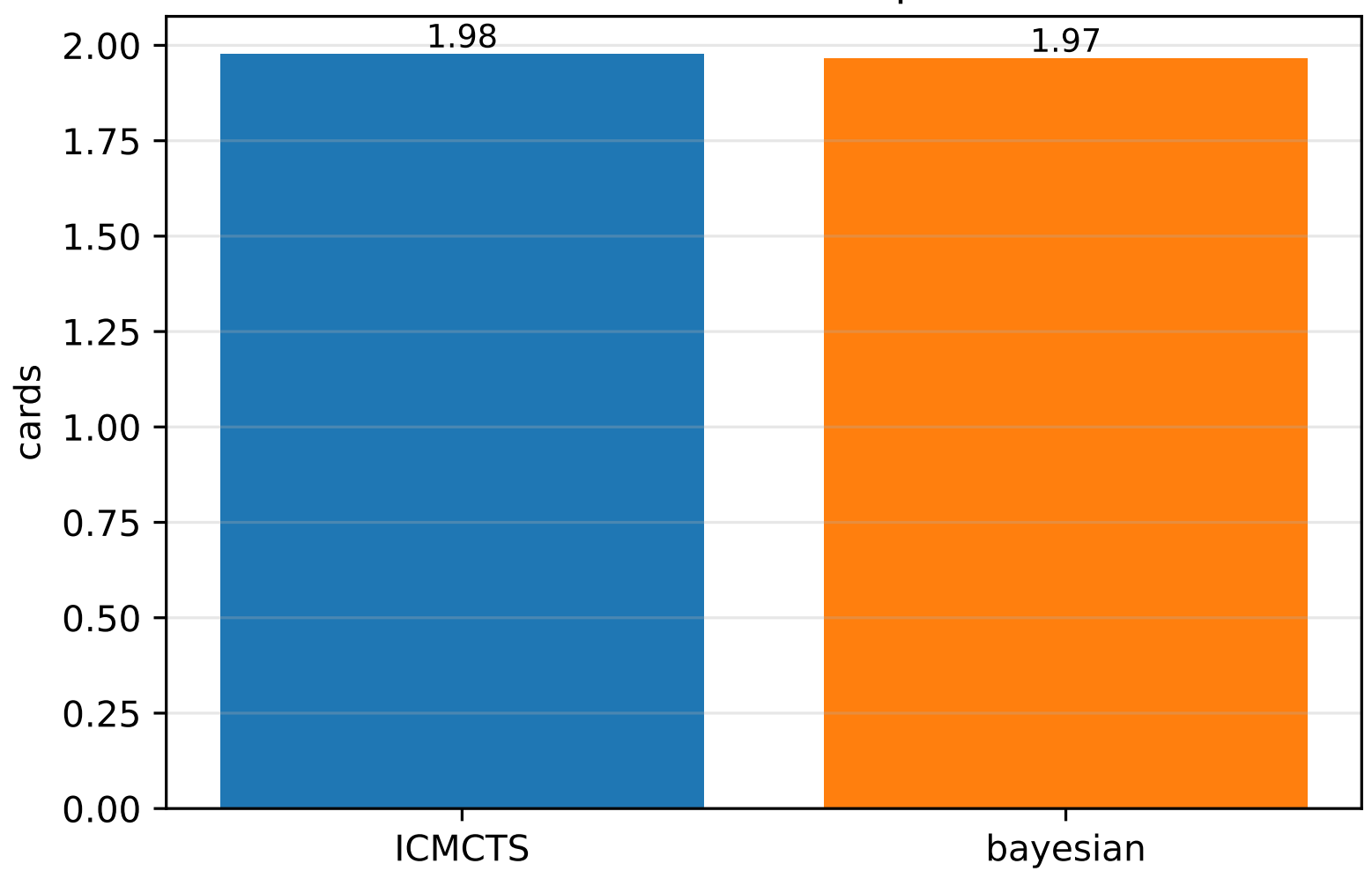
Swap quality (value delta, negative = improvement)



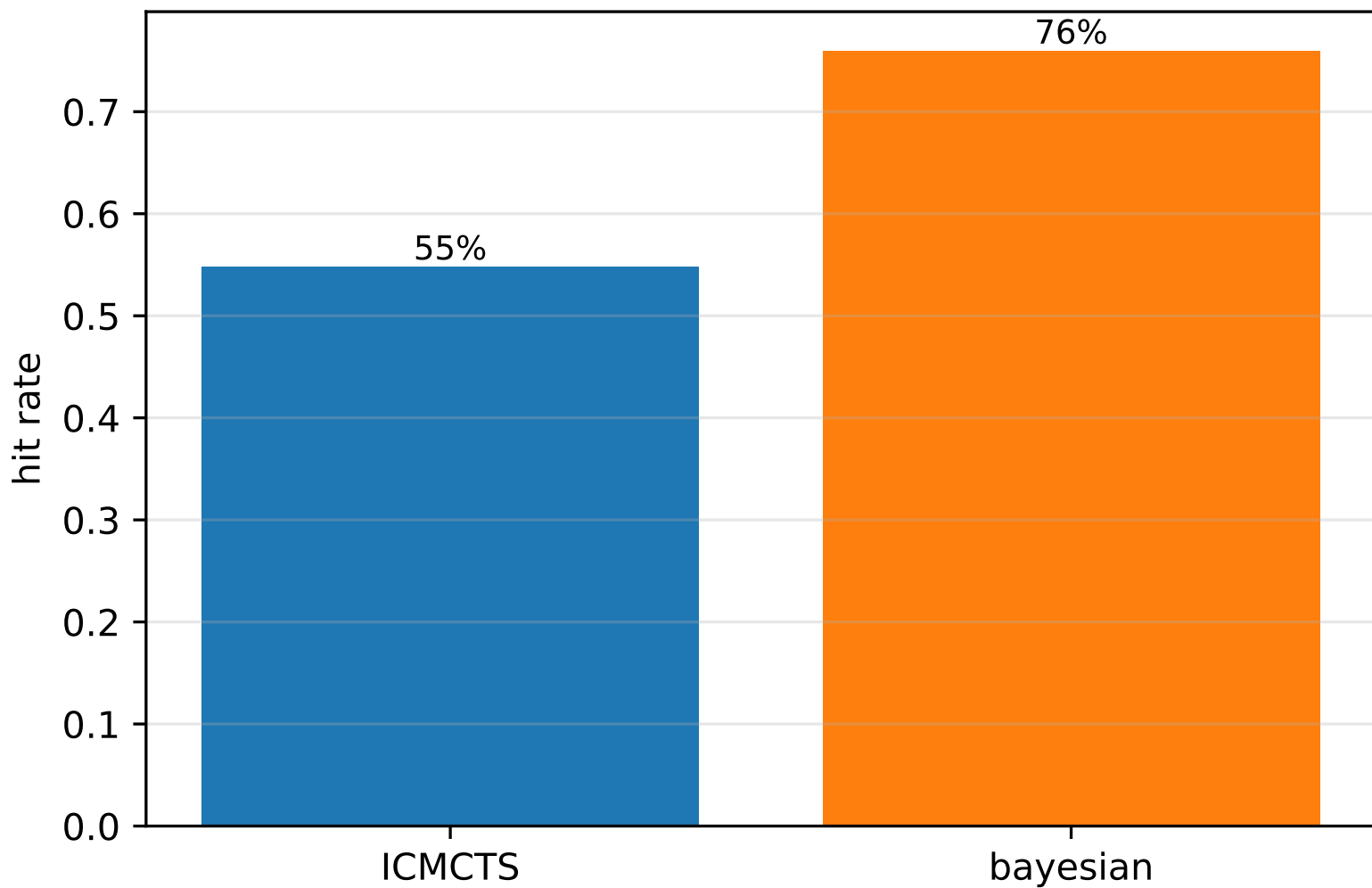
Blind swaps per game (swapped into an unidentified slot)

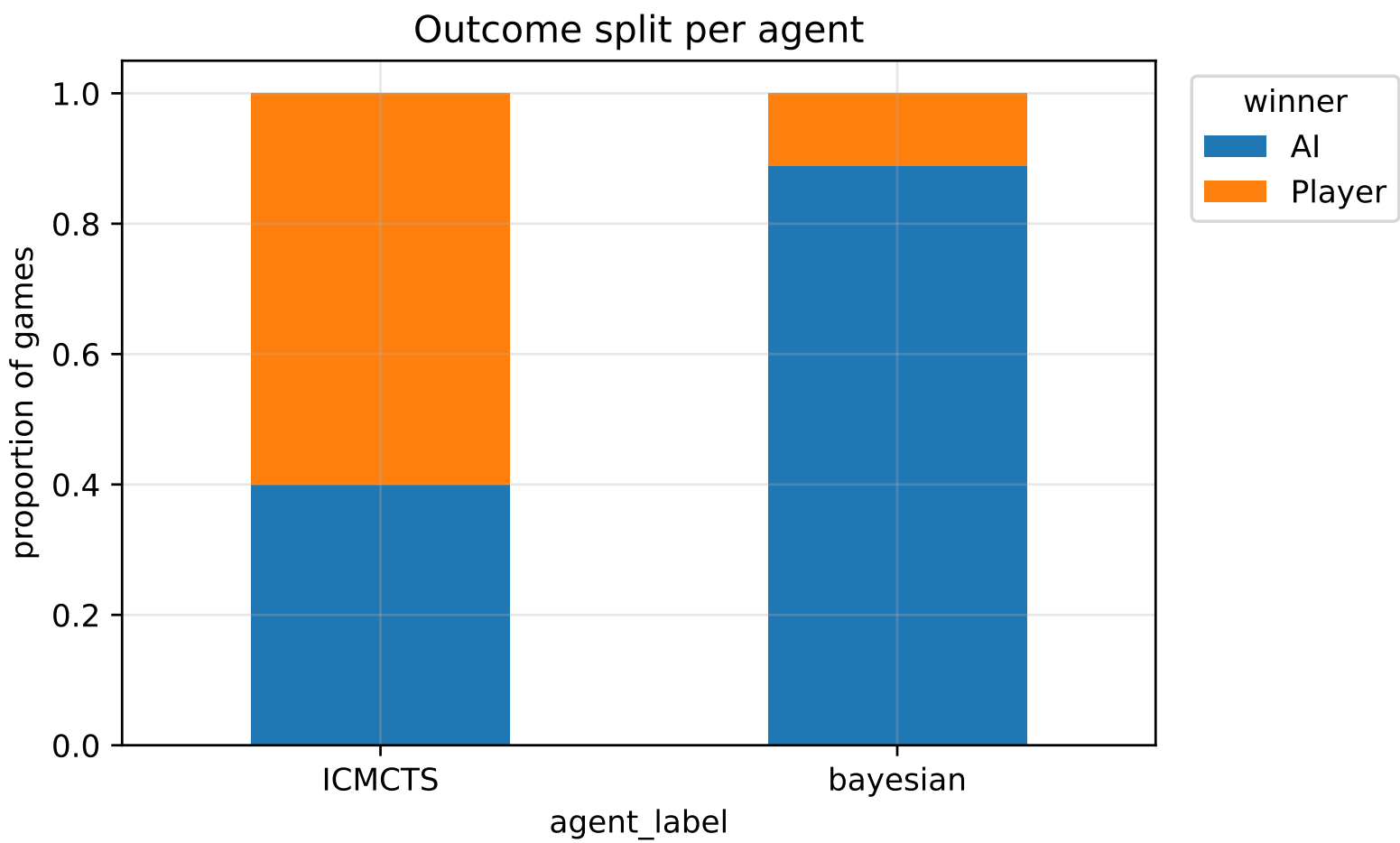


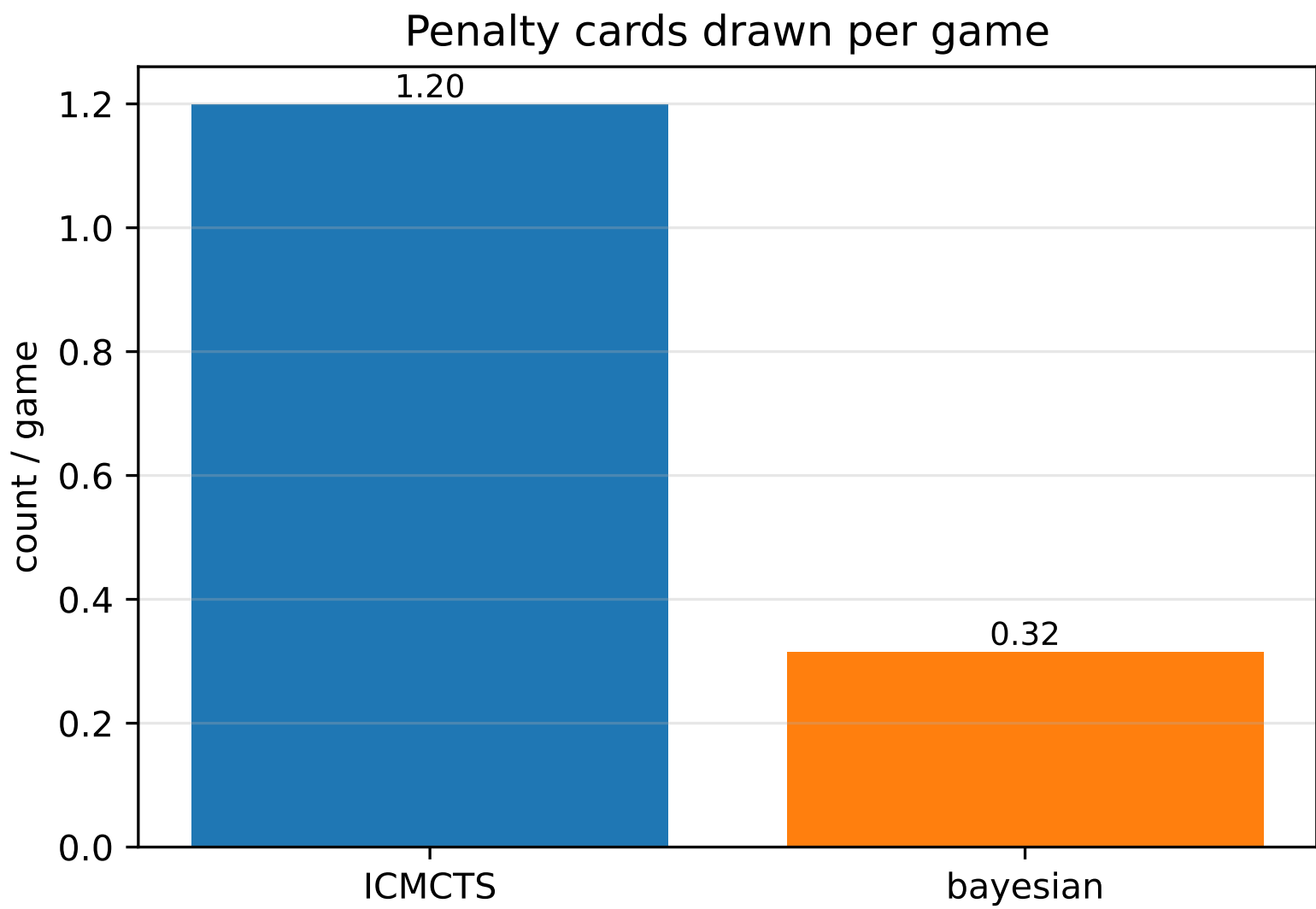
Mean own hidden cards per decision



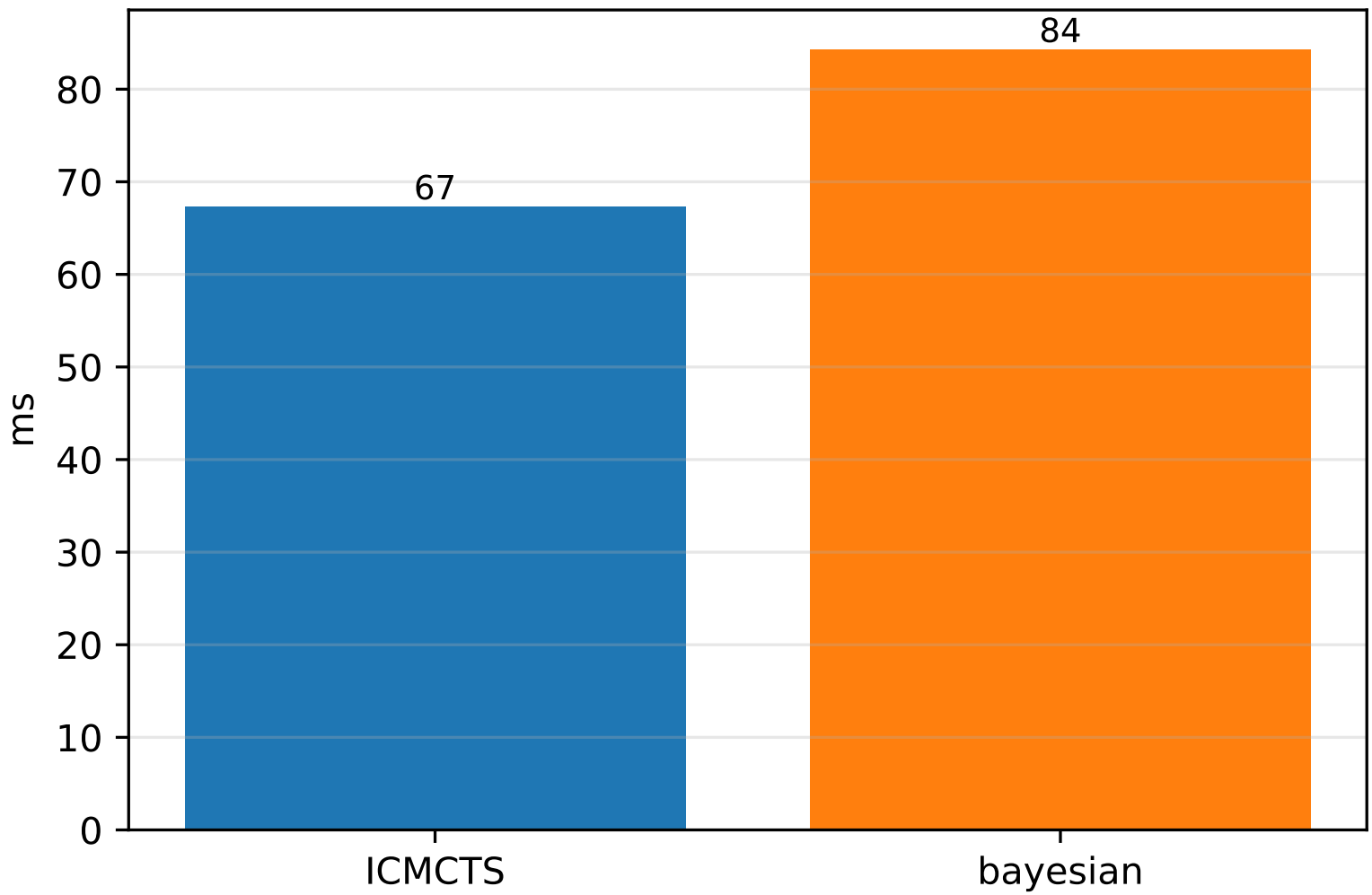
Match hit rate (successes / attempts)



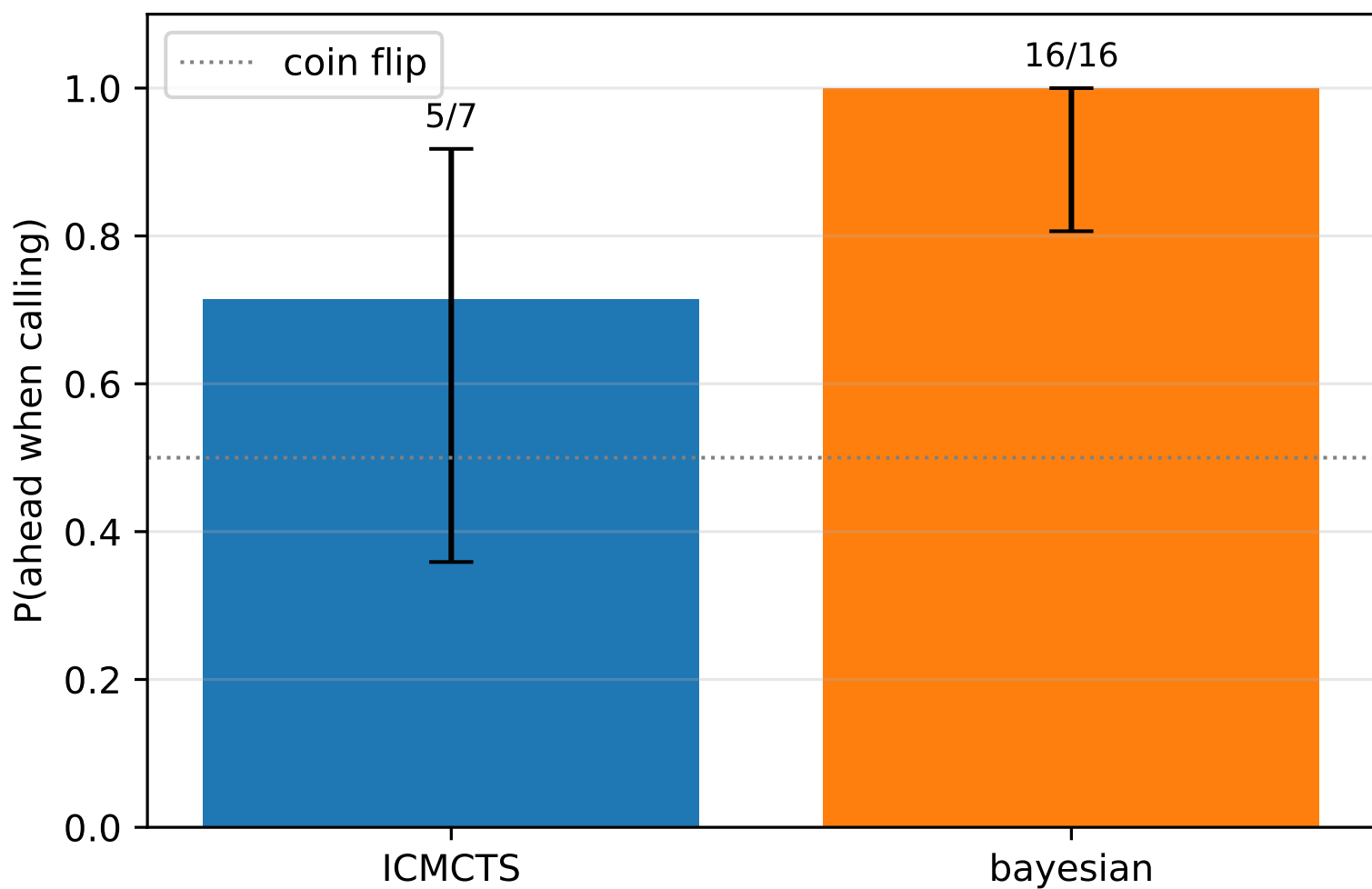




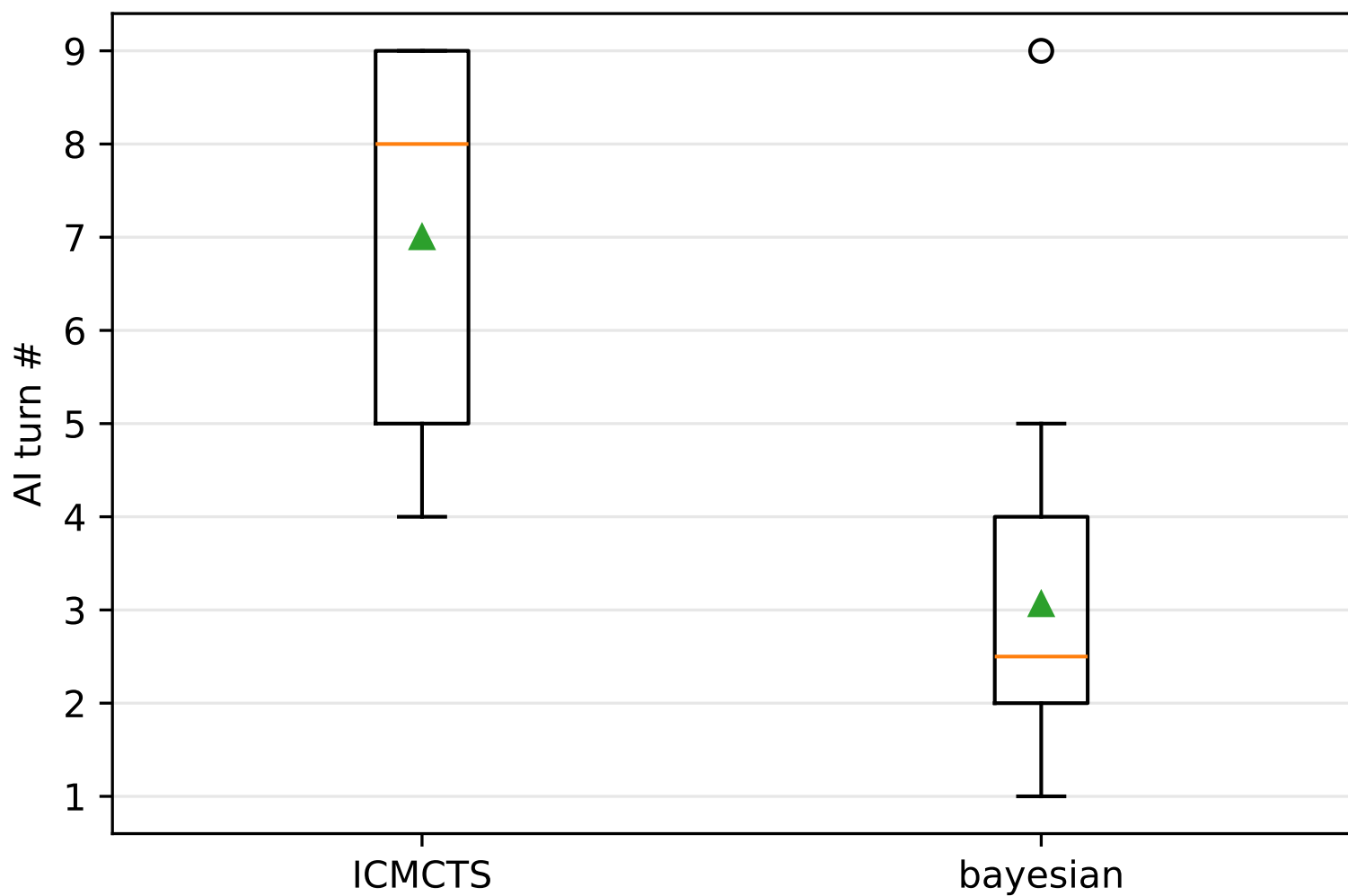
Mean AI decision time per game



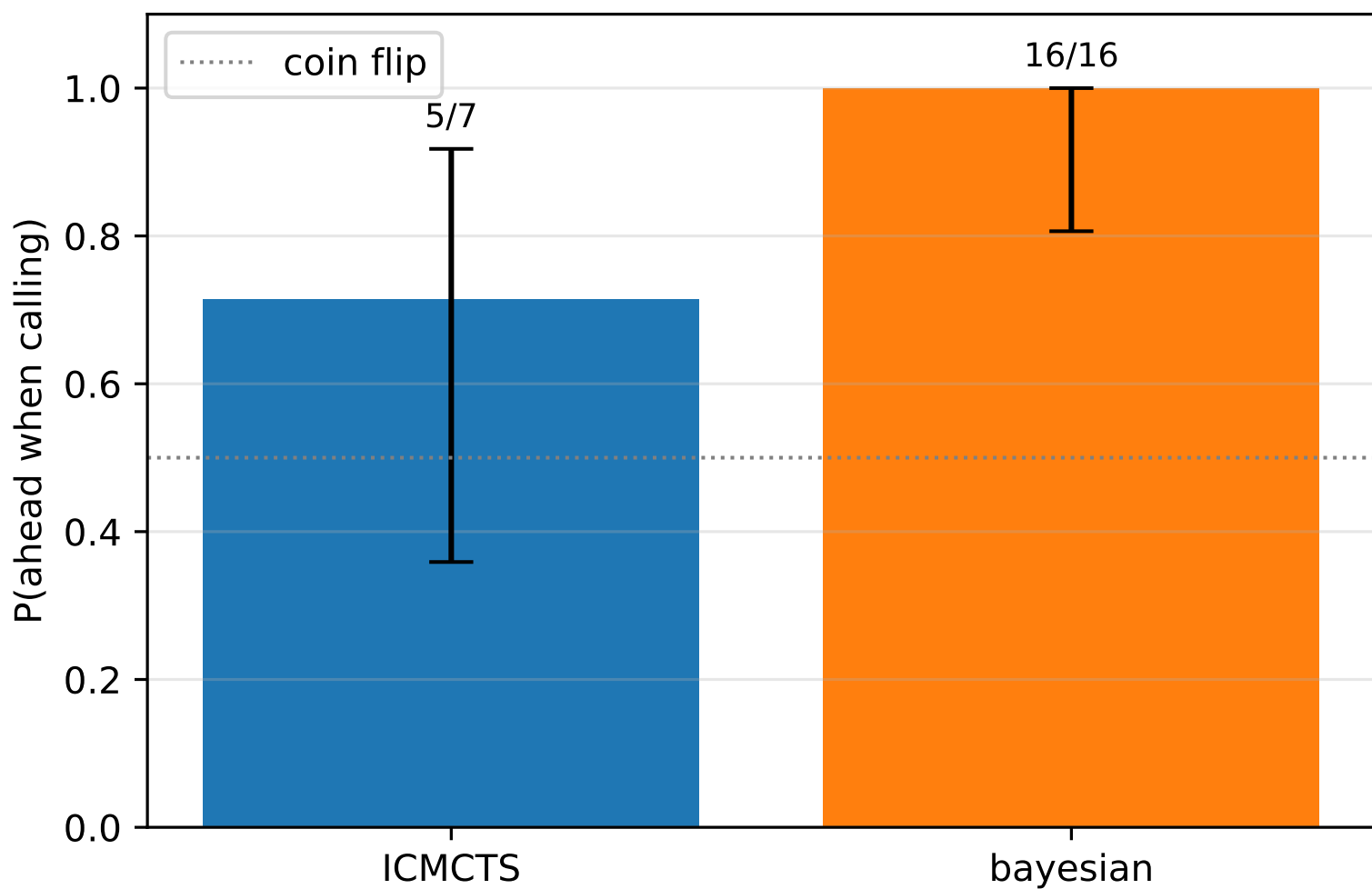
Was the AI ahead when it called Cambio? (95% Wilson CI)



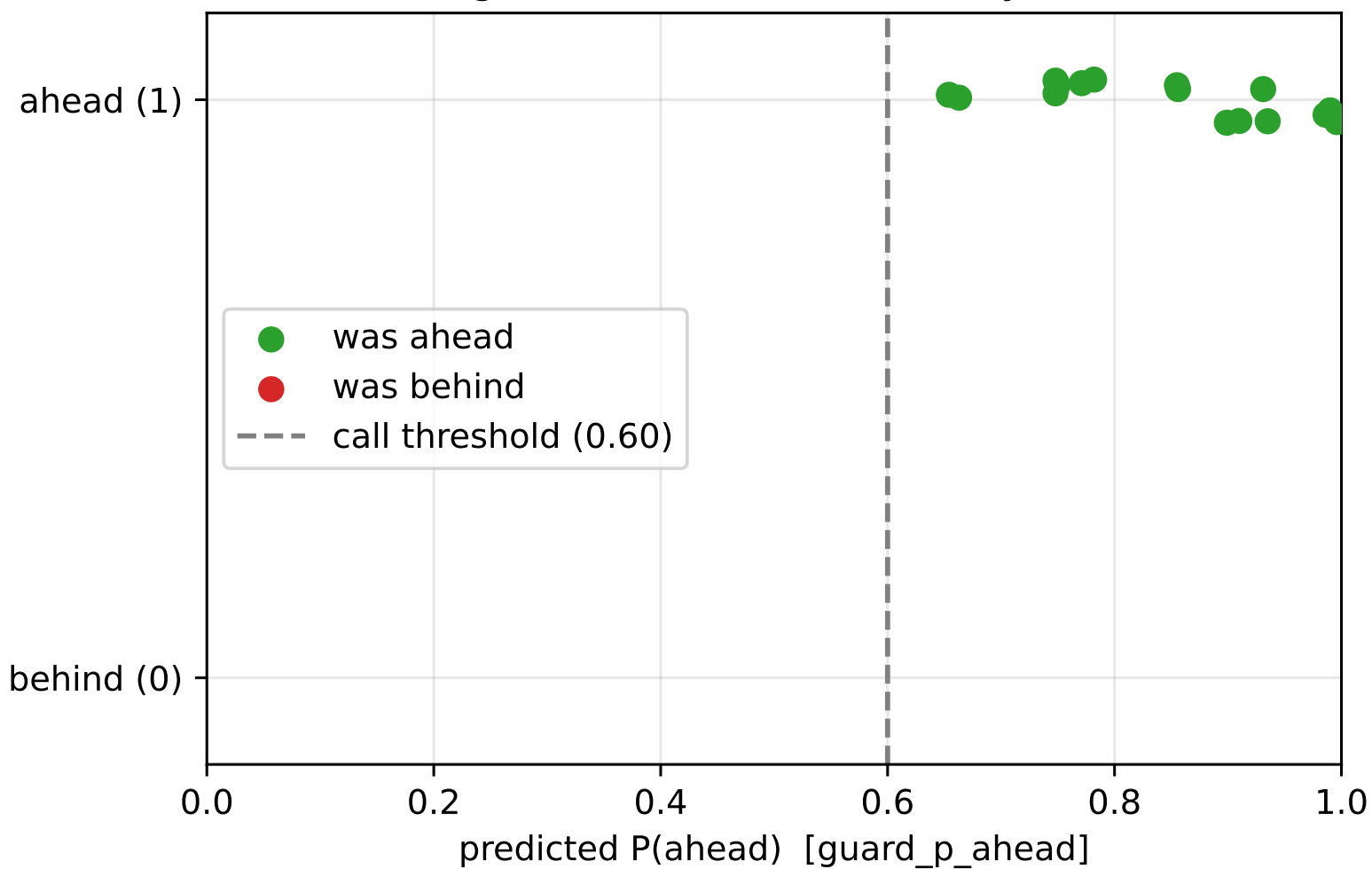
Turn on which the AI called Cambio



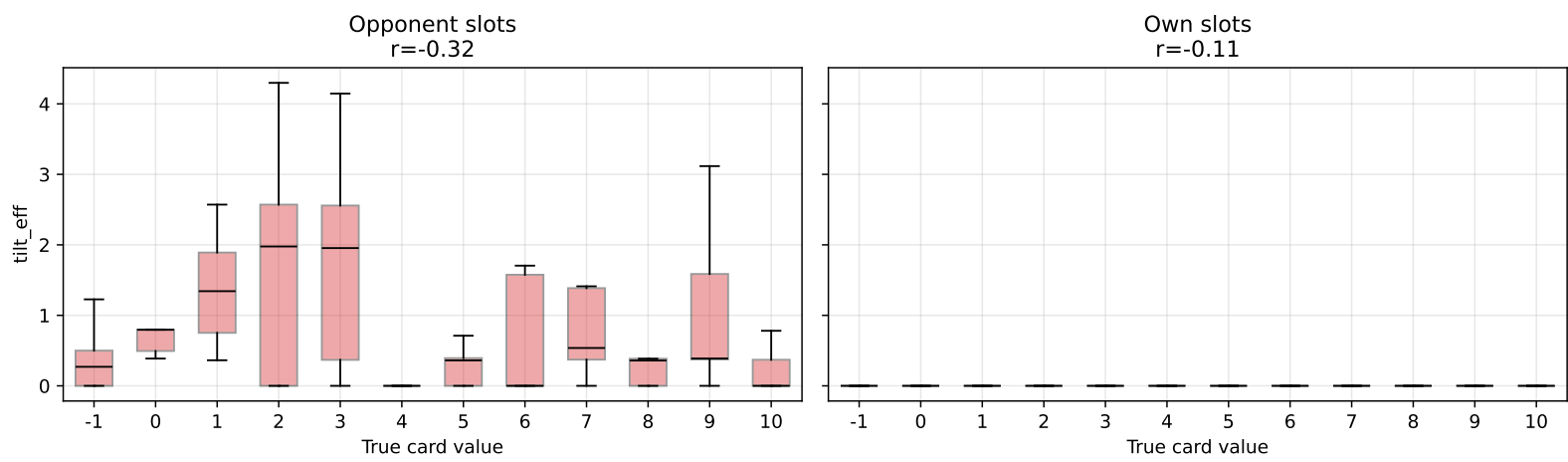
Was the AI ahead when it called Cambio? (95% Wilson CI)



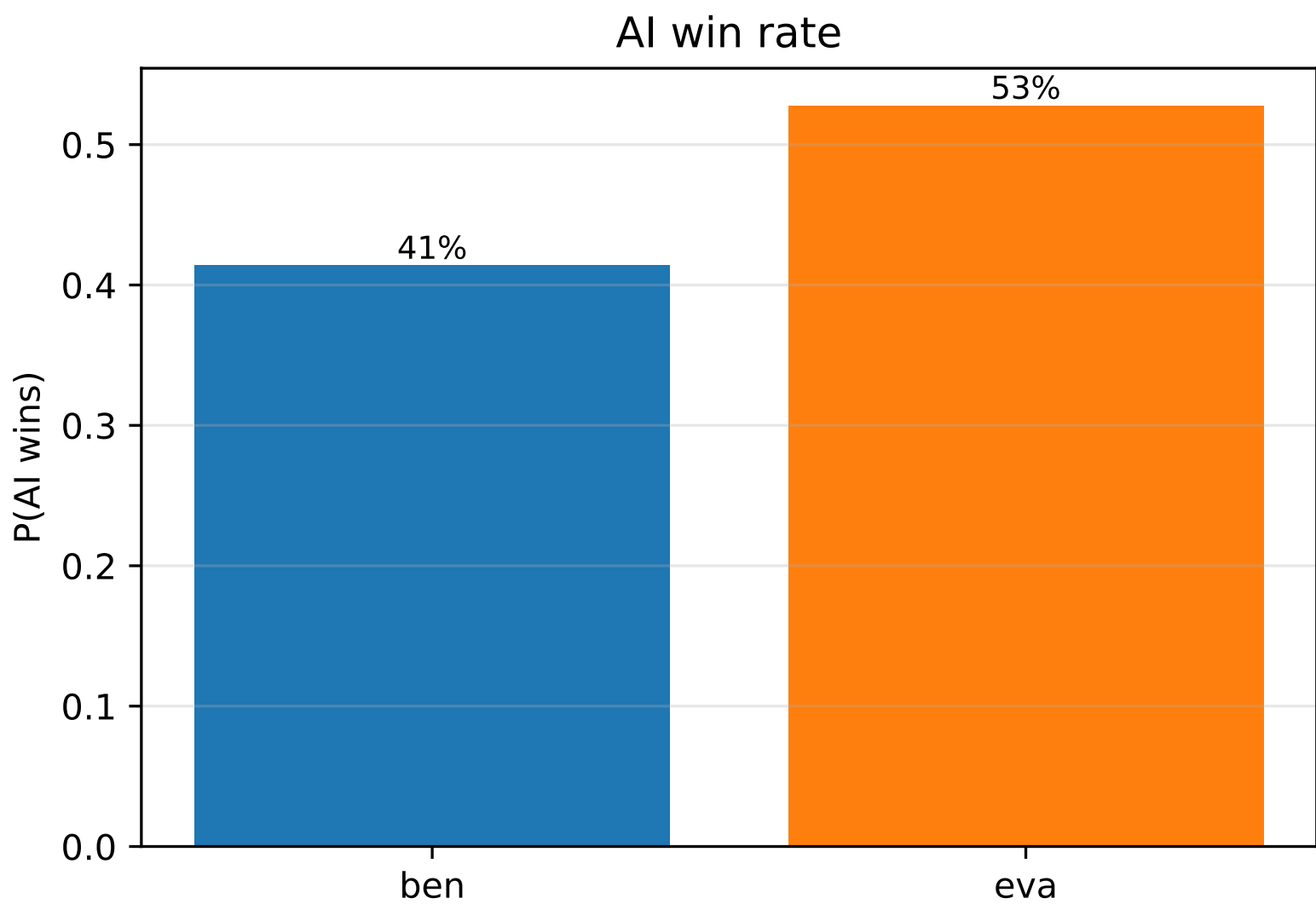
Per-call guard calibration — 16 Bayesian calls

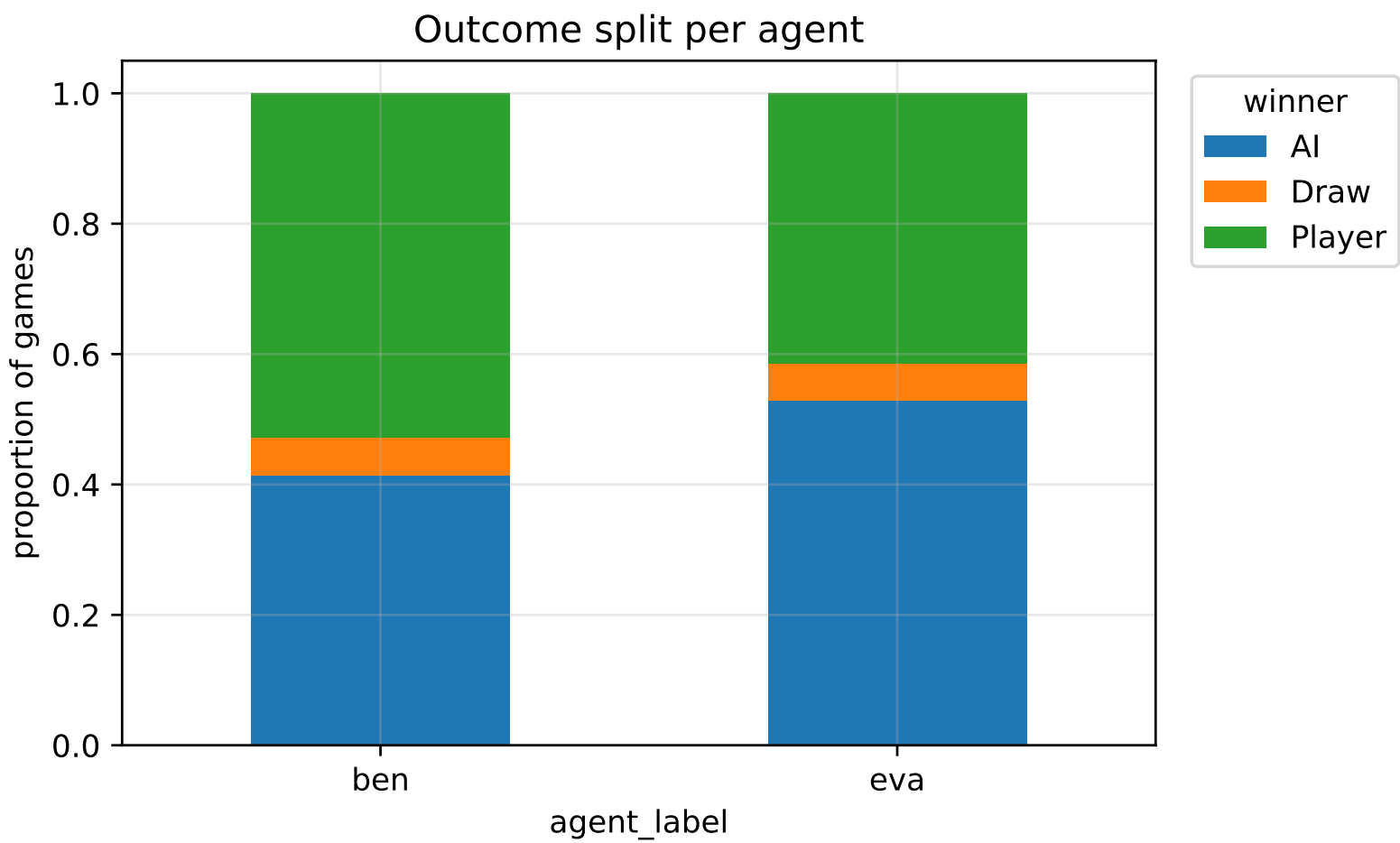


Belief tilt distribution by true card value

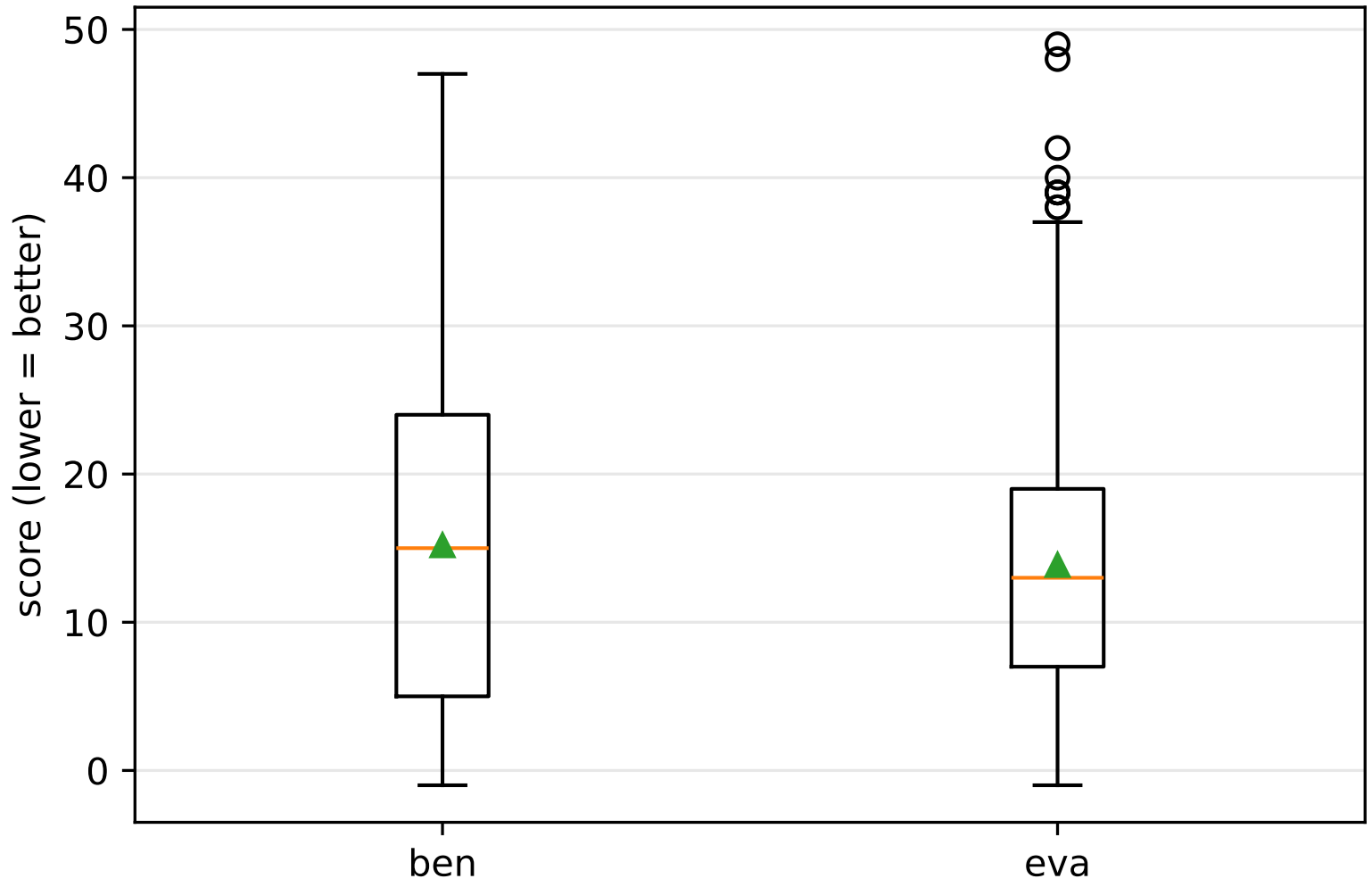


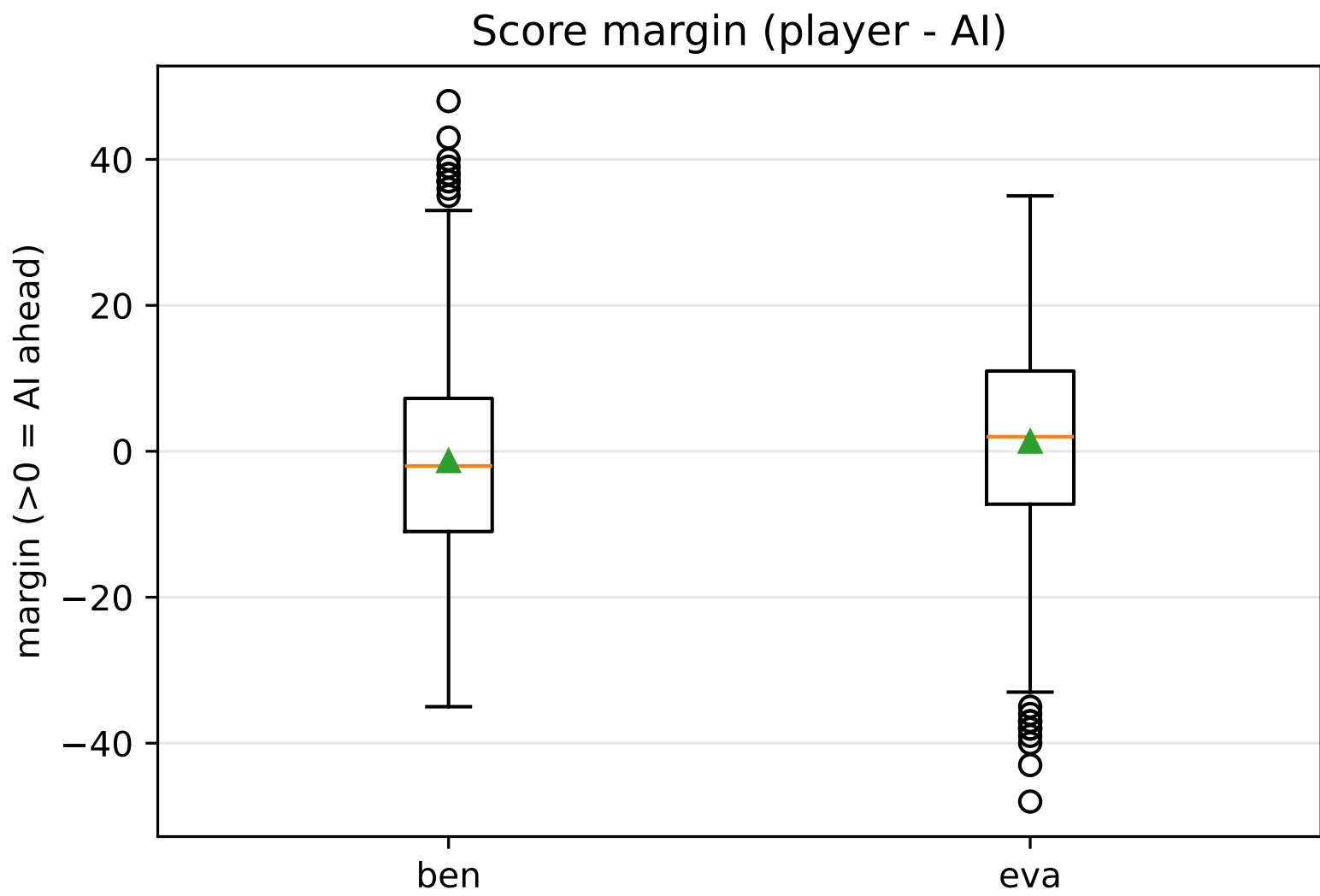
Mirror match (1,000)



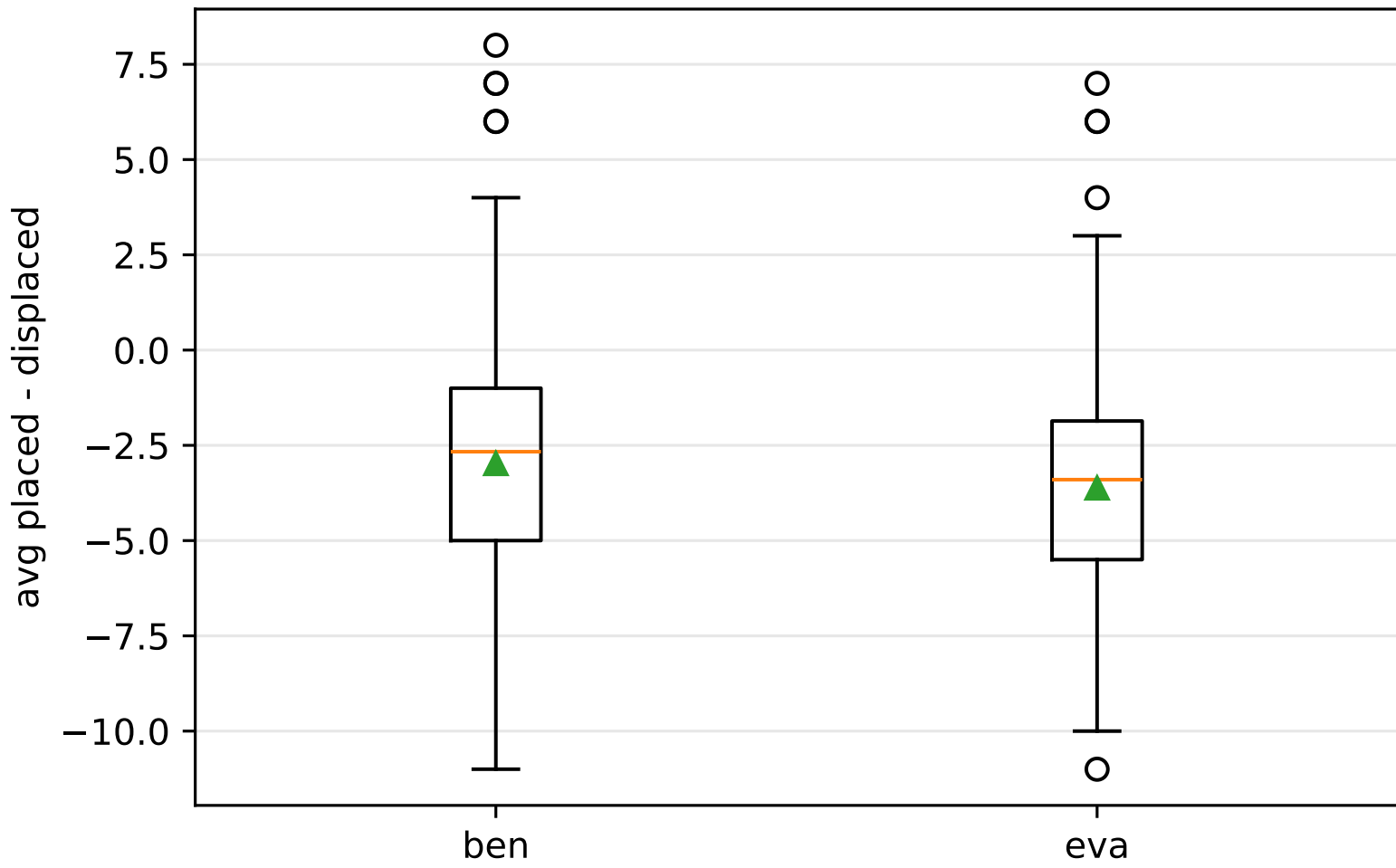


AI final hand value

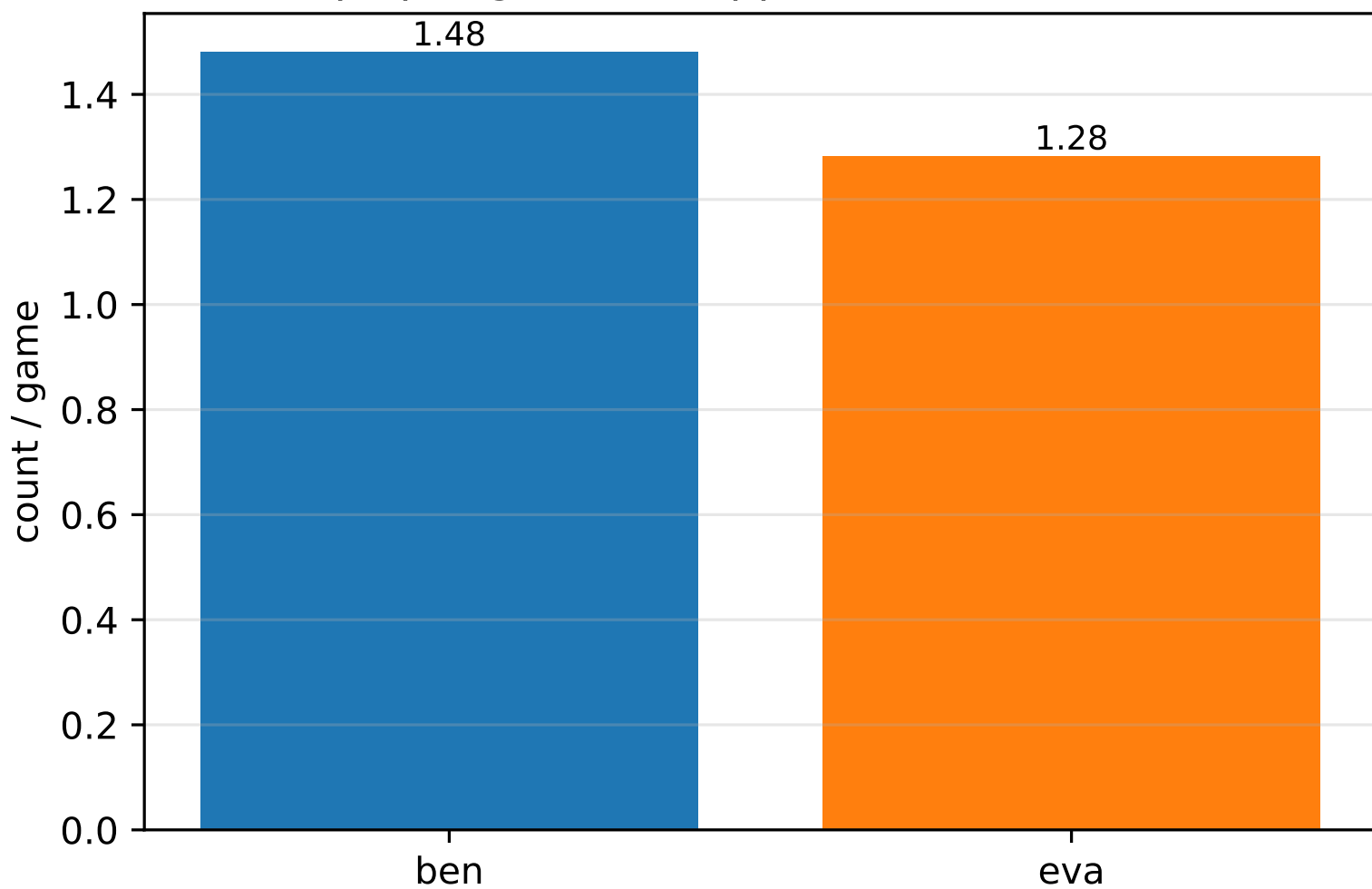




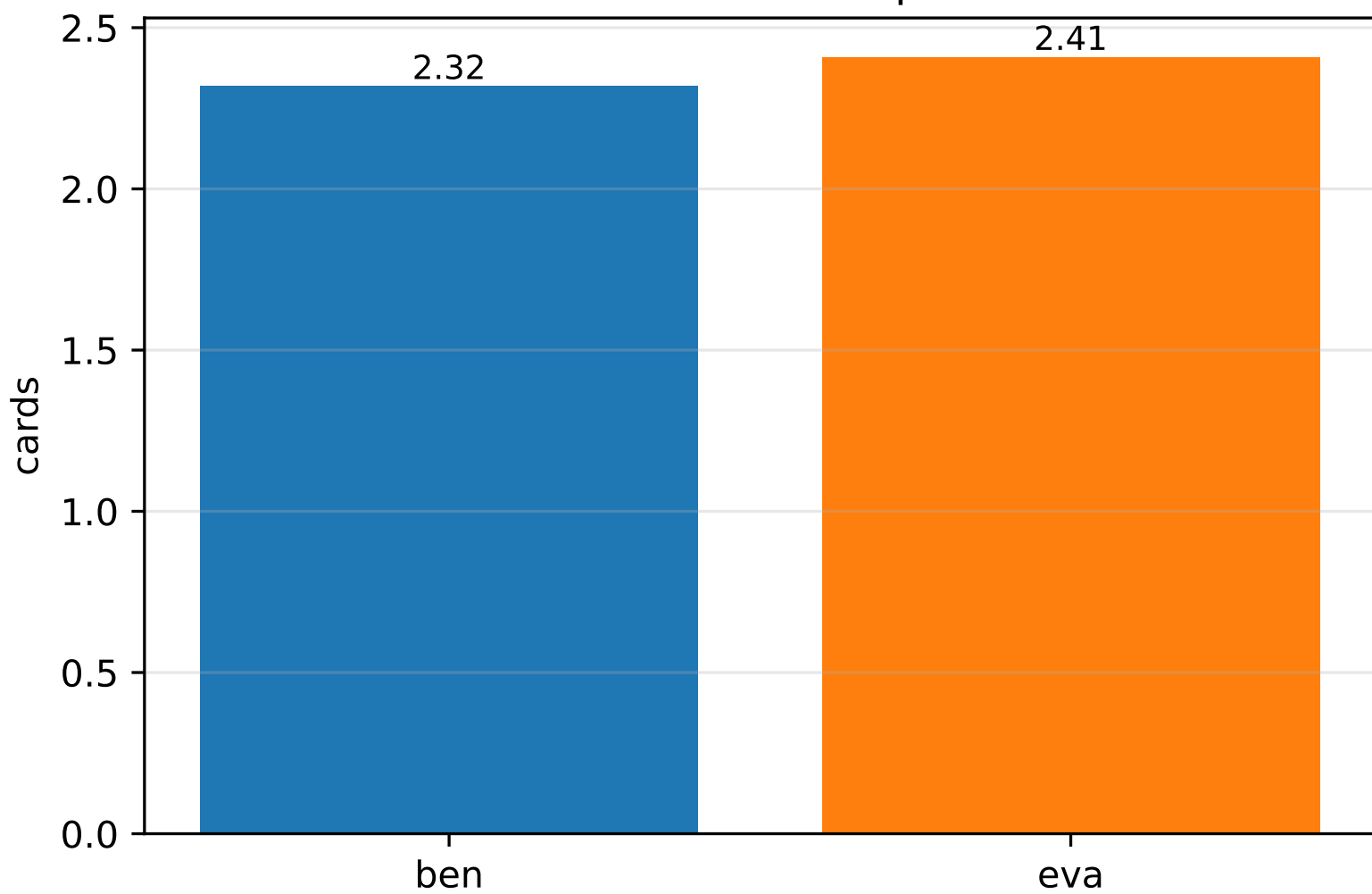
Swap quality (value delta, negative = improvement)



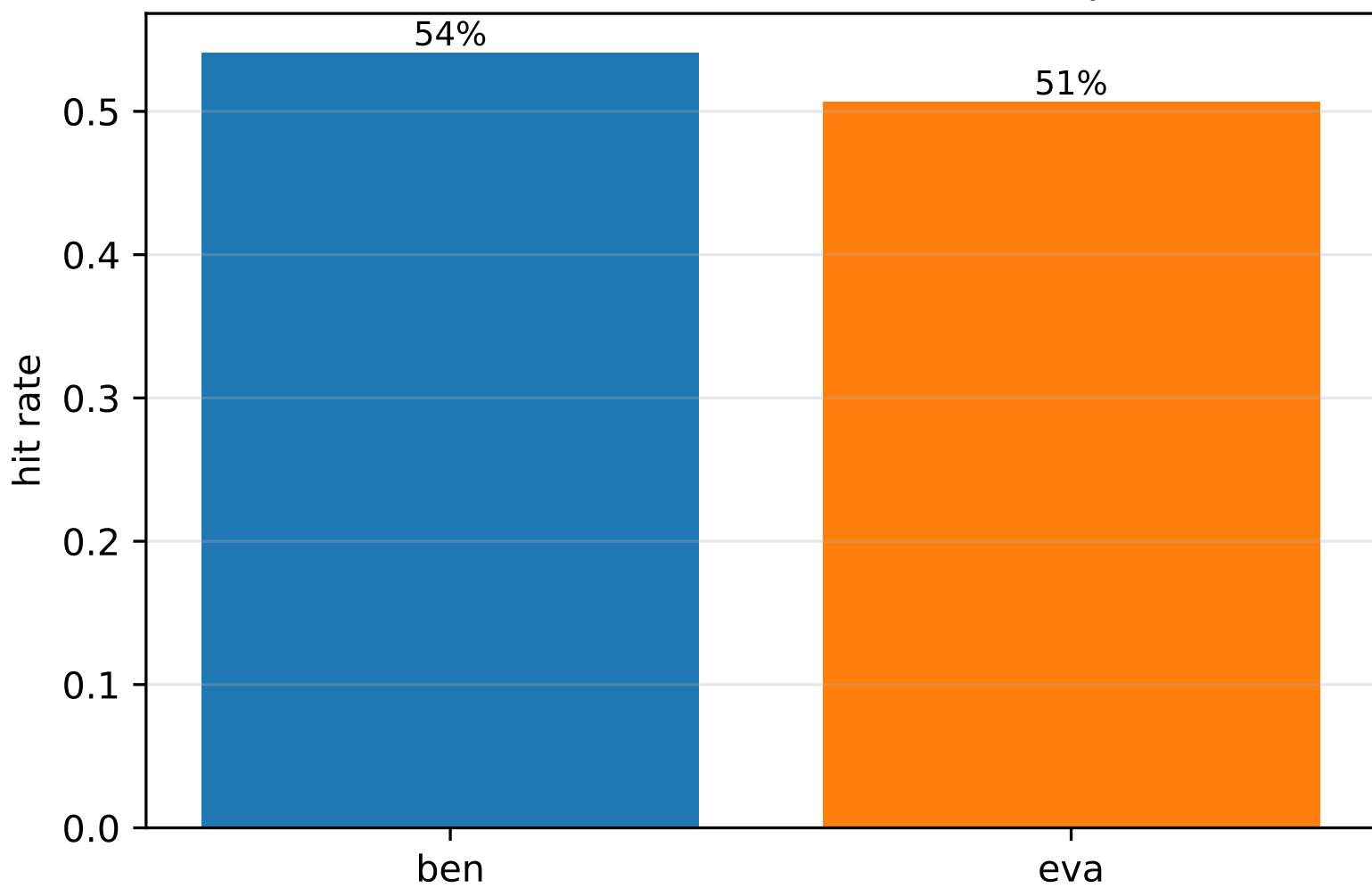
Blind swaps per game (swapped into an unidentified slot)

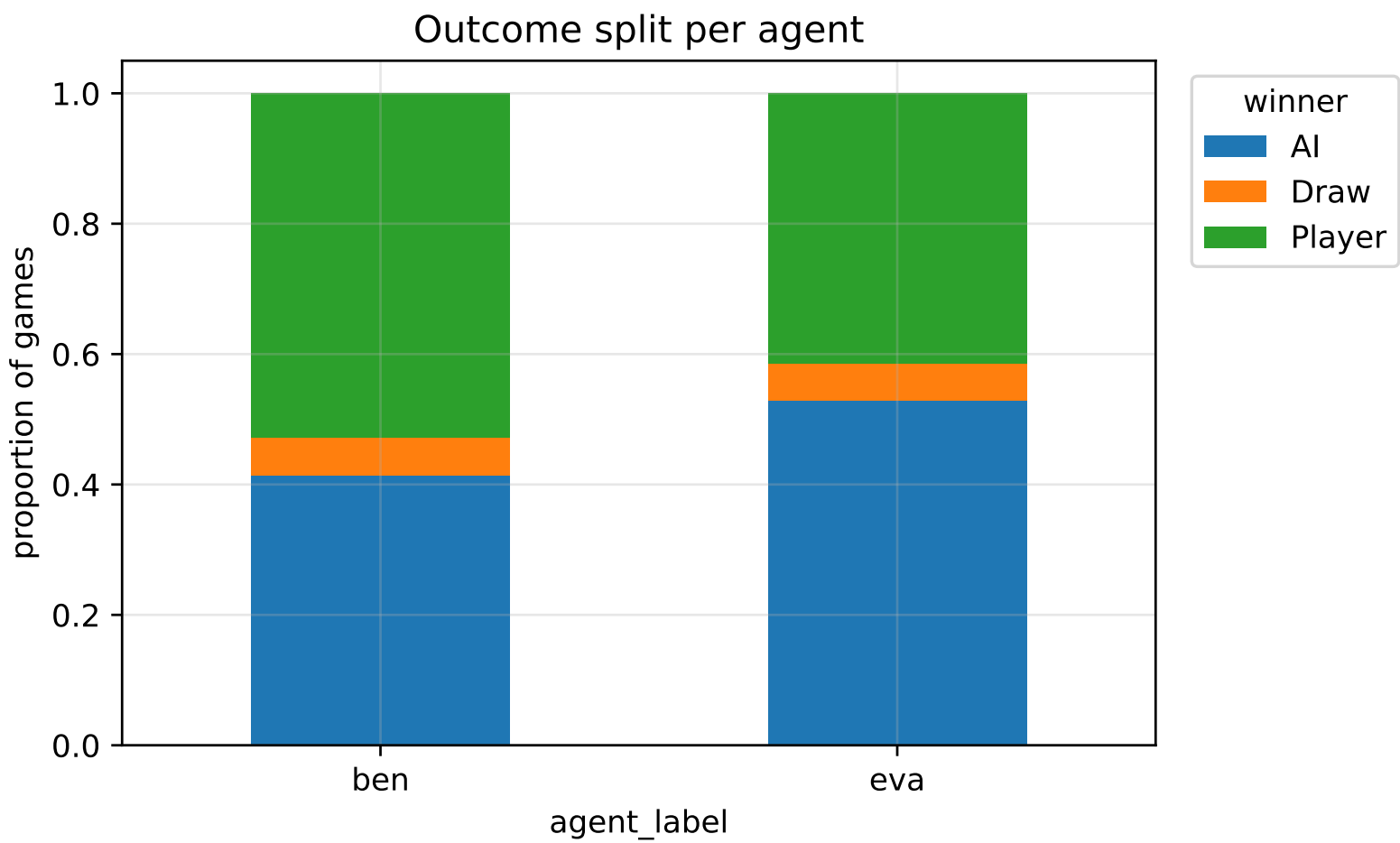


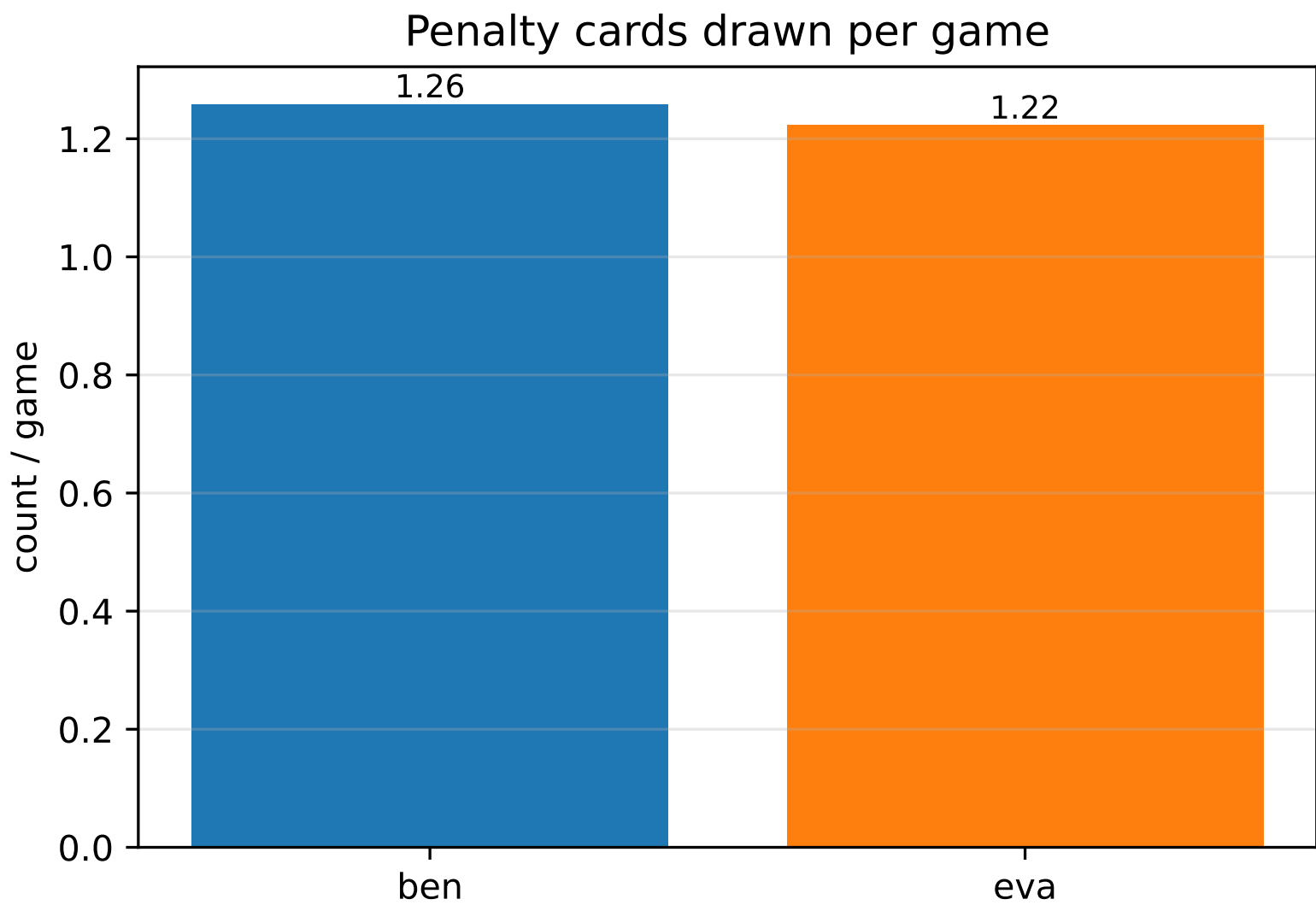
Mean own hidden cards per decision



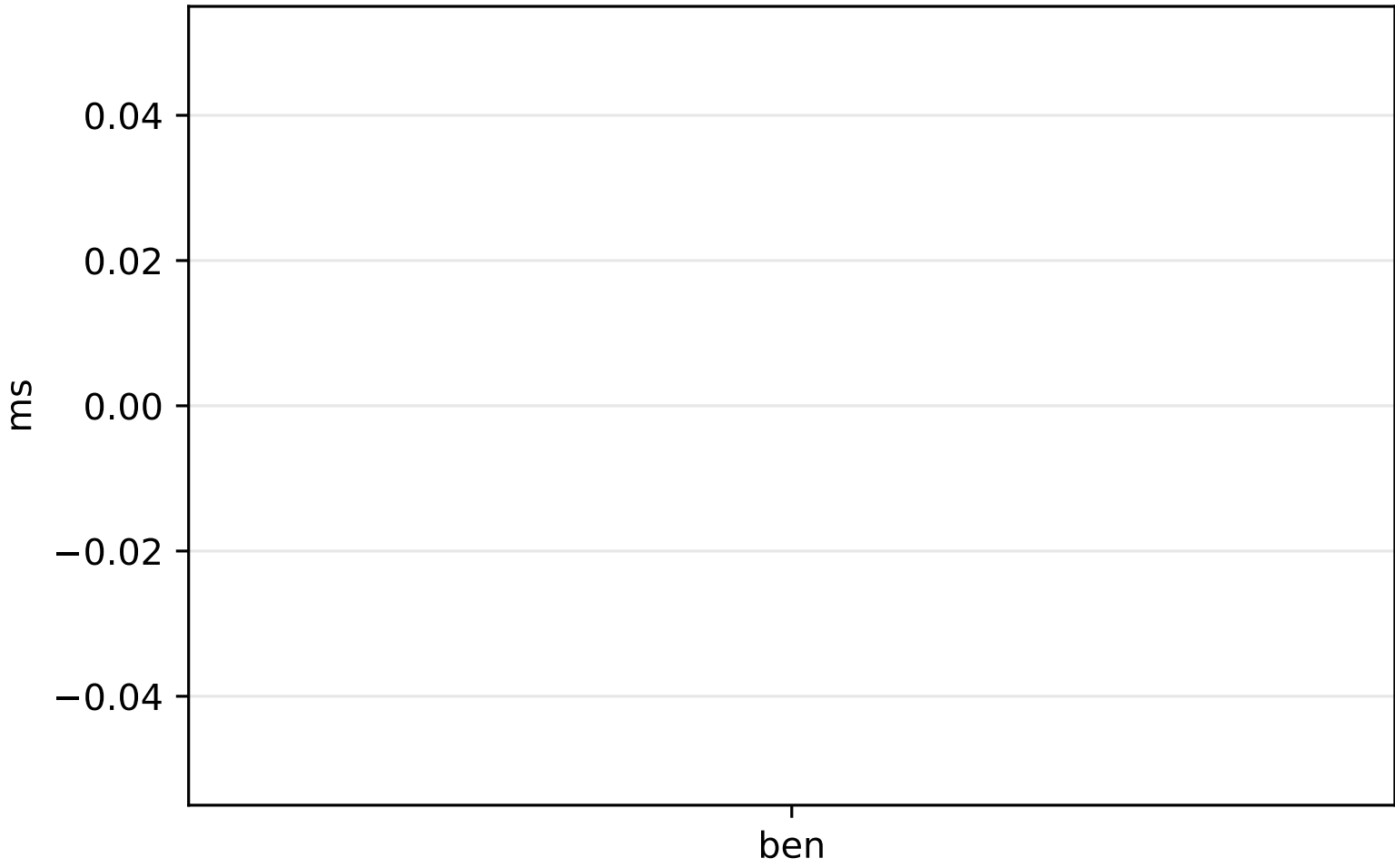
Match hit rate (successes / attempts)



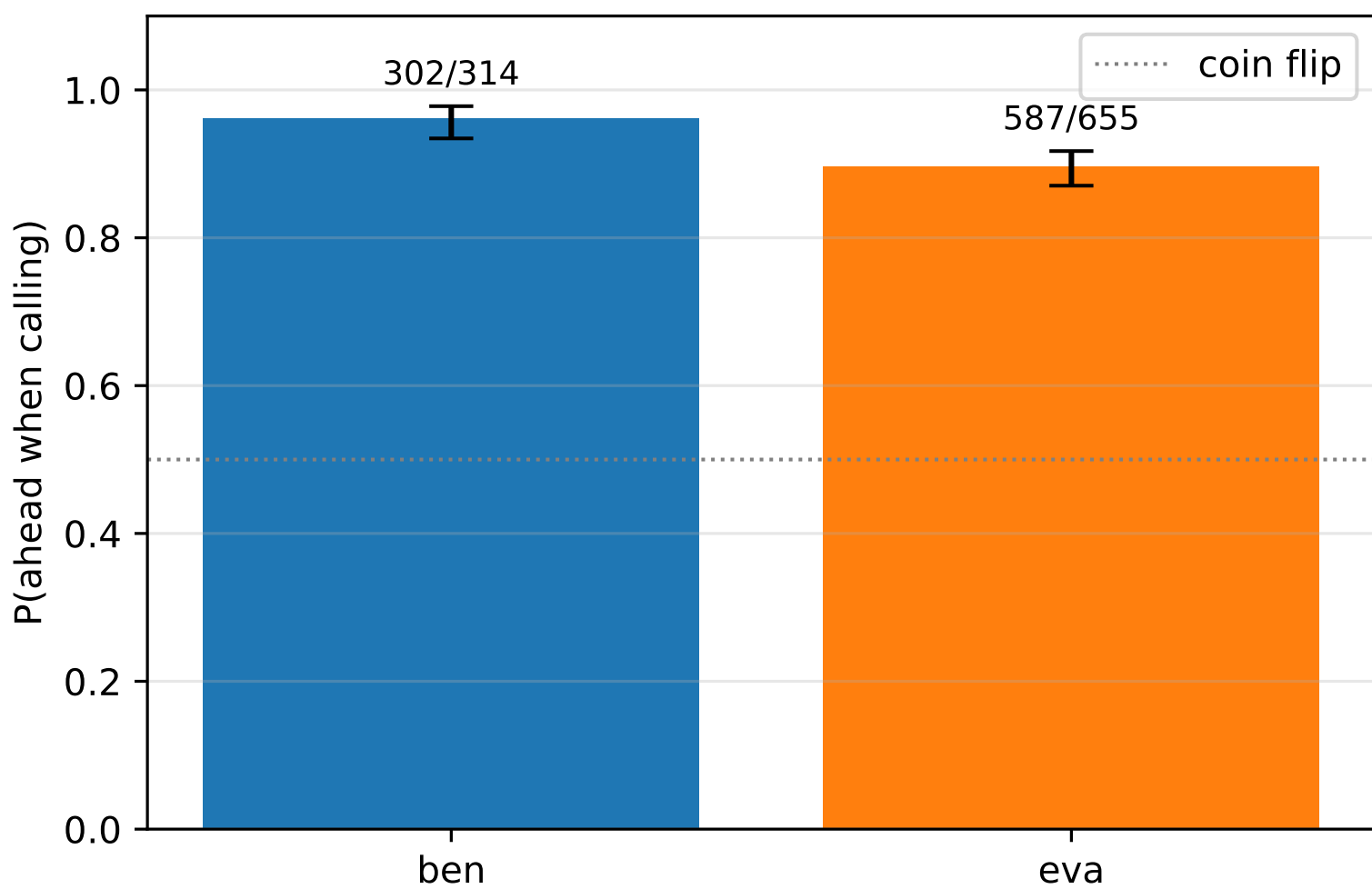




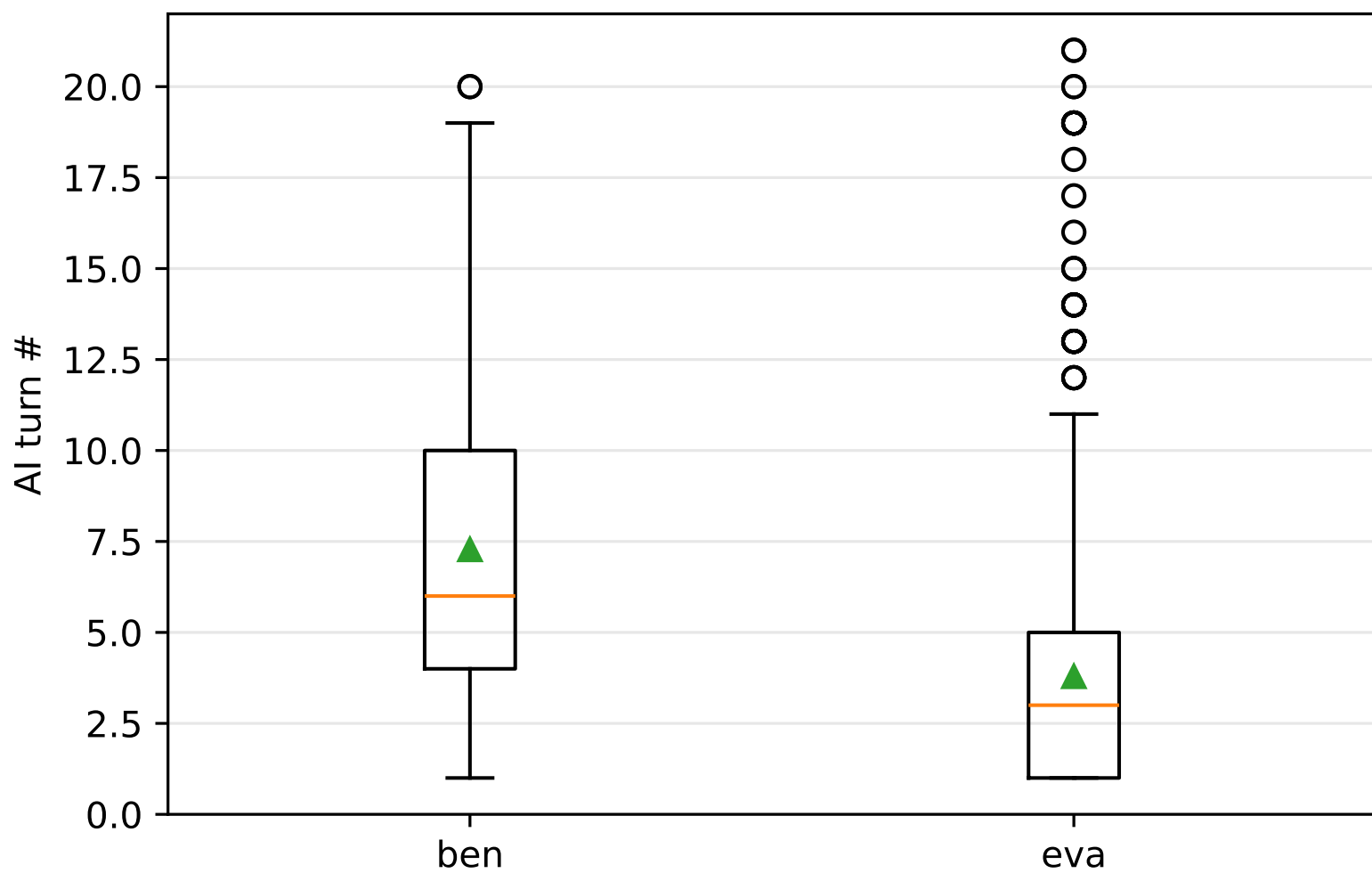
Mean AI decision time per game



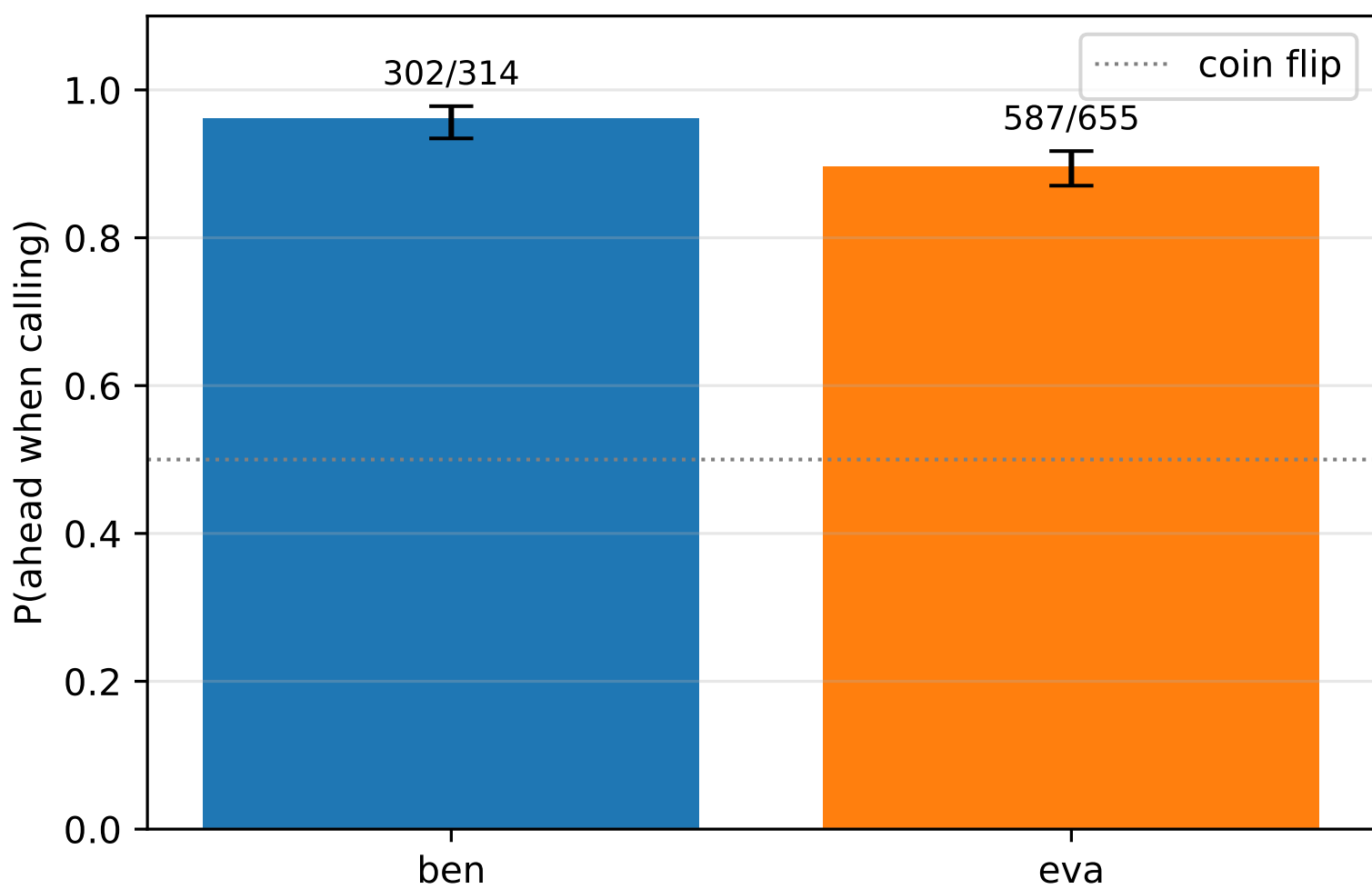
Was the AI ahead when it called Cambio? (95% Wilson CI)



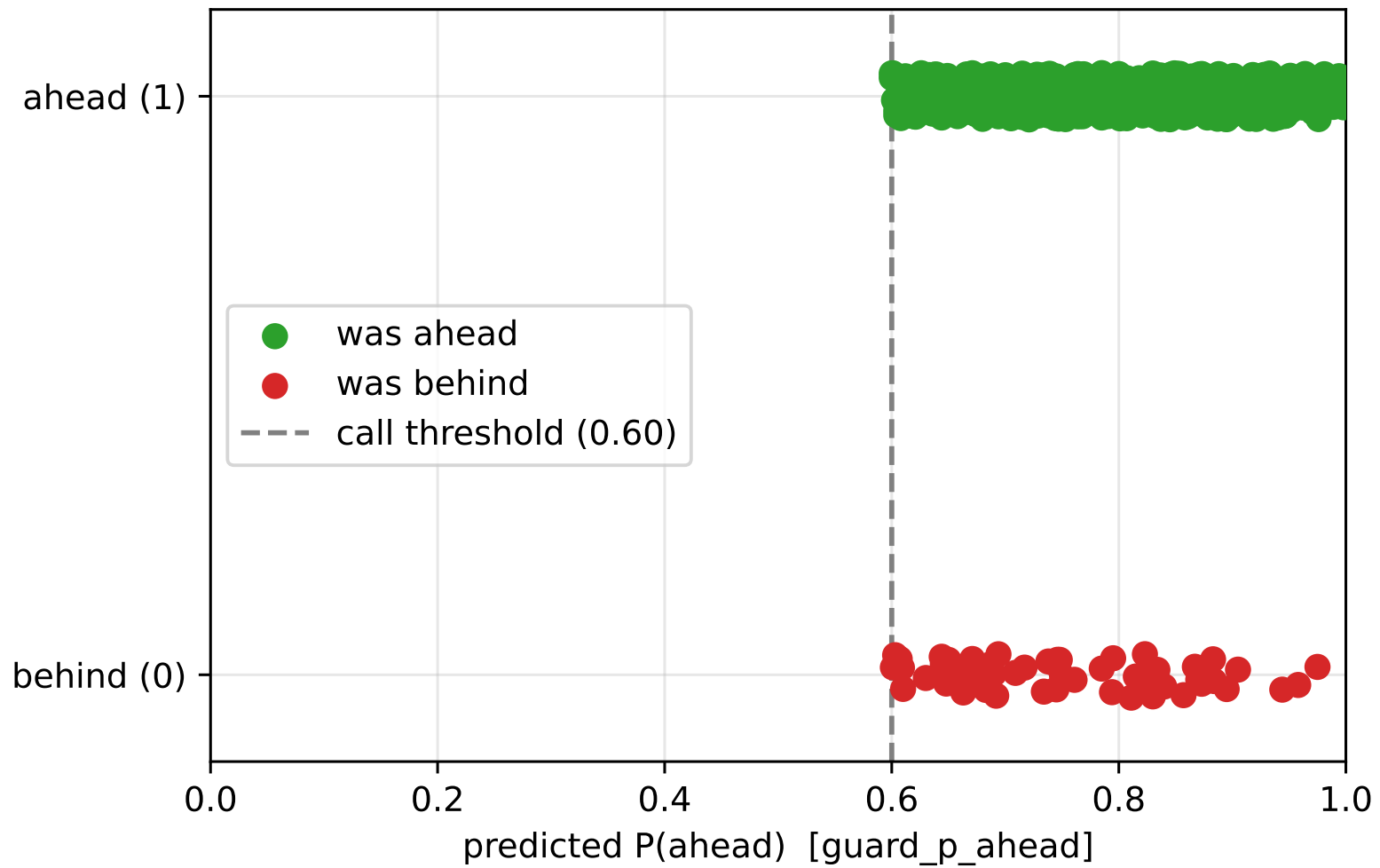
Turn on which the AI called Cambio



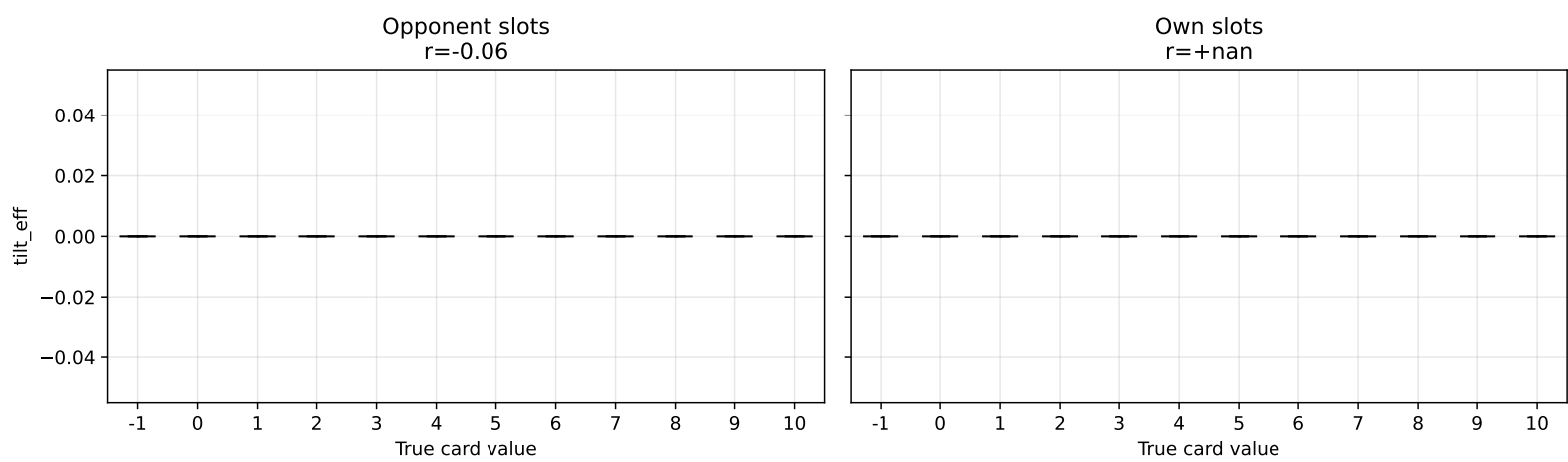
Was the AI ahead when it called Cambio? (95% Wilson CI)



Per-call guard calibration — 641 Bayesian calls



Belief tilt distribution by true card value



Appendix D: GDPR Data Management Data Sign Off Form

For undergraduate or postgraduate student projects supervised by an Abertay staff member.

This form **MUST** be included in the student's thesis/dissertation. Note that failure to do this will mean that the student's project cannot be assessed/examined.

Part 1: Supervisors to Complete

By signing this form, you are confirming that you have checked and verified your student's data according to the criteria stated below (e.g., raw data, completed questionnaires, superlab/Eprime output, transcriptions etc.)

Student Name:	JOSÉ PABLO GONZÁLEZ POBLETTE		
Student Number:	2507739		
Lead Supervisor Name:	Jagdeep Singh Ahluwalia		
Lead Supervisor Signature	<i>Jagdeep Singh Ahluwalia</i>		
Project title:	Adaptive Difficulty Through Bayesian Opponent Modelling in ISMCTS-Based Card Game Agents		
Study route:	PhD ☐	MbR ☐	MPhil ☐
	Undergraduate ☐	Postgraduate taught x	PhD by Publication ☐

Part 2: Student to Complete

	Initial here to confirm 'Yes'
I confirm that I have handed over all manual records from my research project (e.g., consent forms, transcripts) to my supervisor for archiving/storage	Yes
I confirm that I have handed over all digital records from my research project (e.g., recordings, data files) to my supervisor for archiving/storage	Yes
I confirm that I no longer hold any digital records from my research project on any device other than the university network and the only data that I may retain is a copy of an anonymised data file(s) from my research	Yes
I understand that, for undergraduate projects, my supervisor may delete manual/digital records of data if there is no foreseeable use for that data (with the exception of consent forms, which should be retained for 10 years)	Yes

A handwritten signature in black ink, featuring a large, stylized 'G' and 'P' with a vertical line through the 'P'.

Student signature: _____

Date: 24/08/2026